

Modelado generativo de cadenas laterales de proteínas mediante difusión torsional en espacios no euclidianos



Universitat
Oberta
de Catalunya



UNIVERSITAT DE
BARCELONA

Noelia Fernández Espiñeira

Máster Universitario:
Bioinformática y Bioestadística

Área:
Diseño de Fármacos y Biología Estructural

Tutor:
Brian Jiménez García

Profesor/a responsable de la asignatura:
Anna Garcia-Elias

3 de junio de 2026

© (Noelia Fernández Espiñeira)

Reservados todos los derechos. Está prohibido la reproducción total o parcial de esta obra por cualquier medio o procedimiento, comprendidos la impresión, la reprografía, el microfilme, el tratamiento informático o cualquier otro sistema, así como la distribución de ejemplares mediante alquiler y préstamo, sin la autorización escrita del autor o de los límites que autorice la Ley de Propiedad Intelectual.

FICHA DEL TRABAJO FINAL

Título del trabajo:	Modelado generativo de cadenas laterales de proteínas mediante difusión torsional en espacios no euclidianos
Nombre del autor:	Noelia Fernández Espiñeira
Nombre del director/a:	Brian Jiménez García
Nombre del PRA:	Anna Garcia-Elias
Fecha de entrega:	06/2026
Titulación o programa:	Máster Universitario en Bioinformática y Bioestadística
Área del Trabajo Final:	Diseño de fármacos y biología estructural
Idioma del trabajo:	Castellano
Palabras clave	Difusión Torsional, Estadística circular, Distribución Von Mises

Resumen del Trabajo

El modelado preciso de las cadenas laterales de los aminoácidos es una limitación crítica en biología estructural computacional: aunque herramientas como AlphaFold2 predicen el *backbone* con alta fidelidad, la posición de los ángulos diedros χ_1 – χ_4 sigue siendo imprecisa, comprometiendo el diseño de fármacos y la comprensión de la reactividad molecular.

Este trabajo desarrolla **ChiGNN**, un modelo de inteligencia artificial generativa basado en difusión torsional sobre el espacio circular S^1 , aplicando ruido distribuido según la distribución de **Von Mises** para recuperar conformaciones de cadenas laterales a partir de un *backbone* fijo. La arquitectura emplea una Red Neuronal de Grafos (GNN) entrenada sobre 597 proteínas de alta resolución ($<2,0$ Å) extraídas de PDB-REDO, con partición 80/10/10 a nivel de proteína para evitar *data leakage*.

El modelo, con 80.404 parámetros, se entrenó durante 50 épocas con AdamW y *CosineAnnealingLR*, obteniendo el mejor *checkpoint* en la época 42 (val_loss = 0,0885). La evaluación sobre el conjunto de test arroja un MAE circular de χ_1 de 56,41° [IC 95%: 55,63°; 57,15°] y una tasa de recuperación de rotámeros del 53,9% a $\pm 40^\circ$, superando claramente al *baseline* aleatorio y al *baseline* modal en χ_2 – χ_4 .

El análisis de calibración mediante varianza circular de Von Mises reporta un coeficiente de Spearman $\rho = 0,299$ ($p < 0,001$), evidenciando una señal real de autodiagnóstico del modelo. Los diagramas de rosa confirman la reproducción cualitativa de la trimodalidad rotamérica característica.

Se concluye que la difusión torsional con estadística circular constituye un marco técnicamente sólido para el modelado de cadenas laterales, con ventajas en calibración de incertidumbre respecto a métodos determinísticos, si bien existe una brecha sustancial respecto a SCWRL4 que requiere escalado del *dataset* y arquitecturas equivariantes en trabajos futuros.

Abstract

Accurate modeling of amino acid side chains remains a critical limitation in computational structural biology. While tools such as AlphaFold2 predict the protein backbone with high fidelity, the positions of dihedral angles χ_1 – χ_4 remain imprecise, directly impacting drug design and the understanding of molecular reactivity.

This work develops **ChiGNN**, a generative AI model based on torsional diffusion over the circular space S^1 , applying noise drawn from the **Von Mises distribution** to recover side-chain conformations from a fixed backbone. The architecture employs a Graph Neural Network (GNN) trained on 597 high-resolution protein structures (<2.0 Å) from PDB-REDO, using an 80/10/10 protein-level split to prevent data leakage.

The model, comprising 80,404 trainable parameters, was trained for 50 epochs using AdamW with CosineAnnealingLR, yielding the best checkpoint at epoch 42 (val_loss = 0.0885). Evaluation on the test set reports a circular MAE for χ_1 of 56.41° [95% CI: 55.63° ; 57.15°] and a rotamer recovery rate of 53.9% at $\pm 40^\circ$, clearly outperforming random and modal baselines for χ_2 – χ_4 .

A confidence calibration analysis using Von Mises circular variance yields a Spearman coefficient $\rho = 0.299$ ($p < 0.001$), demonstrating a genuine self-diagnostic signal. Rose plots confirm qualitative reproduction of the canonical trimodal rotameric distribution.

The study concludes that torsional diffusion with circular statistics provides a technically sound framework for side-chain modeling, offering uncertainty quantification advantages over deterministic methods. However, a substantial performance gap relative to SCWRL4 ($\sim 83\%$ recovery) remains, requiring larger datasets and equivariant architectures in future work.

Índice

1. Introducción	1
1.1. Contexto y justificación del Trabajo	1
1.2. Descripción general	2
1.3. Objetivos del Trabajo	3
1.4. Impacto en sostenibilidad, ético-social y de diversidad	4
1.5. Enfoque y método seguido	5
1.6. Planificación del trabajo	6
Estado del proyecto en la entrega de la PEC3 (inicios de mayo de 2026)	9
Causas de la desviación.....	10
Impacto y conclusión	10
1.7. Breve resumen de productos obtenidos	11
1.8. Breve descripción de los otros capítulos de la memoria	12
1.9. Declaración sobre el uso de inteligencia artificial generativa	13
2. Estado del arte	16
2.1. Fundamentos de la estructura proteica: backbone, cadenas laterales y rotámeros	16
2.2. Métodos clásicos de predicción de cadenas laterales	21
2.3. Predicción de estructuras proteicas mediante aprendizaje profundo	22
2.4. Modelos generativos de difusión en biología estructural	23
2.5. Redes neuronales de grafos equivariantes para proteínas	25
2.6. AlphaFold3 y la convergencia hacia la difusión all-atom.....	26
2.7. Estadística circular aplicada a ángulos diedros	27
2.8. Resumen comparativo de métodos	27
3. Materiales y métodos.....	30
3.1. Fase 1: Cimentación y preparación de los datos.....	30
3.2. Fase 2: Implementación del modelo de IA.....	35

3.3.	Fase 3: Evaluación bioestadística	39
4.	<i>Resultados</i>	43
4.1.	Resultados Fase 1: Cimentación y preparación de los datos	43
4.2.	Resultados Fase 2: Implementación del modelo de IA.....	46
4.3.	Resultados Fase 3: Evaluación bioestadística	50
5.	<i>Discusión</i>	58
5.1.	Interpretación de los resultados	58
5.2.	Análisis de calibración de confianza	59
5.3.	Limitaciones del estudio.....	59
5.4.	Contribuciones originales	60
6.	<i>Conclusiones y trabajos futuros</i>	61
6.1.	Conclusiones.....	61
6.2.	Trabajos futuros	62
7.	<i>Glosario</i>	64
8.	<i>Bibliografía</i>	72
9.	<i>Anexos</i>	75
9.1.	Repositorio de GitHub	75
9.2.	Reproducibilidad del proyecto	77
9.3.	Secuencia de código del proyecto y salidas.....	80

Lista de figuras

Figura 1. Diagrama de Gantt para la planificación de tareas. Distribución semanal de las cuatro fases del proyecto (Cimentación, Implementación, Análisis y Finalización) a lo largo de las 13 semanas de desarrollo. -- [pág 7. \[§ 1.6\]](#)

Figura 2. Geometría del backbone proteico. Dos planos peptídicos consecutivos flanquean el carbono alfa central. Los únicos grados de libertad son ϕ (N-C α) y ψ (C α -C). El ángulo del enlace peptídico es prácticamente constante, $\omega \approx 180^\circ$. -- [pág 11. \[§ 2.1\]](#)

Figura 3. Ejemplo del diagrama de Ramachandran. Los datos experimentales se acumulan en las regiones predichas teóricamente, correspondientes a las estructuras secundarias canónicas (hélice α , lámina β y giros). -- [pág 13. \[§ 2.1\]](#)

Figura 4. Ángulos diedros χ en una cadena lateral. Diagrama mostrando χ_1 (N-C α -C β -C γ), χ_2 (C α -C β -C γ -C δ), χ_3 (C β -C γ -C δ -C ϵ) y χ_4 (C γ -C δ -C ϵ -N ζ). -- [pág 14. \[§ 2.1\]](#)

Figura 5. Conformaciones rotaméricas del ángulo χ_1 en proyecciones de Newman sobre el enlace C α -C β . De izquierda a derecha: gauche⁻ ($g^- \approx -60^\circ$, más frecuente), trans ($t \approx 180^\circ$) y gauche⁺ ($g^+ \approx +60^\circ$, menos frecuente). -- [pág 15. \[§ 2.1\]](#)

Figura 6. Distribución de ángulos χ_1 de los estados rotaméricos en el dataset curado (156.407 residuos, 36 sectores de 10°). Rose plot (histograma polar) con orientación convencional (0° en la parte superior, sentido horario). Se confirma la trimodalidad rotamérica característica. -- [pág 36. \[§ 4.1\]](#)

Figura 7. Curva de aprendizaje del modelo ChiGNN. Pérdida circular ($1 - \cos(\theta_{\text{pred}} - \theta_{\text{real}})$) de entrenamiento (azul) y validación (naranja) a lo largo de 50 épocas. La línea punteada verde marca el mejor checkpoint (época 42, val_loss = 0,0885). -- [pág 39. \[§ 4.2\]](#)

Figura 8. Análisis de rendimiento de ChiGNN por tipo de aminoácido. Panel izquierdo: MAE circular de χ_1 ordenado de mayor a menor error, con líneas de referencia en 20° y 40° . Panel derecho: tasa de recuperación de rotámeros $\pm 40^\circ$, con línea de referencia al 83% correspondiente a SCWRL4. -- [pág 42. \[§ 4.3\]](#)

Figura 9. Distribución rotamérica experimental vs. predicha por ChiGNN. Rose plots de los ángulos χ_1 (N-C α -C β -C γ), χ_2 (C α -C β -C γ -C δ), χ_3 (C β -C γ -C δ -C ϵ) y χ_4 (C γ -C δ -C ϵ -C ζ , exclusivo de ARG y LYS). Verde: distribución experimental; azul: predicciones ChiGNN. Cada gráfico emplea 36 bins de 10° . -- [pág 43. \[§ 4.3\]](#)

Figura 10. *Análisis de calibración de confianza de ChiGNN. Panel izquierdo: diagrama de dispersión varianza circular vs. MAE real con línea de tendencia ($\rho = 0,299$, $p < 0,001$). Panel derecho: error medio por decil de incertidumbre (D1 = menor, D10 = mayor incertidumbre).* -- [pág 44. \[§ 4.3\]](#)

Figura 11. *Benchmarking de ChiGNN frente a baselines. Panel izquierdo: MAE circular ($^{\circ}$), menor es mejor. Panel derecho: tasa de recuperación de rotámetros $\pm 40^{\circ}$ (%), mayor es mejor. Los tres modelos (aleatorio, modal, ChiGNN) se comparan para los cuatro ángulos diedros χ_1 – χ_4 .* -- [pág 45. \[§ 4.3\]](#)

1. Introducció

1.1. Contexto y justificación del Trabajo

En la actualidad, la biología estructural es capaz de predecir el plegamiento de las proteínas de forma fiel a la realidad, sin embargo, la configuración de las cadenas laterales de los aminoácidos sigue presentando imprecisiones. [1]

La necesidad de esclarecer la posición de los residuos de los aminoácidos de las proteínas está en que estas cadenas laterales son las que determinan la función biológica: son las que forman los sitios activos de las enzimas y las que interactúan con los fármacos. Sin una posición exacta de sus átomos, es imposible realizar un diseño de fármacos preciso o entender procesos de señalización celular a nivel molecular.

Este tema es de máxima relevancia por dos razones principales:

- **Relevancia Biomédica:** La mayoría de las enfermedades se deben a interacciones moleculares que ocurren en la superficie de las proteínas, donde las cadenas laterales son protagonistas.
- **Frontera Tecnológica:** La comunidad científica (incluyendo proyectos como AlphaFold3) está migrando hacia modelos de IA generativa para resolver este problema, ya que los métodos tradicionales están llegando a su límite de precisión. [2]

Al comenzar este trabajo, el problema se aborda principalmente de dos formas:

- **Librerías de Rotámeros:** Se utilizarán bases de datos estadísticas de posiciones "favoritas" que los aminoácidos suelen adoptar. Es un enfoque discreto y limitado que no siempre se adapta bien al entorno específico de cada proteína.
- **Campos de fuerza físicos:** Simulaciones basadas en física que buscan la energía mínima. Aunque son precisas, también son extremadamente costosas computacionalmente y lentas para el análisis de grandes volúmenes de datos.

La aportación principal de este Trabajo Final de Máster es el desarrollo y evaluación de un modelo basado en Difusión Torsional. El resultado que se pretende obtener es un sistema capaz de:

- **Modelado Continuo:** Superar las limitaciones de las librerías discretas, permitiendo que los ángulos de las cadenas laterales se muevan libremente en un espacio circular.
- **Precisión Atómica:** Obtener conformaciones de cadenas laterales con una desviación angular mínima respecto a los valores experimentales, cuantificada mediante el Error Absoluto Medio circular (MAE circular) sobre los ángulos χ_1 – χ_4 .
- **Herramienta Bioestadística:** Proporcionar un marco de evaluación basado en estadística circular (Von Mises) que permita cuantificar la incertidumbre de la predicción.

1.2. Descripción general

Este Trabajo de Final de Máster (TFM) consiste en el desarrollo y evaluación de un modelo de Inteligencia Artificial generativa basado en Difusión Torsional para la recuperación de las cadenas laterales de las proteínas a partir de un esqueleto o *backbone* fijo. El proyecto aborda una limitación crítica en la biología estructural actual: mientras que herramientas como AlphaFold2 predicen con éxito la cadena principal, la posición de los átomos de las cadenas laterales (determinada por los ángulos diedros χ_1 , χ_2 , χ_3 y χ_4) sigue siendo inexacta, lo que afecta directamente a la comprensión de la reactividad química y la unión de ligandos.

Para llevar a cabo este trabajo, el plan de ejecución se ha estructurado en tres fases principales durante un periodo de tres meses:

- Cimentación y preparación de datos: Se establece un entorno de trabajo en Google Colab utilizando Python (PyTorch Geometric y Biopython) para procesar un conjunto de datos curado de estructuras de alta resolución extraídas de PDB-REDO.
- Implementación del modelo de IA: Se desarrolla ChiGNN, una arquitectura personalizada lite inspirada en los principios de DiffPack, pero que utiliza redes neuronales de grafos (GNN). La clave técnica reside en el uso de la distribución de Von Mises para modelar el proceso de difusión, lo cual es esencial debido a la naturaleza circular y periódica de los ángulos de las proteínas.
- Evaluación bioestadística y benchmarking: El trabajo concluye con un análisis riguroso de la precisión del modelo, comparando su tasa de recuperación de rotámetros y el Error Absoluto Medio (MAE) frente a herramientas clásicas como SCWRL4 y modelos de vanguardia como AlphaFold3. Se pone especial énfasis en

la validación mediante diagramas de rosa (Rose Plots) y la calibración de la confianza del modelo.

En definitiva, este TFM pretende demostrar que el uso de estadística circular aplicada a modelos de difusión permite obtener estructuras proteicas más precisas y útiles para el diseño de fármacos, cumpliendo con los estándares de rigor académico y responsabilidad social que definen el ámbito de la Bioinformática y la Bioestadística.

1.3. Objetivos del Trabajo

Objetivos Generales:

1. **Desarrollar un modelo de aprendizaje profundo basado en Difusión Torsional** para la predicción y recuperación de las conformaciones de las cadenas laterales de una proteína a partir de un esqueleto (backbone) fijo.
2. **Evaluar el rendimiento del modelo mediante métricas de estadística circular**, estableciendo un marco comparativo frente a los métodos tradicionales de librerías de rotámeros y herramientas de estado del arte.

Para alcanzar los objetivos generales, se plantean los siguientes objetivos específicos.

Objetivos específicos relacionados con el primer objetivo general:

- Generar un conjunto de datos curado a partir de estructuras de alta resolución (resolución $< 2.0\text{\AA}$ PDB-REDO) que permita el entrenamiento y validación del modelo mediante pares de coordenadas de esqueleto y ángulos diedros.
- Modelar el proceso de difusión directa aplicando ruido basado en la distribución de Von Mises a los ángulos diedros χ_1 , χ_2 , χ_3 y χ_4 capturando la periodicidad de los ángulos en un espacio circular.
- Entrenar una Red Neuronal de Grafos (GNN) que actúe como proceso de difusión inversa para predecir el gradiente de la distribución de ángulos y recuperar las posiciones atómicas originales.

Objetivos específicos relacionados con el segundo objetivo general:

- Cuantificar la tasa de recuperación de rotámeros, midiendo el porcentaje de ángulos predichos con un error inferior a 20° y 40° respecto a los valores experimentales.

- Validar la distribución rotamérica de las predicciones mediante el uso de estadística circular y diagramas de rosa para asegurar que el modelo respeta las agrupaciones de ángulos naturales (clústeres en -60° , 60° y 180°).
- Determinar la calibración de confianza del modelo, analizando si la varianza circular de Von Mises de las muestras del proceso reverse se correlaciona con el MAE circular real de las predicciones.

1.4. Impacto en sostenibilidad, ético-social y de diversidad

Este Trabajo de Final de Máster se alinea con la competencia transversal de "Compromiso ético y global", actuando de forma honesta y sostenible para diseñar soluciones que mejoren la práctica profesional en la bioinformática. A continuación, se detallan los impactos identificados en sus tres dimensiones:

1. Dimensión de Sostenibilidad

El proyecto se encuadra principalmente en el **ODS 9 (Industria, innovación e infraestructura)**, al desarrollar herramientas de IA de vanguardia para la biología estructural.

- Impacto positivo: La transición de métodos experimentales de laboratorio ("wet lab") a modelos computacionales de alta precisión (*in silico*) supone un ahorro significativo de recursos físicos, reducción de residuos químicos y menor consumo energético en comparación con la cristalografía de rayos X. También reduce el tiempo de cómputo necesario para la predicción atómica, lo que se traduce en un menor consumo energético de los centros de datos en comparación con las simulaciones de dinámica molecular extensas.
- Huella ecológica: El entrenamiento de modelos de *Deep Learning* genera una huella de carbono debido al consumo energético de las GPUs. Para minimizarla, se ha optado por el uso de infraestructuras en la nube (Google Colab) y arquitecturas de grafos (GNN) que optimizarán el uso de la GPU.

2. Dimensión Ética y de Responsabilidad Social

Este trabajo se alinea con el **ODS 3 (Salud y bienestar)** en pro de desarrollar una técnica para optimizar el diseño de fármacos y con el **ODS 16 (Paz, justicia e instituciones sólidas)** a través del rigor científico y promover la ciencia abierta al público.

- Comportamiento ético: El desarrollo se basa en principios de "Open Science", utilizando conjuntos de datos públicos y asegurando que los resultados y el código fuente sean accesibles mediante un repositorio de GitHub para permitir la reproducibilidad.
- Responsabilidad social: Al situarse en la "frontera" de la biología estructural, este modelo puede acelerar el diseño de fármacos y la comprensión de enfermedades, contribuyendo al bien común y a la mejora de la salud global. Se han respetado los principios deontológicos de la profesión bioestadística en el tratamiento de los datos.

3. Dimensión de Diversidad y Derechos Humanos

El proyecto impacta en el **ODS 10 (Reducción de las desigualdades)** al democratizar el acceso a herramientas de alta tecnología.

- Perspectiva de género: En la revisión de la literatura se referencian aportaciones científicas velando por la visibilidad de autoras y autores por igual.
- Diversidad en los datos: Se reflexiona sobre si los datasets estructurales utilizados representan la diversidad biológica necesaria. El diseño del software se ha realizado bajo principios de código abierto para garantizar que investigadores de instituciones con menos recursos puedan acceder a estas herramientas de vanguardia, reduciendo la brecha tecnológica.

1.5. Enfoque y método seguido

Para abordar el problema de la recuperación de las cadenas laterales de las proteínas a partir de un esqueleto fijo, se han considerado diversas estrategias metodológicas basadas en el estado del arte de la biología estructural.

Posibles estrategias

1. Métodos tradicionales basados en bibliotecas de rotámeros: Estrategias clásicas como SCWRL4, que utilizan funciones de energía y búsqueda discreta en bibliotecas de conformaciones predefinidas.
2. Modelos de aprendizaje profundo predictivo: Uso de Redes Neuronales de Grafos (GNN) para realizar una regresión directa de los ángulos diedros.
3. Modelos generativos de IA basados en difusión: Implementación de procesos de difusión torsional que modelan la distribución de probabilidad de los ángulos en un espacio continuo y periódico.

Estrategia elegida

La estrategia seleccionada para este trabajo consiste en el desarrollo de ChiGNN, que es una arquitectura propia *inspirada* en DiffPack. Se opta por implementar un flujo de trabajo personalizado en Python y Google Colab que utilice los principios de la **difusión inversa y las GNN** para recuperar las conformaciones originales.

Valoración de la estrategia

Esta es la estrategia más apropiada para alcanzar los objetivos por las siguientes razones:

- Innovación y relevancia: Los modelos de difusión representan la frontera actual en la predicción de estructuras proteicas, habiendo sido adoptados recientemente por herramientas líderes como AlphaFold3.
- Tratamiento de la periodicidad: A diferencia de los modelos de regresión estándar, el enfoque de difusión propuesto utiliza la distribución de Von Mises, lo que permite manejar adecuadamente la naturaleza circular y periódica de los ángulos diedros χ_1 , χ_2 , χ_3 y χ_4 .
- Rigor bioestadístico: Esta metodología facilita la evaluación de métricas de confianza y la creación de diagramas de rosa (Rose Plots) para analizar si el modelo captura correctamente la naturaleza rotamérica de las cadenas laterales.
- Viabilidad técnica: El desarrollo de una versión "lite" permite la ejecución del proyecto en entornos de computación en la nube como Google Colab, utilizando librerías como PyTorch Geometric y Biopython, garantizando la obtención de resultados evaluables en un plazo relativamente corto.

1.6. Planificación del trabajo

1.6.1. Planificación del trabajo a priori

La planificación del proyecto se estructura en 13 semanas y se divide en 4 bloques. Las tareas a realizar con la vinculación a los objetivos específicos se plasman a continuación en la Tabla 1.

Tabla 1. Cronograma de tareas y su vinculación a los objetivos específicos.

Bloque	Tarea	Duración	Vinculación a objetivo específico
Fase 1: Cimentación	T1.1: Configuración del entorno en Google Colab con PyTorch Geometric y Biopython	Semana 1-2	General
	T1.2: Filtrado de estructuras de alta resolución (<2.0Å) de PDB-REDO	Semana 3	O.E.1
	T1.3: Generación del dataset curado (.pt/.csv) con pares esqueleto-cadena lateral	Semana 4	O.E.1
Fase 2: Implementación	T2.1: Implementación del proceso de difusión directa (ruido de Von Mises) y visualización en py3Dmol	Semana 5-6-7	O.E.2
	T2.2: Diseño y entrenamiento de la Red Neuronal de Grafos (GNN) para la difusión inversa	Semana 7-8-9	O.E.3
	T2.3: Recuperación de las primeras conformaciones y ajuste de hiperparámetros	Semana 9	O.E.3
Fase 3: Análisis	T3.1: Benchmarking estadístico: comparación de MAE y precisión angular frente a SCWRL4 y AlphaFold3	Semana 10	O.E.4
	T3.2: Análisis de Rose Plots para validación rotamérica y calibración de confianza	Semana 11	O.E.5 y 6
Fase 4: Finalización	T4.1: Redacción de la memoria final y preparación de figuras	Semana 12	General
	T4.2: Preparación del repositorio GitHub y demo para la defensa	Semana 13	General

A continuación, se observa un diagrama de Gantt que muestra la planificación de las actividades a realizar cada semana.

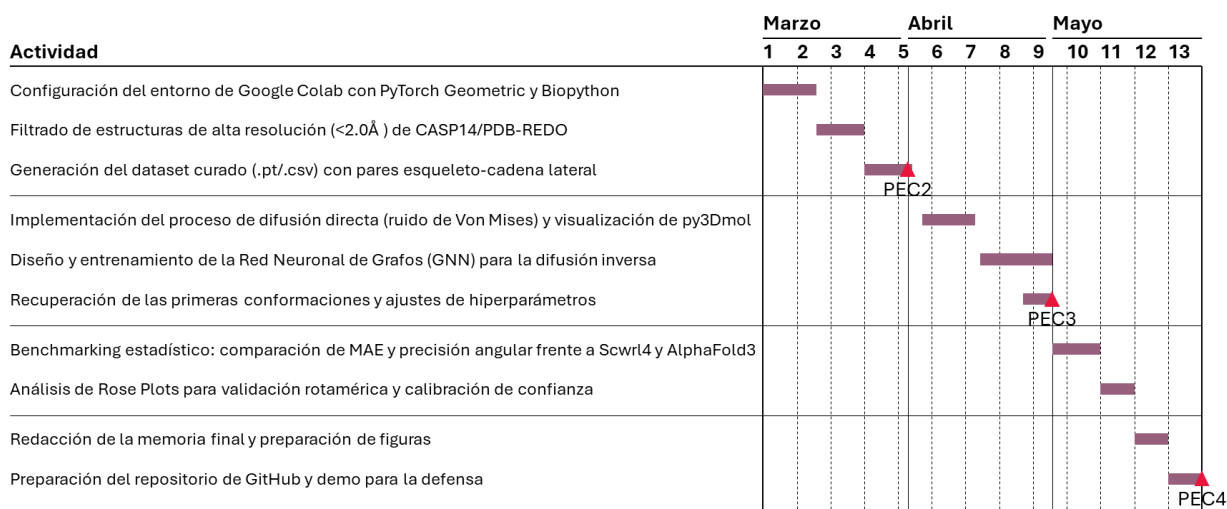


Figura 1. Diagrama de Gantt para la planificación de tareas.

Fuente: Elaboración propia.

Para asegurar el cumplimiento del calendario, se establecen los siguientes hitos.

- Hito 1 (Semana 4): Dataset validado y listo para entrenamiento. **PEC2**.
- Hito 2 (Semana 8): Modelo de difusión inversa funcional con capacidad de recuperación. **PEC3**.
- Hito 3 (Semana 10): Evaluación bioestadística completa y comparativa finalizada
- Hito 4 (Semana 12): Entrega de la memoria final y código fuente. **PEC4**.

Se identifican los siguientes factores que podrían impactar en el desarrollo del TFM, junto con los planes de mitigación.

1. Alcance excesivo del modelo (Riesgo técnico): Intentar replicar la arquitectura completa de modelos masivos como AlphaFold3 en un tiempo limitado.
 - *Mitigación:* Se ha optado por un enfoque de "versión lite" centrada exclusivamente en la difusión torsional de ángulos χ , lo que acota el problema a una dimensión manejable.
2. Limitaciones de cómputo (Riesgo de tiempo): El entrenamiento de GNNs puede exceder los límites de tiempo de las GPUs gratuitas de Google Colab.
 - *Mitigación:* Optimización del dataset para reducir el número de nodos por grafo y uso de técnicas de *checkpointing* para guardar el progreso del modelo.
3. Baja precisión del modelo: El modelo puede no haber aprendido suficientemente bien debido al exceso de ruido.

- *Mitigación:* Revisar la concentración de la distribución de Von Mises o simplificar el problema a solo ángulo χ_1 , χ_2 .
- 4. No convergencia de la difusión inversa (Riesgo metodológico): Que el modelo no aprenda a eliminar el ruido de Von Mises adecuadamente.
 - *Mitigación:* Implementar un modelo base simplificado antes de pasar a la arquitectura GNN completa y utilizar librerías robustas como torch-geometric.
- 5. Disponibilidad de datos (Riesgo de recursos): Problemas en la descarga o procesamiento de archivos PDB complejos.
 - *Mitigación:* Uso de datasets pre-procesados como PDB-REDO que ya cuentan con criterios de calidad estandarizados.

1.6.2. Organización temporal real del trabajo

Pese al calendario inicial del proyecto, el desarrollo real presentó una desviación positiva que merece ser explicada.

Estado del proyecto en la entrega de la PEC3 (inicios de mayo de 2026)

En el momento de la entrega de la PEC3, a principios de mayo de 2026, el proyecto había alcanzado un grado de avance significativamente superior al previsto para esa entrega intermedia:

- **Código fuente: 100% completado.** El pipeline completo había sido implementado, validado y documentado en su totalidad. Esto incluía la Fase 1 (preparación del dataset), la Fase 2 (arquitectura y entrenamiento de ChiGNN), y la Fase 3 (evaluación bioestadística completa con todas las métricas, rose plots, análisis de calibración y benchmarking). El mejor checkpoint del modelo (época 42, val_loss = 0,0885) ya estaba guardado en Google Drive.
- **Memoria: 90% redactada.** Los capítulos 1 al 8 (Introducción, Estado del arte, Materiales y métodos, Resultados, Discusión, Conclusiones, Glosario y Bibliografía) estaban completamente redactados y revisados. El único componente pendiente era el Capítulo 9 (Anexos), que requería la preparación y documentación del repositorio de GitHub y la guía de reproducibilidad.

En la Tabla 2 se refleja el avance real de la ejecución del proyecto.

Tabla 2. Cronograma de ejecución real del proyecto

PEC / Hito	Período	Estado en la entrega
PEC1	Inicios de febrero 2026	Planificación, definición de objetivos y revisión del estado del arte.
PEC2	Inicios de marzo 2026	Dataset curado (597 proteínas, 156.407 residuos), pipeline de extracción validado. (Fase 1)
PEC3	Inicios de mayo 2026	Código al 100% completado (Fase 2 i Fase 3). Memoria al 90% redactada (pendiente únicamente los anexos).
PEC4 / Entrega final	Inicios de junio 2026	Memoria completa con todos los apartados. Repositorio GitHub

Causas de la desviación

- **Eficiencia en la Fase 1.** La disponibilidad y calidad de la API de RCSB y de la base de datos PDB-REDO permitió que el pipeline de adquisición y curación del dataset se completara en el tiempo previsto sin las incidencias técnicas contempladas en el plan de mitigación de riesgos.
- **Convergencia anticipada del modelo.** ChiGNN alcanzó su mejor checkpoint en la época 42 de 50 planificadas, sin necesidad de ajustes extensivos de hiperparámetros, lo que liberó tiempo para adelantar la redacción de resultados y discusión.
- **Trabajo simultáneo código-memoria.** La estrategia de redactar cada sección de la memoria en paralelo con su implementación técnica permitió documentar las decisiones de diseño en el momento en que se tomaban, reduciendo el tiempo de redacción final.

Impacto y conclusión

La desviación temporal no ha comprometido en ningún caso la calidad ni la completitud del trabajo. Muy al contrario, el haber concluido el código y la mayor parte de la memoria con anterioridad a la entrega final ha permitido dedicar el tiempo restante a la elaboración de los apartados de los anexos (Capítulo 9), la revisión formal del conjunto del documento, la preparación del repositorio de GitHub y la elaboración de la presentación. Todos los objetivos específicos planteados en la PEC1 han sido alcanzados satisfactoriamente.

1.7. Breve resumen de productos obtenidos

Al finalizar este proyecto, se habrán obtenido una serie de productos tangibles y medibles que demuestran la viabilidad del uso de modelos generativos de IA en la biología estructural.

Los entregables finales son:

- Memoria del TFM: Un documento académico exhaustivo que documenta el estado del arte de la difusión torsional, la lógica del modelo (distribución de Von Mises), el proceso de entrenamiento de la GNN y la evaluación bioestadística. Incluirá además el análisis de impacto ético, social y de sostenibilidad exigido por la normativa.
- Productos (Repositorio, dataset y modelo):
 - Repositorio de GitHub: Contendrá el código fuente estructurado en Python, los scripts de preprocesamiento y el modelo final.
 - Dataset curado: Dos archivos derivados de PDB-REDO con estructuras de alta resolución ($<2.0\text{\AA}$): el dataset en formato PyTorch Geometric listo para el entrenamiento de grafos (`dataset_torsional_diffusion.pt`) y su versión tabular legible (`dataset_sidechains_curated.csv`).
 - Modelo entrenado (checkpoint): Modelo ChiGNN correspondientes al mejor checkpoint del entrenamiento, guardados en el archivo `chignn_best.pt`.
- Presentación virtual: Un recurso multimedia que sintetiza el proceso de investigación, la metodología de IA generativa y la defensa de los resultados obtenidos ante el tribunal.

Resultados técnicos y científicos (medibles):

El éxito del producto desarrollado se evaluará mediante los siguientes indicadores cuantitativos:

- Tasa de recuperación de rotámeros: Se espera obtener un porcentaje de ángulos predichos con un error inferior a 20° y 40° que sea competitivo frente a métodos clásicos como SCWRL4.
- Error Absoluto Medio (MAE): Cuantificación de la desviación angular media para cada uno de los cuatro ángulos diedros de la cadena lateral.

- Validación de clústeres rotaméricos: Generación de Rose Plots que demuestren que las predicciones del modelo se agrupan correctamente en los ángulos naturales de -60° , 60° y 180° .
- Calibración de confianza: Un análisis de correlación que demuestre si el error reportado por el proceso de "denoising" del modelo se alinea con la desviación (MAE circular) real de la estructura.

1.8. Breve descripción de los otros capítulos de la memoria

El presente apartado ofrece una guía de lectura sintética de los capítulos que componen esta memoria, con el objeto de orientar al lector sobre el contenido y la función de cada uno de ellos.

- El Capítulo 2 (Estado del arte) sitúa el trabajo en el contexto científico actual. Se revisan los fundamentos de la estructura proteica —backbone, cadenas laterales y rotámeros—, los métodos clásicos de predicción basados en librerías de rotámeros (SCWRL4, Rosetta), la evolución del aprendizaje profundo aplicado a proteínas (AlphaFold2, ESMFold, RoseTTAFold), los modelos generativos de difusión en biología estructural (DDPM, DiffPack, AlphaFold3), las redes neuronales de grafos equivariantes (EGNN, GVP, ProteinMPNN) y los fundamentos de la estadística circular aplicada a ángulos diedros. El capítulo concluye con una tabla comparativa de los métodos revisados que justifica las decisiones metodológicas adoptadas.
- El Capítulo 3 (Materiales y métodos) describe el diseño experimental completo en tres fases. La Fase 1 detalla el pipeline de adquisición y curación del dataset: consulta a la API REST del RCSB con filtros de resolución ($<2,0$ Å), método (cristalografía de rayos X) y tipo de polímero (proteína), descarga de estructuras re-refinadas de PDB-REDO, extracción de ángulos diedros χ_1 – χ_4 mediante Biopython y serialización del dataset en formato PyTorch Geometric con máscara booleana `y_mask`. La Fase 2 describe la arquitectura ChiGNN (80.404 parámetros, cuatro capas GCNConv con BatchNorm y conexiones residuales), el proceso forward de Von Mises, el bucle de entrenamiento con AdamW y CosineAnnealingLR, y el proceso reverse mediante DDIM adaptado. La Fase 3 detalla las métricas de evaluación basadas en estadística circular (MAE circular, tasa de recuperación de rotámeros, varianza de Von Mises, correlación de Spearman) y el diseño del benchmarking frente a baselines.
- El Capítulo 4 (Resultados) presenta los resultados cuantitativos y cualitativos de las tres fases. Se incluyen las métricas de calidad del dataset (597 proteínas, 156.407 residuos, distribución rotamérica de χ_1 con trimodalidad confirmada), la curva de

aprendizaje del modelo con diagnóstico de convergencia (mejor checkpoint en época 42, val_loss = 0,0885), las métricas de evaluación sobre el conjunto de test —MAE circular de $\chi_1 = 56,41^\circ$ [IC 95%: 55,63°; 57,15°], recuperación $\pm 40^\circ$ del 53,9%—, el análisis de error por tipo de aminoácido, los diagramas de rosa comparativos y el análisis de calibración de confianza ($p = 0,299$, $p < 0,001$).

- El Capítulo 5 (Discusión) interpreta los resultados en el contexto del estado del arte, analiza el comportamiento diferencial del modelo por aminoácido, examina las implicaciones de la calibración de confianza como ventaja competitiva frente a métodos determinísticos, e identifica las limitaciones del estudio en tres categorías: dataset, arquitectura y proceso de evaluación. Se enumeran asimismo las contribuciones originales del trabajo.
- El Capítulo 6 (Conclusiones y trabajos futuros) sintetiza las conclusiones en correspondencia con cada objetivo específico planteado y propone líneas de desarrollo futuro organizadas en tres bloques: escalado del dataset y arquitectura, mejoras en el proceso de difusión, y evaluación y validación externa.
- El Capítulo 7 (Glosario) recoge en orden alfabético los términos técnicos de mayor relevancia empleados en la memoria, abarcando tanto los conceptos del estado del arte como los específicos de la implementación y la evaluación bioestadística.
- El Capítulo 8 (Bibliografía) lista las referencias citadas a lo largo del trabajo siguiendo el formato establecido por la normativa del máster.
- El Capítulo 9 (Anexos) incluye material complementario de soporte: el código fuente completo del notebook de Google Colab y la documentación del repositorio de GitHub.

1.9. Declaración sobre el uso de inteligencia artificial generativa

En cumplimiento de la normativa académica de la Universitat Oberta de Catalunya (UOC) y de los principios de transparencia científica que rigen la elaboración de trabajos de investigación, la autora declara de forma explícita el uso de herramientas de inteligencia artificial generativa (IAG) durante la realización del presente Trabajo Final de Máster.

1.9.1. Herramientas utilizadas y alcance de su uso

Durante el desarrollo del TFM se hizo uso de los siguientes sistemas de inteligencia artificial generativa:

- **Asistentes de lenguaje natural (LLM):** Se emplearon herramientas de IA conversacional (concretamente, Gemini) para asistir en tareas de resolución de dudas conceptuales sobre implementación en PyTorch y PyTorch Geometric, y depuración de errores de código (debugging) durante el desarrollo del pipeline.
- **Herramientas de generación de código asistida:** Se utilizaron sugerencias de autocompletado durante la redacción del código en Google Colab para acelerar la escritura de fragmentos de código repetitivos o boilerplate, como la configuración de los DataLoaders o la implementación de métricas circulares.

1.9.2. Lo que NO ha sido generado por IA

La autora declara expresamente que los siguientes elementos del TFM son de autoría propia y no fueron generados ni sustituidos por IA generativa:

- El diseño conceptual y la arquitectura de ChiGNN, incluyendo la elección de la distribución de Von Mises como proceso de difusión torsional y la justificación de su idoneidad respecto a alternativas gaussianas.
- El planteamiento metodológico, los objetivos de investigación y las hipótesis de trabajo.
- La implementación íntegra del código fuente del pipeline, incluyendo los módulos de preprocesamiento, la arquitectura de la GNN, el bucle de entrenamiento y todos los análisis de evaluación bioestadística de la Fase 3.
- El análisis, la interpretación y la discusión crítica de los resultados obtenidos (Capítulos 4 y 5).
- Las conclusiones del trabajo (Capítulo 6).
- La síntesis de la presente memoria, la presentación y del repositorio de GitHub.
- La selección, lectura y síntesis de las referencias bibliográficas.

1.9.3. Uso responsable y verificación

En todos los casos en que se utilizó IA generativa, el contenido resultante fue revisado, contrastado con fuentes primarias y modificado por la autora para garantizar su precisión técnica y su adecuación al contexto científico del trabajo. La autora es responsable de la integridad y veracidad de todo el contenido incluido en esta memoria.

El uso de estas herramientas se ha realizado de conformidad con el marco ético de la UOC y no ha supuesto en ningún caso la delegación de tareas de razonamiento crítico, diseño experimental o análisis científico en los sistemas de IA.

2. Estado del arte

La predicción de la conformación de cadenas laterales de proteínas es un problema central en biología estructural computacional que se sitúa en la intersección de tres tradiciones de investigación: la estadística de rotámeros, la predicción de estructuras proteicas mediante aprendizaje profundo y los modelos generativos de difusión. Esta sección revisa el estado del arte en cada una de estas áreas y sitúa el presente TFM en el contexto de la literatura reciente.

2.1. Fundamentos de la estructura proteica: backbone, cadenas laterales y rotámeros

2.1.1. La cadena polipeptídica: estructura química y geometría del backbone

Las proteínas son polímeros lineales constituidos por aminoácidos unidos covalentemente mediante enlaces peptídicos. Cada aminoácido aporta tres átomos a la cadena principal del polipéptido —denominada *backbone* o cadena principal—: el nitrógeno amídico (N), el carbono alfa ($C\alpha$) y el carbono carbonílico (C). La secuencia repetida $N-C\alpha-C(=O)-N-C\alpha-C$ forma el esqueleto covalente de la proteína [3].

Una propiedad geométrica crítica de este esqueleto es la planaridad del enlace peptídico. El doble enlace parcial entre el carbono carbonílico ($C=O$) y el nitrógeno amídico ($N-H$), resultado de la deslocalización electrónica, confiere al enlace peptídico un carácter de doble enlace parcial que impide la rotación libre en torno al eje $C-N$. Como consecuencia, los seis átomos que forman cada unidad peptídica — $C\alpha(i)$, $C(i)$, $O(i)$, $N(i+1)$, $H(i+1)$, $C\alpha(i+1)$ — se mantienen en un mismo plano, denominado plano peptídico [3]. Esta restricción reduce drásticamente el espacio conformacional accesible a la cadena, ya que de los tres tipos de enlace del *backbone* solo dos son libremente rotativos: el enlace $N-C\alpha$ (que define el ángulo ϕ) y el enlace $C\alpha-C$ (que define el ángulo ψ).

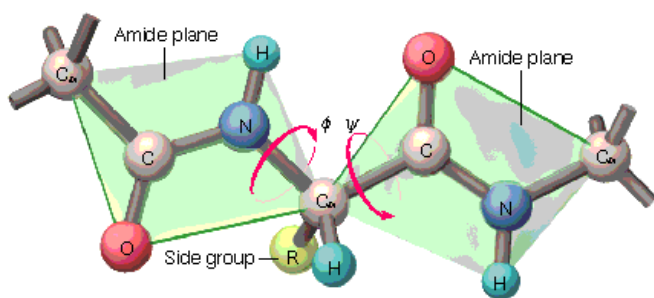


Figura 2. Geometría del backbone proteico. Dos planos peptídicos consecutivos flanquean el carbono alfa central. Los únicos grados de libertad son ϕ ($N-C\alpha$) y ψ ($C\alpha-C$). El ángulo del enlace peptídico es prácticamente constante, $\omega \approx 180^\circ$.

Fuente: Sancho, J. (2021). Lección 3. Enlace peptídico y estructura secundaria. Estructura de macromoléculas 2022-23. Universidad de Zaragoza / BIFI. Disponible en: <https://jsancho.bifi.es/leccion-3-enlace-peptidico-y-estructura-secundaria/> [consultado abril 2026].

2.1.2. Los ángulos diedros ϕ y ψ : el principio de Ramachandran

Un ángulo diedro (o ángulo torsional) se define como el ángulo formado por dos planos que comparten un eje —en este caso, el eje del enlace de referencia—, medido entre cuatro átomos consecutivos. Para ϕ , los cuatro átomos son $C(i-1)-N-C\alpha-C$; para ψ , son $N-C\alpha-C-N(i+1)$ [3, 4].

En los años 1960, G. N. Ramachandran y colaboradores de la Universidad de Madrás trataron los átomos como esferas rígidas y calcularon computacionalmente qué combinaciones de ϕ y ψ eran estéricamente posibles —es decir, cuáles no producían colisiones entre átomos no enlazados [3]. El resultado fue el diagrama de Ramachandran, un mapa bidimensional ϕ - ψ en el que las regiones permitidas corresponden a las estructuras secundarias observadas empíricamente: la hélice α ($\phi \approx -60^\circ$, $\psi \approx -45^\circ$), la lámina β ($\phi \approx -120^\circ$, $\psi \approx +130^\circ$) y los giros (*turns*). Ramachandran y sus colaboradores demostraron además que el efecto de las cadenas laterales sobre los ángulos permitidos del *backbone* es, en su mayor parte, debido únicamente al $C\beta$; las cadenas laterales de mayor tamaño afectan las regiones permitidas solo en un pequeño porcentaje [4].

Una versión empírica del diagrama, construida a partir de 100.000 puntos de datos extraídos de estructuras cristalográficas de alta resolución, confirma estos patrones: la gran mayoría de los residuos se agrupan en las regiones predichas teóricamente [5].

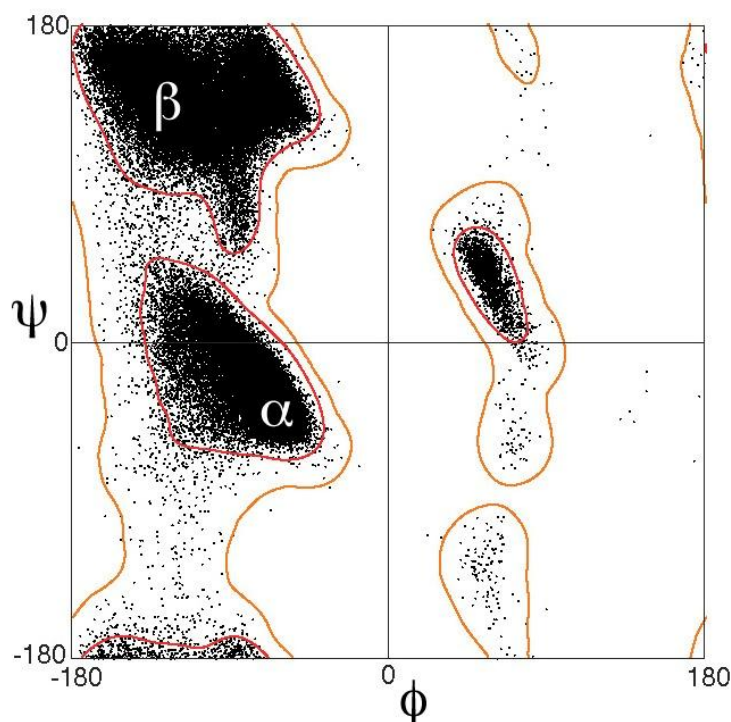


Figura 3. Ejemplo del diagrama de Ramachandran. Los datos experimentales se acumulan en las regiones predichas teóricamente.

Fuente: Dcjrjr (2011). Ramachandran plot general 100K. Wikimedia Commons. CC BY 3.0. Disponible en: https://en.wikipedia.org/wiki/Ramachandran_plot [consultado abril 2026].

2.1.3. Las cadenas laterales y los ángulos diedros χ

Mientras que el *backbone* se describe mediante ϕ y ψ , la conformación tridimensional de las cadenas laterales —los sustituyentes químicos que parten del $C\alpha$ de cada residuo y determinan la identidad del aminoácido— queda definida por una serie adicional de ángulos diedros denominados ángulos chi (χ) [3]. Dependiendo de la longitud de la cadena lateral, un aminoácido puede presentar entre uno y cuatro ángulos chi:

- χ_1 : definido por los átomos $N-C\alpha-C\beta-C\gamma$. Presente en todos los aminoácidos con cadena lateral excepto Gly (sin $C\beta$) y Ala ($C\beta$ terminal, sin $C\gamma$).
- χ_2 : definido por $C\alpha-C\beta-C\gamma-C\delta$. Presente en residuos con cadenas de al menos cuatro átomos pesados.
- χ_3 y χ_4 : presentes únicamente en residuos con cadenas largas: lisina (Lys), arginina (Arg), metionina (Met), glutamato (Glu) y glutamina (Gln).

Las cadenas laterales son funcionalmente críticas: son ellas las que definen los sitios activos enzimáticos, las superficies de unión de ligandos y fármacos, los puentes de hidrógeno inter-residuo que estabilizan la estructura terciaria y el empaquetamiento del

núcleo hidrofóbico de la proteína [6]. Una predicción inexacta de los ángulos χ —del orden de 20–30° de error en χ_1 — puede desplazar los átomos del sitio activo lo suficiente como para invalidar los resultados de un experimento de *docking* molecular o de cálculo de energía de unión.

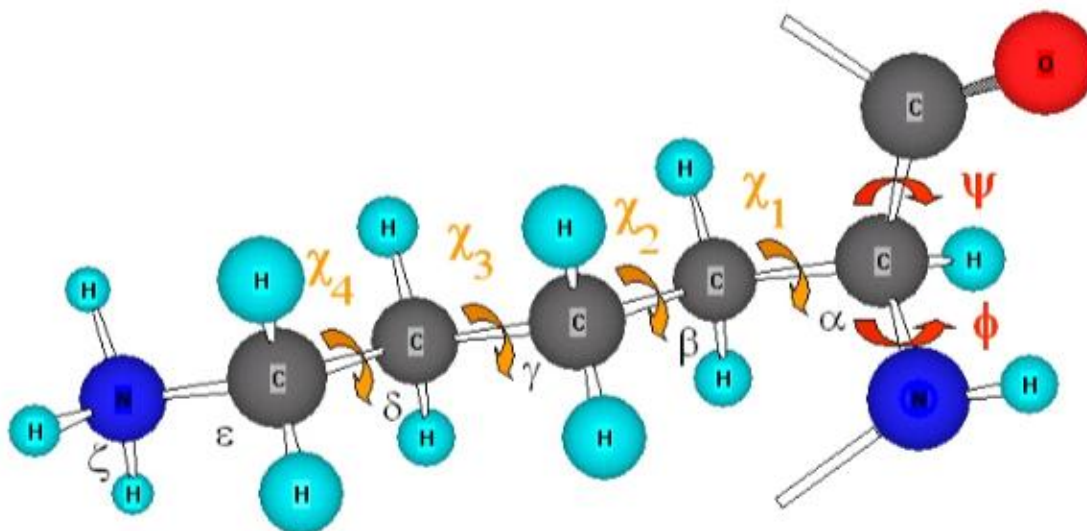


Figura 4. Ángulos diedros χ en una cadena lateral. Diagrama mostrando χ_1 (N–C α –C β –C γ), χ_2 (C α –C β –C γ –C δ), χ_3 (C β –C γ –C δ –C ϵ) y χ_4 (C γ –C δ –C ϵ –N ζ).

Fuente: Sancho, J. (2021). Lección 3. Enlace peptídico y estructura secundaria. Estructura de macromoléculas 2022-23. Universidad de Zaragoza / BIFI. Disponible en: <https://jsancho.bifi.es/leccion-3-enlace-peptidico-y-estructura-secundaria/> [consultado abril 2026].

2.1.4. Los rotámeros: disposición del espacio conformacional

Un hallazgo empírico fundamental, establecido a partir del análisis estadístico de miles de estructuras cristalográficas de alta resolución, es que los ángulos χ no adoptan valores continuos de forma uniforme, sino que se concentran en torno a tres estados preferentes, denominados rotámeros [7, 8]:

- *Gauche*[−] (g^- , $\approx -60^\circ$): el sustituyente gamma ocupa la posición *gauche negativa* respecto al nitrógeno amídico, minimizando las interacciones estéricas con el grupo carbonilo del *backbone*. Es la conformación más frecuente en estructuras de alta resolución.

- Trans (t , $\approx 180^\circ$): estado extendido en el que el sustituyente gamma se sitúa en posición *anti*. Energéticamente favorable para residuos con cadenas largas (Lys, Arg, Glu).
- Gauche⁺ (g^+ , $\approx +60^\circ$): el sustituyente gamma está en posición *gauche positiva*, con mayores impedimentos estéricos respecto al carbonilo del *backbone*. Conformación minoritaria cuya frecuencia varía considerablemente según el tipo de aminoácido.

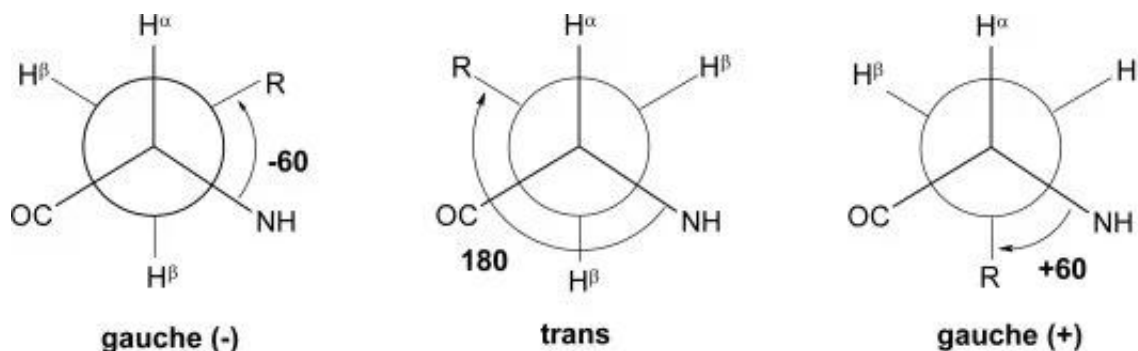


Figura 5. Conformaciones rotaméricas del ángulo χ_1 en proyecciones de Newman sobre el enlace $C\alpha-C\beta$. En orden, de izquierda a derecha, de más común a menos.

Fuente: Sancho, J. (2021). Lección 3. Enlace peptídico y estructura secundaria. Estructura de macromoléculas 2022-23. Universidad de Zaragoza / BIFI. Disponible en: <https://jsancho.bifi.es/leccion-3-enlace-peptidico-y-estructura-secundaria/> [consultado abril 2026].

Este comportamiento trimodal refleja los mínimos de energía de los potenciales de torsión asociados a las interacciones estéricas y electrostáticas entre sustituyentes. Janin et al. [7] y posteriormente Ponder y Richards [8] formalizaron el concepto de librería de rotámeros como tablas de probabilidad condicional $P(\chi_1, \chi_2, \dots | \text{aminoácido})$, que se convirtieron en el núcleo estadístico de todos los métodos clásicos de predicción de cadenas laterales.

Dunbrack y Karplus [9] demostraron un refinamiento crítico: los rotámeros no son independientes del *backbone* sino que presentan una dependencia conformacional respecto a los ángulos ϕ y ψ del residuo, formalizada como la librería *backbone-dependent* $P(\chi | \phi, \psi)$. Esta librería, continuamente actualizada con estructuras de mayor resolución [10], es el estándar de referencia (*gold standard*) contra el que se comparan todos los métodos modernos.

2.1.5. Relevancia funcional y limitaciones de los métodos clásicos

Como se ha comentado, las cadenas laterales son, en última instancia, las responsables de la función biológica de las proteínas: forman los sitios activos enzimáticos, participan en las interacciones proteína–ligando que son el objetivo del diseño farmacéutico, establecen puentes de hidrógeno que estabilizan la estructura terciaria y definen el empaquetamiento del núcleo hidrofóbico [6] .

El concepto de rotámero tiene, sin embargo, una limitación conceptual inherente: trata la conformación de la cadena lateral como un estado discreto único. Las proteínas son sistemas dinámicos y las cadenas laterales, especialmente en regiones de superficie, pueden coexistir como mezcla de rotámeros en equilibrio conformacional. Esta distribución continua de conformaciones —no capturada por los métodos clásicos basados en la asignación del rotámero de máxima probabilidad— es precisamente el problema que los modelos generativos de difusión torsional, como el desarrollado en el presente TFM, abordan de forma explícita.

2.2. Métodos clásicos de predicción de cadenas laterales

Los métodos clásicos de modelado de cadenas laterales comparten una arquitectura común: dado un backbone fijo, buscan la combinación de rotámeros que minimiza la energía del sistema. Las diferencias entre ellos están en la función de energía empleada y el algoritmo de optimización.

2.2.1. SCWRL: búsqueda en librería de rotámeros

SCWRL (*Side Chain placement With a Rotamer Library*) es el método de referencia durante más de dos décadas en la predicción de cadenas laterales. En su versión original Canutescu, A. A. [11] utilizaba la librería backbone-dependent de Dunbrack para asignar el rotámero más probable a cada residuo. La versión SCWRL4 [12], actualmente el índice comparativo estándar, reformula el problema como un grafo de interacción entre residuos y lo resuelve mediante un algoritmo de propagación de creencias (belief propagation). Alcanza una precisión de recuperación de χ_1 del ~83 % en estructuras estándar, pero su rendimiento se degrada significativamente en regiones con alta densidad de contactos o en estructuras apo (sin ligando), donde las cadenas laterales adoptan conformaciones alternativas.

2.2.2. Rosetta y métodos basados en campo de fuerza

El programa Rosetta [13] incorpora la predicción de cadenas laterales como un módulo interno de su pipeline de modelado estructural. Utiliza una función de energía empírica que combina términos de Lennard-Jones, solvatación de Lazaridis-Karplus y un potencial estadístico derivado de la librería de rotámeros. La búsqueda conformacional se realiza mediante Monte Carlo o, más recientemente, mediante el protocolo FastRelax [14]. Aunque los resultados son superiores a SCWRL4 en tareas de modelado global, el costo computacional es significativamente mayor, lo que limita su uso en estudios a gran escala.

2.2.3. Limitaciones generales de los métodos clásicos

Todos los métodos clásicos comparten una limitación conceptual: modelan la conformación de la cadena lateral como un problema de optimización determinista, encontrando el estado de mínima energía pero sin capturar la *distribución* de conformaciones posibles. Las proteínas son sistemas dinámicos con flexibilidad conformacional intrínseca: una cadena lateral puede existir como mezcla de rotámeros en equilibrio, especialmente en regiones de superficie. Esta limitación es particularmente relevante en estructuras apo, donde la ausencia del ligando permite mayor movilidad conformacional [15]. Los métodos generativos descritos en la sección 2.4 abordan directamente este problema.

2.3. Predicción de estructuras proteicas mediante aprendizaje profundo

2.3.1. AlphaFold2: la revolución del backbone

El sistema AlphaFold2, desarrollado por DeepMind [16], supuso un cambio de paradigma en la biología estructural computacional al alcanzar una precisión en la predicción del backbone próxima a la resolución experimental (mediana TM-score > 0,9). Su arquitectura combina redes de atención sobre representaciones de pares de residuos (Evoformer) con un módulo de estructura (Structure Module) que predice directamente las coordenadas atómicas del backbone mediante marcos de referencia rígidos.

Sin embargo, AlphaFold2 presenta limitaciones reconocidas en la predicción de cadenas laterales, especialmente en estructuras apo y en residuos de superficie con alta

movilidad. El modelo predice una única conformación por residuo, sin modelar la distribución conformacional. Estudios comparativos demuestran que la precisión de χ_1 de AlphaFold2 en el conjunto de prueba CAMEO se sitúa en torno al 70–75 %, sustancialmente inferior a su rendimiento en backbone [17]. Este es precisamente el problema que aborda el presente TFM.

2.3.2. RoseTTAFold y modelos de redes de atención estructural

RoseTTAFold [18] propuso de forma independiente una arquitectura de triple vía (1D secuencia, 2D pares de residuos, 3D coordenadas atómicas) que alcanza rendimiento comparable a AlphaFold2. La clave de su diseño es la propagación bidireccional de información entre las tres representaciones, permitiendo que las coordenadas 3D informen la predicción de la secuencia y viceversa. Aunque mejora ligeramente la predicción de cadenas laterales respecto a AlphaFold2, mantiene el mismo paradigma de predicción puntual sin capturar la incertidumbre conformacional.

2.3.3. ESMFold: predicción directa desde secuencia

ESMFold [19] demostró que los modelos de lenguaje de proteínas (ESM-2, entrenado sobre 250 millones de secuencias) codifican suficiente información estructural para permitir la predicción de estructuras de forma directa, sin necesidad de múltiples alineaciones de secuencia (MSA). Su velocidad de inferencia (órdenes de magnitud superior a AlphaFold2) lo hace adecuado para estudios proteómicos a gran escala, aunque con una ligera penalización en precisión para estructuras complejas. En cuanto a cadenas laterales, presenta limitaciones similares a AlphaFold2.

2.4. Modelos generativos de difusión en biología estructural

2.4.1. Fundamentos de los modelos de difusión

Los modelos de difusión denoising (DDPM, *Denoising Diffusion Probabilistic Models*) [20] son modelos generativos que aprenden a generar muestras de una distribución compleja mediante un proceso en dos etapas. El **proceso forward** corrompe progresivamente los datos añadiendo ruido gaussiano hasta que la distribución se aproxima a un ruido blanco. El **proceso reverse** aprende a deshacer esta corrupción paso a paso, permitiendo generar nuevas muestras partiendo de ruido puro. Song et al. [21] reformularon los modelos de difusión en el marco continuo de las ecuaciones

diferenciales estocásticas (SDE), estableciendo la base teórica unificada que subyace a todos los métodos modernos de difusión en biología estructural.

2.4.2. Difusión torsional: el espacio angular S¹

La aplicación de modelos de difusión a ángulos diedros moleculares requiere adaptar el formalismo estándar al espacio de los ángulos, que es periódico y no euclidiano. Jing et al. [22] introdujeron el concepto de ***difusión torsional*** en el contexto de la predicción de confórmeros de moléculas pequeñas (DiffSBDD). La clave conceptual es reemplazar el ruido gaussiano estándar por ruido descrito por la **distribución de Von Mises**, la análoga circular de la distribución normal, definida como:

$$VM(\theta | \mu, \kappa) = \exp(\kappa \cos(\theta - \mu)) / (2\pi I_0(\kappa))$$

donde $I_0(\kappa)$ es la función de Bessel modificada de orden cero y κ es el parámetro de concentración (análogo al recíproco de la varianza). En el proceso forward, κ disminuye progresivamente hasta 0, punto en que la distribución se aproxima a la distribución uniforme sobre el círculo (equivalente al ruido puro en el espacio angular). Esta formulación garantiza la periodicidad y evita los artefactos de corte que aparecerían si se trataran los ángulos como variables lineales.

2.4.3. DiffSBDD y modelos de difusión para moléculas pequeñas

Luo et al. [23] y Schneuing et al. [24] demostraron que los modelos de difusión torsional son capaces de generar conformaciones de moléculas pequeñas estadísticamente indistinguibles de las experimentales, superando a los métodos basados en campos de fuerza clásicos como ETKDG en términos de diversidad conformacional y cobertura de la distribución. Estos trabajos establecieron la prueba de concepto de que el formalismo de difusión torsional es aplicable a sistemas moleculares reales y sientan las bases metodológicas para su extensión a proteínas.

2.4.4. DiffPack: difusión torsional para cadenas laterales

DiffPack [25] es el trabajo más directamente relacionado con el presente TFM. Zhang et al. propusieron un modelo de difusión torsional específicamente diseñado para la recuperación de cadenas laterales proteicas. Su arquitectura combina una red neuronal de grafos equivariante (EGNN) para el encoding del backbone con un proceso de difusión sobre el espacio de ángulos χ . El modelo aprende la distribución condicional

$P(\chi_1, \dots, \chi_4 \mid \text{backbone})$ en lugar del rotámero de máxima probabilidad, permitiendo el muestreo de conformaciones diversas.

Los resultados reportados en DiffPack superan a SCWRL4 en todas las métricas evaluadas: tasa de recuperación de χ_1 del 87,5 % (frente al 83,3 % de SCWRL4), mejora notable en residuos de superficie y mayor robustez en regiones de alta flexibilidad. El código fuente está disponible públicamente en GitHub, aunque su arquitectura completa es computacionalmente costosa. El presente TFM implementa una versión propia simplificada (“lite”) e inspirada en DiffPack adaptada al entorno de Google Colab con GPU T4.

2.5. Redes neuronales de grafos equivariantes para proteínas

La representación de proteínas como grafos moleculares, donde los residuos son nodos y los contactos o vecindades secuenciales son aristas, ha impulsado el desarrollo de arquitecturas de redes neuronales especializadas. Un requisito fundamental en este dominio es la **equivariancia** respecto a las simetrías del espacio físico (rotaciones, traslaciones y reflexiones del grupo $E(3)$, Espacio euclidiano en 3D): las predicciones de coordenadas atómicas deben transformarse de forma coherente cuando la proteína es rotada o trasladada en el espacio.

2.5.1. EGNN: redes de grafos equivariantes

Satorras et al. [26] propusieron la **Equivariant Graph Neural Network (EGNN)**, una arquitectura que garantiza la equivariancia $E(3)$ sin recurrir a representaciones tensoriales de orden superior, costosas computacionalmente. La actualización de la representación de cada nodo combina features escalares (invariantes) y coordenadas vectoriales (equivariantes) de forma separada. EGNN es la arquitectura base utilizada en DiffPack y es una opción natural para el presente proyecto por su equilibrio entre expresividad y eficiencia computacional.

2.5.2. SchNet y PaiNN: convoluciones continuas en espacio

SchNet [27] introdujo las convoluciones continuas en espacio para sistemas moleculares, donde los filtros de convolución son funciones continuas de la distancia interatómica. PaiNN [28] extendió este enfoque incluyendo representaciones vectoriales equivariantes, mejorando la capacidad del modelo para capturar la geometría direccional. Ambas arquitecturas han demostrado alto rendimiento en tareas de

predicción de propiedades moleculares y son alternativas viables a EGNN para el encoding del backbone.

2.5.3. GVP-GNN: redes de vectores geométricos

Jing et al. [29] desarrollaron las **Geometric Vector Perceptron GNNs (GVP-GNN)**, una arquitectura que opera simultáneamente sobre features escalares y vectoriales, garantizando la equivariancia mediante una separación explícita de los canales de procesamiento. GVP-GNN fue diseñada específicamente para proteínas y ha demostrado alta eficiencia en tareas de predicción de estructura inversa (design) y modelado de interacciones proteína-proteína. Es la arquitectura principal de ProteinMPNN [30], uno de los métodos de referencia en diseño computacional de secuencias proteicas.

2.6. AlphaFold3 y la convergencia hacia la difusión all-atom

La publicación de AlphaFold3 [31] en mayo de 2024 marcó un hito en el campo al adoptar un módulo de difusión all-atom como componente central de la predicción estructural. A diferencia de AlphaFold2, que predice el backbone mediante representaciones invariantes y añade las cadenas laterales como postproceso, AlphaFold3 genera directamente las coordenadas atómicas de toda la proteína (incluyendo cadenas laterales, ligandos, ácidos nucleicos y moléculas de agua) mediante un proceso de difusión en el espacio de coordenadas cartesianas.

Este cambio arquitectónico tiene implicaciones profundas. En primer lugar, valida la tesis de que los modelos de difusión son el enfoque adecuado para capturar la naturaleza estocástica y multimodal de las conformaciones de cadenas laterales. En segundo lugar, establece el marco de referencia para los modelos especializados: mientras AlphaFold3 opera en el espacio cartesiano con un proceso de difusión gaussiano, los modelos de difusión torsional como DiffPack operan en el espacio de ángulos diedros, lo que los hace más eficientes para la tarea específica de recuperación de cadenas laterales dado un backbone conocido.

Recientemente, Chai-1 [32] y Boltz-1 [33] han propuesto modelos alternativos de difusión all-atom con arquitecturas más eficientes y resultados comparables a AlphaFold3 en los benchmarks estándar. La convergencia de múltiples grupos hacia la difusión como paradigma de predicción estructural confirma la relevancia metodológica del enfoque adoptado en este TFM.

2.7. Estadística circular aplicada a ángulos diedros

El análisis estadístico de ángulos diedros requiere el uso de **estadística circular**, una rama de la estadística especializada en el tratamiento de variables definidas sobre el círculo S^1 [34]. Las métricas estándar de estadística euclidiana no son aplicables directamente a variables angulares: la media aritmética de 350° y 10° es 180° , mientras que la media circular correcta es 0° . De forma análoga, la distancia entre dos ángulos debe calcularse como la distancia geodésica sobre el círculo, no la diferencia absoluta. La **distribución de Von Mises** es la distribución de referencia en estadística circular, análoga a la distribución normal en el caso lineal [35]. Está parametrizada por su media circular μ y su parámetro de concentración κ (el recíproco de la dispersión). Para valores altos de κ , la distribución se concentra en torno a μ y se aproxima a una gaussiana; para $\kappa \rightarrow 0$, se aproxima a la distribución uniforme sobre el círculo.

En el contexto de la evaluación de modelos de predicción de ángulos χ , las métricas estadísticas estándar requieren adaptación circular. El Error Absoluto Medio (MAE) se define como la distancia circular media: $d(\theta_1, \theta_2) = \pi - |\pi - |\theta_1 - \theta_2||$. La tasa de recuperación de rotámeros se calcula con un umbral circular de $\pm 20^\circ$ o $\pm 40^\circ$ respecto al valor experimental. El uso de histogramas polares (rose plots) como herramienta de visualización de distribuciones angulares es la práctica estándar en bioinformática estructural [36], ya que representa correctamente la periodicidad de los datos y facilita la identificación visual de los estados rotaméricos canónicos.

2.8. Resumen comparativo de métodos

La Tabla 3 resume las características principales de los métodos más relevantes en el contexto de este TFM, ordenados cronológicamente.

Tabla 3. Comparativa de métodos de predicción de cadenas laterales y predicción estructural.

Método	Año	Paradigma	Cadenas laterales	Limitación principal
SCWRL4	2009	Librería rotámeros + belief propagation	$\chi_1 \sim 83 \%$	Predicción determinista; sin incertidumbre
Rosetta	2001–20	Campo de fuerza + Monte Carlo	$\chi_1 \sim 84 \%$	Alto coste computacional
AlphaFold2	2021	Deep learning (Evoformer)	$\chi_1 \sim 70\text{--}75 \%$	Predicción puntual; sin distribución conformacional
RoseTTAFold	2021	Deep learning (triple-track)	Similar a AF2	Sin modelado de incertidumbre
DiffPack	2023	Difusión torsional + EGNN	$\chi_1 \sim 87,5 \%$	Alto coste; no accesible en Colab nativo
AlphaFold3	2024	Difusión all-atom cartesiana	SOTA all-atom	Modelo cerrado; no reproducible
Este TFM	2026	Difusión torsional lite + GNN secuencial	En evaluación	Versión simplificada orientada a bioestadística

La tabla pone de manifiesto la brecha existente entre los métodos clásicos (deterministas, sin modelado de incertidumbre) y los métodos basados en difusión (generativos, capaces de muestrear distribuciones conformacionales). El presente TFM se sitúa en este segundo paradigma, con el objetivo adicional de enfatizar la evaluación bioestadística rigurosa mediante estadística circular y análisis de calibración de

confianza, aspectos frecuentemente ausentes en la literatura de ingeniería computacional.

3. Materiales y métodos

A continuación, se explican los materiales y métodos utilizados en este proyecto, dividiéndolos en las 3 fases de desarrollo.

3.1. Fase 1: Cimentación y preparación de los datos

3.1.1. Panel de control del pipeline

Dado que las sesiones de Google Colab son efímeras y el pipeline completo requiere aproximadamente 3–4 horas de cómputo en su primera ejecución, se implementó un mecanismo de control de re-ejecución selectiva que permite reanudar el trabajo en sesiones posteriores sin necesidad de regenerar los artefactos ya disponibles en Drive.

El Bloque 0 define un conjunto de variables booleanas (*flags*) que determinan qué bloques costosos deben ejecutarse de nuevo y cuáles pueden omitirse cargando directamente el resultado almacenado en Google Drive. Este panel se estructura en tres niveles:

El primero es un switch maestro (`FORCE_ALL`), que cuando se establece a `True` fuerza la re-ejecución completa del pipeline desde cero, ignorando cualquier artefacto previo en Drive. Es el modo recomendado para la primera ejecución o cuando se desea garantizar la reproducibilidad total del experimento.

El segundo nivel son los switches individuales por fase (`FORCE_DOWNLOAD`, `FORCE_EXTRACTION`, `FORCE_DATASET`, `FORCE_TRAINING`, `FORCE_INFERENCE`), que permiten forzar la re-ejecución de un único bloque específico sin afectar al resto. Solo tienen efecto cuando `FORCE_ALL` es `False`.

El tercer nivel es la lógica de cascada, que garantiza la coherencia del pipeline: si una fase se re-ejecuta, todas las fases dependientes de ella también se activan automáticamente. Por ejemplo, si se fuerza la re-extracción de ángulos diedros (`FORCE_EXTRACTION = True`), el dataset de grafos (`FORCE_DATASET`), el entrenamiento (`FORCE_TRAINING`) y la inferencia (`FORCE_INFERENCE`) se re-ejecutan en cascada, evitando que el modelo entrenado quede desincronizado con los datos de entrada.

En la configuración por defecto (todos los flags en `False`), el pipeline detecta los artefactos existentes en Drive y los carga directamente, reduciendo el tiempo de una sesión de reanudación a aproximadamente 9 minutos. Al inicio de cada sesión, el panel

imprime un resumen del estado de cada fase, indicando si será re-ejecutada o cargada desde Drive, lo que facilita la auditoría del estado del experimento antes de lanzar la ejecución completa.

3.1.2. Entorno de computación y software

Todo el pipeline de datos y el entrenamiento del modelo se ejecutaron sobre Google Colaboratory (Colab), una plataforma de computación en la nube que proporciona acceso gratuito a unidades de procesamiento gráfico (GPU). La sesión de cómputo fue configurada con acelerador de hardware **Tesla T4** (15,6 GB de memoria VRAM, CUDA 12.8), verificado al inicio de cada sesión mediante la función `torch.cuda.is_available()`. El uso de GPU es un requisito no negociable del proyecto: tanto el proceso de difusión torsional como el entrenamiento de la red neuronal de grafos son computacionalmente inviables sobre CPU para el volumen de datos utilizado.

Dado que las sesiones de Colab son efímeras (los archivos locales se pierden al desconectarse), se montó Google Drive como sistema de ficheros persistente mediante la API `google.colab.drive`. Se definió una estructura de directorios centralizada bajo la carpeta `MyDrive/TFM_SideChain/` con tres rutas globales: (1) `pdb_data/` para los archivos estructurales descargados, (2) `dataset_sidechains_curated.csv` para el dataset tabular legible, y (3) `dataset_torsional_diffusion.pt` para el dataset serializado de grafos. Esta centralización garantiza que todos los bloques del notebook compartan las mismas rutas y que el pipeline pueda reanudarse tras un reinicio de sesión sin pérdida de datos. Las dependencias de software se instalaron con versiones fijadas para garantizar reproducibilidad:

- **PyTorch 2.10.0** con soporte CUDA 12.8 como framework principal de deep learning.
- **PyTorch Geometric** (incluyendo `torch-scatter` y `torch-sparse` compilados contra las versiones exactas de PyTorch y CUDA) para la representación y procesamiento de grafos moleculares.
- **Biopython** para el parsing de archivos PDB, extracción de vectores atómicos y cálculo de ángulos diedros mediante `Bio.PDB.calc_dihedral`.
- **py3Dmol** para la visualización interactiva en 3D de estructuras proteicas directamente en el notebook.
- **NumPy** y **Pandas** para el procesamiento numérico y la manipulación del dataset tabular.

- **Matplotlib** para la generación de figuras estadísticas, en particular los histogramas circulares (rose plots) de distribución rotamérica.

3.1.3. Selección del dataset y criterios de calidad

Las estructuras proteicas de entrenamiento se obtuvieron consultando la API REST de búsqueda avanzada del Protein Data Bank (RCSB PDB) mediante peticiones POST al endpoint <https://search.rcsb.org/rcsbsearch/v2/query>. La selección no fue arbitraria: se aplicaron tres filtros en combinación lógica AND, cada uno con justificación estructural y estadística explícita.

- **Filtro 1 — Resolución cristalográfica < 2,0 Å.** Las posiciones atómicas de las cadenas laterales solo son fiables por debajo de este umbral. A resoluciones superiores, los ángulos diedros experimentales incorporan un error asociado a la incertidumbre en las coordenadas atómicas que contaminaría el entrenamiento del modelo. Este criterio es el estándar en la literatura de predicción de rotámeros [37].
- **Filtro 2 — Método experimental: cristalografía de rayos X.** Se excluyeron estructuras determinadas por RMN y Cryo-EM. La cristalografía de rayos X es el estándar de oro para la determinación de ángulos diedros de cadenas laterales, ya que proporciona posiciones atómicas promediadas sobre el cristal con menor ambigüedad conformacional que el RMN (donde los ángulos representan promedios de ensemble en solución).
- **Filtro 3 — Tipo de polímero: proteína.** Sin este filtro, la query devuelve estructuras de ácidos nucleicos (ADN/ARN) y complejos mixtos que no poseen ángulos chi de aminoácidos. Su inclusión contaminaría el dataset con filas vacías y degradaría la calidad del entrenamiento.

La consulta recuperó un total de 1.000 identificadores PDB. Posteriormente, los archivos estructurales se descargaron desde **PDB-REDO** en lugar del PDB original. PDB-REDO es una base de datos que re-refina automáticamente cada estructura usando algoritmos modernos de refinamiento cristalográfico (REFMAC5), produciendo coordenadas atómicas más precisas con mejor geometría de enlace y menor número de errores de empaquetamiento. Para un modelo de difusión torsional, entrenarse sobre ángulos chi más precisos se traduce directamente en predicciones más exactas. Se descargó el archivo {pid}_final.pdb de cada entrada, que representa la estructura completamente re-refinada. De los 1.000 IDs consultados, 598 tenían versión disponible en PDB-REDO (el resto eran entradas muy antiguas o con problemas de refinamiento) y de estos, 597 fueron descargados correctamente, sin error en el procesamiento.

3.1.4. Ingeniería de características (Feature engineering)

Cada archivo PDB descargado fue procesado con Biopython para extraer, por residuo, los dos tipos de información que constituyen el dataset: las características de entrada (features X) que el modelo recibirá como input, y los ángulos diedros objetivo (targets Y) que deberá predecir.

- **Features X — Coordenadas 3D del backbone.** Para cada residuo válido se extrajeron las coordenadas cartesianas (x, y, z) de los tres átomos principales de la cadena principal: N (nitrógeno amídico), C α (carbono alfa) y C (carbono carbonílico). Esto produce un tensor de dimensiones [N_residuos $\times$ 9] por proteína, que actúa como representación geométrica del nodo en el grafo molecular. El backbone se asume conocido y fijo, emulando el escenario real en que AlphaFold2 predice el backbone con alta precisión, pero las cadenas laterales permanecen inciertas.
- **Targets Y — Ángulos diedros chi (χ_1 – χ_4).** Los cuatro ángulos diedros de la cadena lateral se calcularon mediante Bio.PDB.calc_dihedral() sobre los vectores atómicos correspondientes. La dependencia anidada entre ángulos respeta la estructura química real: χ_1 requiere los átomos N–C α –C β –C γ ; χ_2 requiere χ_1 y los átomos C α –C β –C γ –C δ ; y así sucesivamente. Si un átomo intermedio no existe en la estructura, los ángulos superiores se registran como NaN. Esto es biológicamente correcto: serina solo tiene χ_1 , glutamato tiene χ_1 – χ_3 , y arginina y lisina tienen χ_1 – χ_4 .

Se aplicaron los siguientes filtros de calidad sobre los residuos individuales:

- **Exclusión de Glicina y Alanina:** Glicina carece de C β , por lo que χ_1 es imposible de calcular. Alanina tiene C β pero no posee cadena lateral extensa; χ_1 es geoméricamente indefinido. Ambos residuos se excluyeron del dataset.
- **Backbone completo obligatorio:** Los residuos con alguno de los átomos N, C α , C o C β ausente en la estructura fueron descartados. Sin estas cuatro posiciones no es posible construir el nodo del grafo ni calcular χ_1 .
- **Orden secuencial garantizado:** Los residuos de cada cadena se ordenaron por número de secuencia antes del procesamiento. Sin este paso, el grafo molecular construido en la etapa siguiente conectaría residuos en orden arbitrario de lectura del PDB, no en orden peptídico real, lo que introduciría aristas incorrectas.
- **Filtro post-extracción:** Se eliminaron del DataFrame final todos los residuos sin valor de χ_1 (el mínimo necesario para cualquier análisis rotamérico), obteniendo el dataset curado definitivo.

Las coordenadas y ángulos de todos los residuos válidos de todas las proteínas se consolidaron en un único DataFrame de Pandas con las columnas: `pdb_id`, `residue_name`, `residue_idx`, `N_x/y/z`, `CA_x/y/z`, `C_x/y/z`, `chi1`, `chi2`, `chi3`, `chi4`. Este archivo CSV intermedio se guardó en Drive como respaldo tabular legible.

A continuación, el DataFrame se transformó en objetos `torch_geometric.data.Data` —un grafo por proteína— con la siguiente estructura interna:

- **Nodos:** cada residuo es un nodo con features $x [N \times 9]$ (coords backbone) y `aa_type [N]` (tipo de aminoácido codificado como entero 0–19; 20 = desconocido).
- **Aristas:** `edge_index [2 × 2(N–1)]` conecta residuos secuencialmente contiguos en ambas direcciones, modelando la cadena polipeptídica como un grafo lineal no dirigido.
- **Targets:** `y [N × 4]` contiene los ángulos chi (NaN sustituido por 0,0) y `y_mask [N × 4]` (booleana) indica qué posiciones tienen valor experimental real.

3.1.5. Partición del dataset

La partición del dataset en conjuntos de entrenamiento, validación y test se realizó a nivel de proteína (no de residuo). Esta decisión es estadísticamente crítica: si la partición se hiciera a nivel de residuo, residuos de la misma proteína podrían aparecer simultáneamente en train y en test, introduciendo **data leakage**, **fuga de datos**, que inflaría artificialmente las métricas de evaluación. Al dividir por proteína, se garantiza que el modelo es evaluado sobre estructuras completamente inéditas durante el entrenamiento.

Antes de la división, la lista de grafos fue aleatorizada con semilla fija (`random.seed(42)`) para garantizar la reproducibilidad del split. La partición se realizó con proporciones 80/10/10:

- **Train (80 %):** 477 proteínas. Utilizado para la actualización de parámetros del modelo mediante descenso de gradiente.
- **Validación (10 %):** 60 proteínas. Utilizado para el seguimiento de la pérdida durante el entrenamiento y la selección del mejor checkpoint de modelo.
- **Test (10 %):** 60 proteínas. Reservado para la evaluación final de métricas no vistas durante el desarrollo del modelo.

El dataset serializado se guardó en Drive en formato `.pt` mediante `torch.save()` como un diccionario con tres claves ('train', 'val', 'test'). Este fichero único es la entrada directa del bucle de entrenamiento de la GNN. Los DataLoaders de PyTorch Geometric cargan los

grafos con `batch_size=2`, ajustado al límite de memoria VRAM de la GPU T4 y al tamaño variable de los grafos proteicos. Para el conjunto de entrenamiento se activa `shuffle=True` en cada época; para validación y test, `shuffle=False` para garantizar la comparabilidad de métricas entre épocas.

3.2. Fase 2: Implementación del modelo de IA

La segunda fase del proyecto comprende el diseño e implementación del modelo de inteligencia artificial encargado de predecir las conformaciones de las cadenas laterales. Esta fase integra tres componentes computacionalmente interdependientes: el proceso forward de difusión torsional, la arquitectura de la red neuronal de grafos (GNN) y el bucle de entrenamiento supervisado con optimización mediante descenso de gradiente.

3.2.1. Proceso forward de difusión torsional

El modelo sigue el paradigma de los modelos de difusión generativos, que aprenden a invertir un proceso estocástico de degradación progresiva de la señal. En el dominio angular, el proceso forward consiste en añadir ruido Von Mises a los ángulos χ_0 reales para obtener versiones progresivamente ruidosas χ_t . La distribución Von Mises es la análoga circular de la gaussiana y garantiza que el resultado sigue siendo un ángulo válido en el rango $[-\pi, \pi]$.

El proceso se controla mediante el parámetro de concentración κ , que controla la dispersión del ruido. Se implementó un schedule lineal: para $t = 0$, κ alcanza su valor máximo ($KAPPA_MAX = 10,0$), lo que produce una distribución muy concentrada prácticamente idéntica al ángulo original; para $t = T = 100$, κ cae a su valor mínimo ($KAPPA_MIN = 0,01$), aproximando una distribución uniforme sobre el círculo (ruido puro sin información). Este número de pasos $T = 100$ es una versión simplificada respecto a los $T = 1\,000$ del modelo DiffPack original, ajustada a las restricciones computacionales de Google Colab.

El ruido se aplica únicamente sobre las posiciones con valor angular experimental real, identificadas mediante la máscara booleana `y_mask` generada en la fase anterior. Las posiciones sin dato (NaN en el dataset tabular, sustituidos por 0,0 en los grafos) permanecen inalteradas para no contaminar el entrenamiento. La correcta propagación del ruido se verificó mediante un test de sanidad cuantitativo: la distancia angular circular

media (MAE circular) entre χ_0 y χ_t debe aumentar de forma monótona con t , lo que confirma que el proceso forward inyecta ruido de forma progresiva y controlada.

Para la visualización cualitativa del efecto del ruido, se implementó la función `visualize_noise_effect`, que modifica los ángulos χ_1 de los residuos de una proteína real usando la librería Biopython y renderiza la estructura tridimensional resultante con `py3Dmol` para $t \in \{0, 25, 50, 100\}$. Esta visualización ilustra cómo las cadenas laterales se desordenan progresivamente al aumentar t , representando visualmente el problema de denoising que el modelo debe aprender a invertir.

3.2.2. Preparación del batch de difusión para el entrenamiento

En cada iteración del bucle de entrenamiento, la función `prepare_diffusion_batch` transforma un batch de grafos con ángulos reales en los tensores necesarios para el aprendizaje supervisado. Para cada grafo del batch se muestrea aleatoriamente un paso de tiempo $t \in [1, T]$ mediante la función `sample_t`, se aplica el proceso forward para obtener los ángulos ruidosos χ_t , y se codifica t como feature adicional de nodo normalizada al rango $[0, 1]$. Muestrear un valor de t diferente por grafo equivale a entrenar simultáneamente en todos los niveles de ruido del schedule, lo que estabiliza el aprendizaje e impide que el modelo se especialice en un único nivel de degradación.

La función de pérdida empleada es la pérdida circular (`circular_loss`), definida como $1 - \cos(\theta_{pred} - \theta_{real})$, calculada exclusivamente sobre las posiciones con máscara activa. La distancia circular así definida es continua, diferenciable y respeta la periodicidad de los ángulos: el error entre 179° y -179° es de 2° , no de 358° como ocurriría con un MSE clásico. Este aspecto es crítico para el entrenamiento correcto de modelos sobre ángulos diedros.

3.2.3. Arquitectura de la red neuronal de grafos: ChiGNN

El modelo de denoising implementado, denominado ChiGNN, es una red neuronal de grafos que toma como entrada el grafo proteico con ángulos ruidosos χ_t y el paso de tiempo t , y predice los ángulos originales χ_0 . La arquitectura fue diseñada específicamente para este proyecto y se estructura en cuatro módulos funcionales.

El primer módulo es la construcción del vector de features del nodo. Cada residuo es representado por la concatenación de: (1) sus 9 coordenadas de backbone (N, C α , C);

(2) un embedding aprendible de 16 dimensiones del tipo de aminoácido, generado por una capa `nn.Embedding(21, 16)` que permite al modelo descubrir relaciones de similitud rotamérica entre residuos; (3) la codificación sin/cos de los cuatro ángulos chi ruidosos, que produce 8 features continuas respetando la periodicidad angular; y (4) el paso de tiempo t normalizado como feature escalar. El vector resultante tiene una dimensión de entrada de 34 features por nodo.

El segundo módulo es la proyección inicial, que mapea los 34 features de entrada al espacio oculto de 128 dimensiones mediante una capa lineal seguida de activación SiLU. Se eligió SiLU (Sigmoid Linear Unit) frente a ReLU por su suavidad y ausencia del efecto “dying ReLU”, que puede paralizar neuronas en redes profundas.

El tercer módulo consiste en 4 capas de convolución en grafo (GCNConv de PyTorch Geometric), cada una seguida de BatchNorm1d y activación SiLU, con conexiones residuales $h \leftarrow h + h_{\text{nuevo}}$ entre capas consecutivas. Las conexiones residuales evitan el desvanecimiento del gradiente en capas profundas. Se escogió GCNConv sobre alternativas más complejas como EGNN porque las coordenadas del backbone ya están incluidas en las features de entrada, por lo que el modelo puede aprender las relaciones geométricas directamente sin necesidad de equivariancia estricta; además, GCNConv requiere aproximadamente tres veces menos memoria que EGNN, lo que es determinante dado el límite de 15,6 GB de VRAM de la GPU T4.

El cuarto módulo es la cabeza de predicción, un MLP de dos capas que mapea el embedding oculto de 128 dimensiones a 4 valores angulares. Una activación $\tanh \times \pi$ en la última capa fuerza la salida al rango $[-\pi, \pi]$, que es el dominio válido de los ángulos diedros. El modelo cuenta con un total de 80.404 parámetros entrenables.

3.2.4. Bucle de entrenamiento

El modelo se entrenó durante 50 épocas con un tamaño de batch de 2 grafos, ajustado al límite de memoria de la GPU T4. El optimizador empleado fue AdamW con tasa de aprendizaje inicial $LR = 10^{-3}$ y decaimiento de pesos de 10^{-4} . AdamW incorpora weight decay desacoplado, lo que regulariza los pesos sin afectar al sesgo y mejora la generalización respecto a Adam estándar. El scheduler de tasa de aprendizaje fue CosineAnnealingLR con $\text{eta_min} = 10^{-5}$, que reduce la tasa siguiendo una curva coseno hasta prácticamente cero al final del entrenamiento, evitando oscilaciones en el último tramo.

Cada época se divide en dos fases. En la fase de entrenamiento, para cada batch se preparan los ángulos ruidosos mediante `prepare_diffusion_batch`, se ejecuta el forward pass del modelo ChiGNN, se calcula la pérdida circular y se actualizan los pesos mediante retropropagación. Se aplica gradient clipping con norma máxima de 1,0 para prevenir la explosión del gradiente, un problema frecuente en GNNs con conexiones residuales. En la fase de validación, el modelo evalúa el conjunto de validación en modo `torch.no_grad()` sin actualizar parámetros, registrando la pérdida media como criterio de selección del mejor checkpoint.

El mejor checkpoint del modelo se guarda en Google Drive en formato `.pt` siempre que la pérdida de validación mejore respecto al mínimo histórico, almacenando el estado del modelo, el optimizador, la época y el historial de pérdidas. Este mecanismo garantiza que el modelo final utilizado para la evaluación corresponde al menor `val_loss` registrado durante el entrenamiento, y no necesariamente al de la última época.

La curva de aprendizaje se visualizó mediante Matplotlib superponiendo las trayectorias de pérdida de entrenamiento y validación a lo largo de las épocas. Se implementó además un diagnóstico automático que evalúa el gap entre ambas pérdidas al final del entrenamiento: un gap superior a 0,05 señala riesgo de sobreajuste, mientras que una pérdida de entrenamiento superior a 0,5 indica infraajuste.

3.2.5. Proceso reverse: muestreo de cadenas laterales

El proceso reverse implementa el muestreo iterativo de conformaciones: partiendo de ángulos chi uniformemente distribuidos (ruido puro en $t = T$), aplica `n_steps = 20` pasos de denoising mediante el modelo ChiGNN entrenado para recuperar conformaciones plausibles. En cada paso t , el modelo predice χ_0 a partir de χ_t , y la predicción se refina añadiendo ruido con κ_{t-1} para simular el paso anterior de la cadena de Markov, siguiendo una adaptación del algoritmo DDIM. La función se ejecuta bajo `torch.no_grad()` para evitar la acumulación de grafos computacionales innecesarios durante la inferencia.

El proceso reverse se ejecutó sobre todos los grafos del conjunto de test, generando un tensor de predicciones `[N_test, 4]` que, junto con los targets y las máscaras, se serializó en Drive como `test_predictions.pt`. Este archivo es la entrada directa de todos los bloques de evaluación de la Fase 3, permitiendo reutilizar las predicciones sin necesidad de re-ejecutar el proceso reverse en cada nueva sesión.

3.3. Fase 3: Evaluación bioestadística

La tercera fase del proyecto comprende la evaluación rigurosa del modelo ChiGNN sobre el conjunto de test, empleando un conjunto de métricas estadísticas especialmente diseñadas para la naturaleza circular de los ángulos diedros. La evaluación se articula en cinco bloques complementarios: cálculo de métricas base, análisis por tipo de aminoácido, comparación de distribuciones rotaméricas, calibración de confianza y benchmarking contra baselines.

3.3.1. Métricas de evaluación

La métrica principal de evaluación es el error absoluto medio circular (MAE circular), calculado mediante la función `circular_mae_deg` como el promedio de $|\tan 2(\sin(a - b), \cos(a - b))|$ sobre todas las posiciones enmascaradas del conjunto de test, convertido a grados. La distancia circular así definida respeta la periodicidad: como se ha dicho, el error entre 179° y -179° es de 2° , no de 358° como devolvería un MAE euclídeo. Este estimador es el estándar en la literatura de predicción de rotámeros para comparar con métodos como SCWRL4 o DiffPack.

La segunda métrica es la tasa de recuperación de rotámeros (`rotamer_recovery_rate`), definida como el porcentaje de ángulos predichos cuya distancia circular al valor experimental es inferior a un umbral dado. Se calculan dos umbrales complementarios: $\pm 20^\circ$ (criterio estricto empleado en benchmarks competitivos, equivalente a recuperar el rotámero exacto) y $\pm 40^\circ$ (criterio permisivo que refleja la tolerancia funcional real en la mayoría de aplicaciones de modelado estructural). Estas métricas se calcularon para cada uno de los cuatro ángulos chi (χ_1 – χ_4) de forma independiente, considerando únicamente las posiciones con dato experimental confirmado.

3.3.2. Análisis de error por tipo de aminoácido

El MAE circular de χ_1 se desglosó por los 18 tipos de aminoácidos presentes en el dataset (recordando que glicina y alanina fueron excluidos en la fase de datos). Este análisis es estadísticamente relevante porque cada aminoácido posee un perfil rotamérico propio: residuos con conformaciones de cadena lateral bien definidas (PHE, TYR, TRP) presentan típicamente MAE menor, mientras que residuos con alta flexibilidad conformacional (LYS, ARG) muestran MAE mayor. Se excluyeron del cálculo

los aminoácidos con menos de cinco muestras en el conjunto de test para garantizar la fiabilidad estadística del estimador.

Los resultados se visualizaron mediante un panel de dos gráficos de barras horizontales. El panel izquierdo muestra el MAE por aminoácido, ordenado de menor a mayor error, con un gradiente de color verde-rojo (biblioteca RdYlGn de Matplotlib) y líneas de referencia en 20° y 40°. El panel derecho muestra la tasa de recuperación $\pm 40^\circ$, con una línea de referencia al 83% correspondiente al rendimiento reportado de SCWRL4. Ambas figuras se exportaron en formato PNG a 150 dpi para su inclusión en la memoria de la tesis.

3.3.3. Validación visual de distribuciones rotaméricas

Se generaron histogramas polares (rose plots) superponiendo las distribuciones de ángulos chi predichos y experimentales para cada uno de los cuatro ángulos (χ_1 – χ_4). Esta es la validación visual más importante del proyecto: si el modelo ha aprendido correctamente los patrones rotaméricos, los picos de la distribución predicha deben coincidir con los de la distribución experimental en las posiciones canónicas (-60° , $+60^\circ$, 180°), que corresponden a las conformaciones *gauche*⁻, *gauche*⁺ y *trans* respectivamente.

Cada rose plot divide el círculo en 36 bins de 10° y normaliza ambas distribuciones por su máximo para facilitar la comparación de formas independientemente del tamaño de muestra. Una distribución predicha demasiado uniforme indica que el modelo no aprendió los rotámeros; una distribución predicha más estrecha que la experimental señala sobreconfianza (underdispersion). El eje angular del plot circular fue configurado con 0° en la parte superior y sentido horario porque es la convención estándar en bioinformática estructural.

3.3.4. Análisis de calibración de confianza

Un modelo bien calibrado debe mostrar una correlación positiva entre su incertidumbre interna y el error real: las predicciones sobre las que el modelo es más incierto deben corresponder a residuos con mayor error experimental. Para cuantificar esta propiedad, se generaron $K = 10$ muestras independientes del proceso reverse por residuo del conjunto de test y se calculó la varianza circular de Von Mises de dichas muestras como estimador de la incertidumbre del modelo.

La varianza circular se define como $V = 1 - |\langle \exp(i\theta) \rangle|$, donde i es la unidad imaginaria. Este estimador es invariante a la periodicidad y devuelve valores en $[0, 1]$: $V \approx 0$ indica muestras muy concentradas (baja incertidumbre) y $V \approx 1$ indica distribución uniforme sobre el círculo (máxima incertidumbre). Se calculó el coeficiente de correlación de Spearman entre la varianza circular y el MAE real por residuo mediante `scipy.stats.spearmanr`. Se optó por la correlación de Spearman frente a la de Pearson por ser no paramétrica y más robusta a la distribución asimétrica del MAE angular.

Los resultados de calibración se visualizaron mediante dos paneles: un gráfico de dispersión varianza vs. MAE con línea de tendencia, y un gráfico de barras del error medio por decil de incertidumbre. Un modelo bien calibrado debe producir barras crecientes en el segundo panel, lo que indicaría que residuos con alta incertidumbre predicha tienen sistemáticamente mayor error real. Los parámetros de calibración (varianza circular, error por residuo y coeficiente de Spearman) se exportaron a Drive para su uso posterior.

3.3.5. Benchmarking y tabla resumen

El rendimiento de ChiGNN se comparó con dos baselines de referencia. El primer baseline corresponde a la predicción aleatoria uniforme en $[-180^\circ, 180^\circ]$, que representa el peor caso posible y permite cuantificar en cuánto supera el modelo a un sistema sin capacidad de aprendizaje. El segundo baseline es la predicción por rotámero modal: para cada aminoácido, se asigna siempre el ángulo χ_1 más frecuente en el conjunto de entrenamiento, calculado a partir de los histogramas de 36 bins de 10° por aminoácido. Este baseline aproxima el comportamiento de métodos determinísticos como SCWRL4 sin el refinamiento energético, siendo el punto de comparación más honesto disponible en el entorno Google Colab, que no permite instalaciones de software compilado nativo.

La tabla comparativa presenta, para los tres modelos y los cuatro ángulos chi, el MAE circular y la tasa de recuperación $\pm 40^\circ$. Se acompaña de un panel gráfico de barras agrupadas que facilita la comparación visual. Para el MAE de χ_1 , que es la métrica principal del proyecto, se calculó además un intervalo de confianza bootstrap del 95% mediante $n = 1000$ remuestras con reemplazamiento. Se optó por bootstrap frente a métodos paramétricos porque la distribución del MAE circular no es gaussiana y el tamaño muestral varía entre subgrupos de aminoácidos.

La fase finaliza con la generación de una tabla resumen consolidada que reúne todas las métricas calculadas, incluyendo el MAE de χ_1 con intervalo de confianza, las tasas

de recuperación a $\pm 20^\circ$ y $\pm 40^\circ$ para χ_1 y χ_2 , el coeficiente de Spearman de calibración, el número de parámetros del modelo y las épocas de entrenamiento, junto con los valores de referencia de SCWRL4 reportados en la literatura. Esta tabla constituye el principal entregable cuantitativo del proyecto y la base para la discusión de resultados.

Como validación cualitativa final, se implementó una visualización tridimensional comparativa mediante py3Dmol para tres proteínas del conjunto de test: la estructura original con cadenas laterales experimentales (en verde) se superpone con la estructura predicha por ChiGNN (en azul), mostrando que el modelo produce conformaciones tridimensionalmente coherentes y visualmente similares a las estructuras de referencia. La reconstrucción de coordenadas 3D a partir de los ángulos predichos se realiza modificando χ_1 con la función `apply_chi1_to_residue` ya descrita en la fase de implementación.

4. Resultados

4.1. Resultados Fase 1: Cimentación y preparación de los datos

En esta primera fase se ha establecido un entorno de trabajo en Google Colab utilizando Python (PyTorch Geometric y Biopython) para procesar un conjunto de datos curado de estructuras de alta resolución extraídas de PDB-REDO.

4.1.1. Caracterización del Dataset final

Tras aplicar los criterios de selección descritos en la sección 3.1.3 y el pipeline de extracción de la sección 3.1.4, 598 proteínas tenían versión disponible en PDB-REDO, sin embargo, 1 fue descartada por error de procesamiento, siendo el total de proteínas del dataset 597.

Por tanto, el dataset curado contiene **156.407 residuos válidos** distribuidos en **597 proteínas** únicas. La Tabla 4 resume las estadísticas principales del dataset.

Tabla 4. Estadísticas del dataset curado de estructuras proteicas de alta resolución.

Parámetro	Valor
Proteínas seleccionadas (PDB query)	1.000
Proteínas con versión en PDB-REDO	598 (59,8 %)
Proteínas sin errores de procesamiento	597
Residuos totales (con χ_1 definido)	156.407
Aristas totales en el grafo	311.620
Ángulos chi con valor experimental real	323.501 / 625.628 (51,7 %)
Partición Train / Val / Test	477 / 60 / 60 proteínas

La tasa de ángulos chi reales sobre el total posible (51,7 %) refleja la distribución biológica esperada: χ_1 está definido para todos los residuos del dataset (por construcción del filtro), χ_2 para la mayoría (excepto serina, treonina y cisteína), mientras que χ_3 y χ_4 son exclusivos de residuos con cadenas laterales largas (lisina, arginina,

metionina, glutamato, glutamina). La presencia de aproximadamente el 50 % de valores NaN en la matriz de ángulos justifica el uso de la máscara booleana `y_mask` en la función de pérdida del modelo, que ignorará estas posiciones durante el entrenamiento.

La inspección del grafo de una proteína representativa (1akg) confirmó la coherencia de las dimensiones: 14 residuos (nodos), 26 aristas bidireccionales (conectividad secuencial) y 22 ángulos chi reales sobre un máximo de 56 posibles (14 residuos $\times$ 4 ángulos). Ninguna proteína presentó errores durante la construcción del grafo (0 errores sobre 597 procesadas), lo que valida la robustez del pipeline de extracción.

4.1.2. Análisis de Distribución Rotamérica

La Figura 6 muestra el rose plot (histograma polar) de la distribución del ángulo diedro χ_1 sobre los 156.407 residuos del dataset. Se utilizaron 36 sectores angulares de 10° cada uno, con la orientación convencional (0° en la parte superior, sentido horario).

La distribución observada presenta la trimodalidad característica de los rotámeros canónicos de χ_1 , con acumulación marcada en tres regiones angulares:

- **Gauche- (280° - 300° equivalente a -60° o -80°):** estado rotamérico con el sustituyente gamma en posición gauche negativa respecto al nitrógeno del enlace peptídico. Es la conformación más frecuente en las estructuras cristalográficas de alta resolución, ya que minimiza las interacciones estéricas entre el sustituyente gamma y el grupo amino del backbone.
- **Trans (180°):** estado extendido en el que el sustituyente gamma se sitúa en posición *anti* respecto al nitrógeno. Energéticamente favorable para residuos con cadenas laterales largas (lisina, arginina, glutamato), donde aleja los grupos terminales del backbone y reduce la tensión torsional.
- **Gauche+ (60°)** estado rotamérico en el que el sustituyente gamma ocupa la posición gauche positiva respecto al nitrógeno amídico. Es la conformación minoritaria de las tres, ya que genera mayores impedimentos estéricos con el grupo carbonilo del backbone, y su frecuencia varía considerablemente según el tipo de aminoácido.

Este patrón trimodal es consistente con las librerías de rotámeros de referencia [38] y constituye la primera validación bioestadística del dataset: confirma que los ángulos chi extraídos mediante Biopython son biológicamente coherentes y que el pipeline de extracción no ha introducido sesgos sistemáticos en los valores angulares.

La elección de la representación polar en lugar de un histograma cartesiano convencional es estadísticamente relevante: los ángulos diedros son variables periódicas definidas en el círculo, y su representación en el eje lineal de un histograma

cartesiano introduce un artefacto de corte en los extremos $\pm 180^\circ$ que puede distorsionar la apariencia de la distribución. El rose plot evita este problema y será el formato de referencia para comparar las distribuciones de ángulos predichos versus experimentales en la sección de evaluación del modelo.

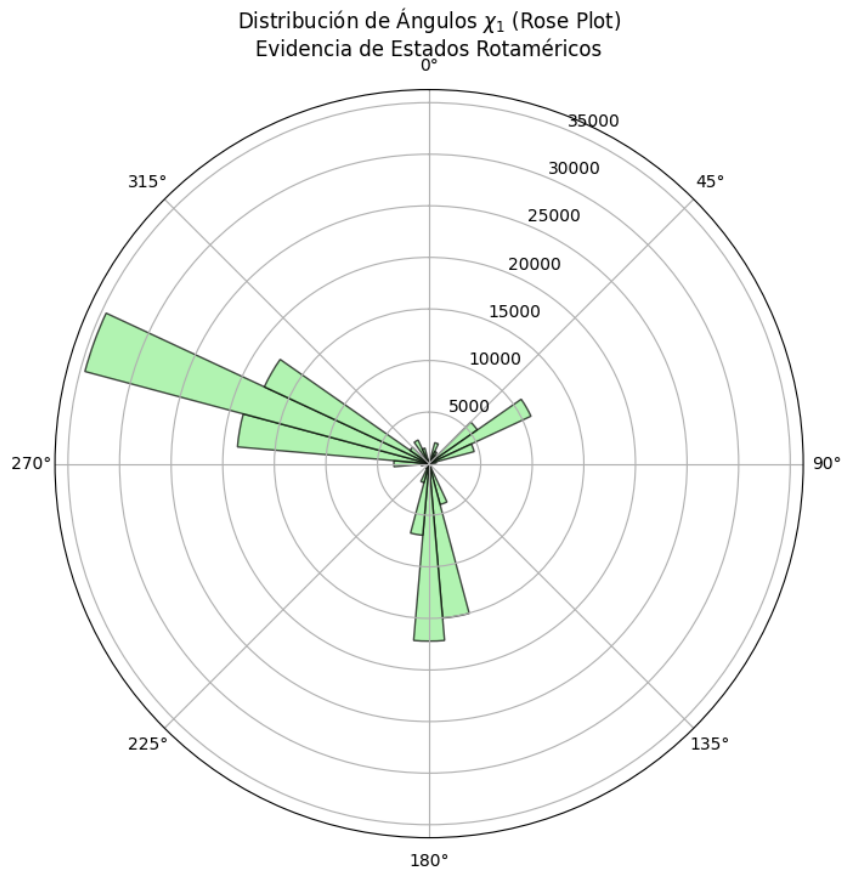


Figura 6. Distribución de ángulos χ_1 de estados rotaméricos.

Fuente: Elaboración propia.

4.1.3. Verificación del Pipeline de Cómputo

Antes de proceder al entrenamiento del modelo, se verificó la integridad del pipeline completo de carga de datos mediante un test de stress sobre el DataLoader de entrenamiento. Se extrajo un único batch de dos proteínas y se inspeccionaron las dimensiones de todos los tensores resultantes (Tabla 5). El test de stress confirma que la cadena completa —serialización en Drive, deserialización, collation de grafos y reindexación de aristas— opera correctamente sin errores.

Tabla 5. Verificación de dimensiones de tensores en un batch de prueba (2 proteínas, 356 residuos totales)

Tensor	Dimensiones	Descripción
x	[356, 9]	Coordenadas backbone (N, C α , C) de los 356 residuos del batch
aa_type	[356]	Tipo de aminoácido codificado (entero 0–20)
y	[356, 4]	Ángulos χ_1 – χ_4 (target del modelo de difusión)
y_mask	[356, 4]	Máscara booleana: 759 / 1.424 posiciones con valor real (53,3 %)
edge_index	[2, 708]	Aristas bidireccionales (354 pares de residuos contiguos $\times$ 2)

La dimensión de edge_index es coherente con la topología esperada: para un batch de dos proteínas con un total de 356 residuos, el número de aristas bidireccionales es $2 \times (356 - 2) = 708$ (descontando los dos extremos de cadena de cada proteína). El valor de 53,3 % de ángulos reales en el batch de prueba es consistente con la tasa global del dataset (51,7 %), lo que confirma que el shuffling, el desorden, no ha introducido sesgos en la distribución de residuos entre batches.

El pipeline de datos queda así completamente validado y listo para alimentar el bucle de entrenamiento de la red neuronal de grafos descrito en la siguiente sección.

4.2. Resultados Fase 2: Implementación del modelo de IA

En esta segunda fase se realiza la implementación del modelo de IA: Se utilizó una arquitectura personalizada “lite” del modelo DiffPack llamada ChiGNN, que se inspira en los principios de DiffPack porque aplica difusión torsional de Von Mises sobre ángulos diedros con una GNN como función de denoising. ChiGNN es una arquitectura GNN diseñada ad hoc para este TFM, inspirada conceptualmente en DiffPack pero implementada de forma independiente y sin reutilización de código del repositorio original. También se describe el proceso forward de Von Mises (verificación cualitativa

y cuantitativa del Schedule de ruido), la arquitectura ChiGNN con sus 80.404 parámetros, la curva de aprendizaje y el proceso reverse sobre el conjunto de test.

La clave técnica reside en el uso de la distribución de Von Mises para modelar el proceso de difusión, lo cual es esencial debido a la naturaleza circular y periódica de los ángulos de las proteínas.

4.2.1. Proceso forward de difusión torsional

La implementación del proceso forward de difusión torsional se realizó sobre el espacio circular S^1 , aplicando ruido distribuido según Von Mises a los ángulos chi reales χ_0 para generar versiones progresivamente degradadas χ_t . El parámetro de concentración κ se redujo linealmente desde $\kappa_{\max} = 10,0$ (distribución muy concentrada, prácticamente idéntica al ángulo original) hasta $\kappa_{\min} = 0,01$ (distribución prácticamente uniforme sobre el círculo) a lo largo de $T = 100$ pasos de difusión.

La validación cuantitativa del proceso forward confirmó su correcto funcionamiento: la distancia angular circular media (MAE circular) entre χ_0 y χ_t aumentó de forma monótona con t , pasando de 0° en $t = 0$ a aproximadamente 90° en $t = 100$ (valor esperado para una distribución uniforme). Este comportamiento monotónico es la condición necesaria para que el modelo de denoising tenga un gradiente de señal bien definido sobre el que aprender.

La verificación cualitativa mediante py3Dmol mostró que para $t \in \{0, 25, 50, 100\}$ las cadenas laterales se desordenan progresivamente de forma coherente: en $t = 0$ la estructura es idéntica a la experimental; en $t = 25$ los rotámeros comienzan a desviarse ligeramente de sus posiciones canónicas; en $t = 50$ la desorientación es notable pero todavía se conservan algunos patrones estructurales; en $t = 100$ las cadenas laterales adoptan posiciones prácticamente aleatorias, representando el estado de ruido puro que el modelo debe aprender a invertir.

4.2.2. Arquitectura ChiGNN y bucle de entrenamiento

El modelo ChiGNN implementado cuenta con 80.404 parámetros entrenables, distribuidos entre el embedding de tipo de aminoácido ($21 \times 16 = 336$ parámetros), la proyección inicial ($34 \rightarrow 128$ dimensiones), cuatro capas GCNConv con BatchNorm y conexiones residuales, y la cabeza de predicción ($128 \rightarrow 4$ valores angulares con activación $\tanh \times \pi$).

El entrenamiento se realizó durante 50 épocas con optimizador AdamW (tasa de aprendizaje inicial 10^{-3} , decaimiento de pesos 10^{-4}) y scheduler CosineAnnealingLR ($\text{eta_min} = 10^{-5}$), sobre el subconjunto de entrenamiento de 477 proteínas con $\text{batch_size} = 2$.

La función de pérdida circular — definida como $1 - \cos(\theta_{\text{pred}} - \theta_{\text{real}})$ — se calculó exclusivamente sobre las posiciones con máscara y_{mask} activa, evitando que las posiciones sin dato experimental (NaN) contribuyeran al gradiente. Se aplicó gradient clipping con norma máxima de 1,0 para estabilizar el entrenamiento en los primeros pasos.

4.2.3. Curva de aprendizaje y diagnóstico del entrenamiento

La Figura 7 muestra la evolución de la pérdida circular de entrenamiento (train loss) y validación (validation loss) a lo largo de las 50 épocas. La línea verde punteada marca la mejor época registrada (época 42, $\text{val_loss} = 0,0885$), correspondiente al checkpoint guardado en Drive que fue utilizado para todas las evaluaciones de la Fase 3.

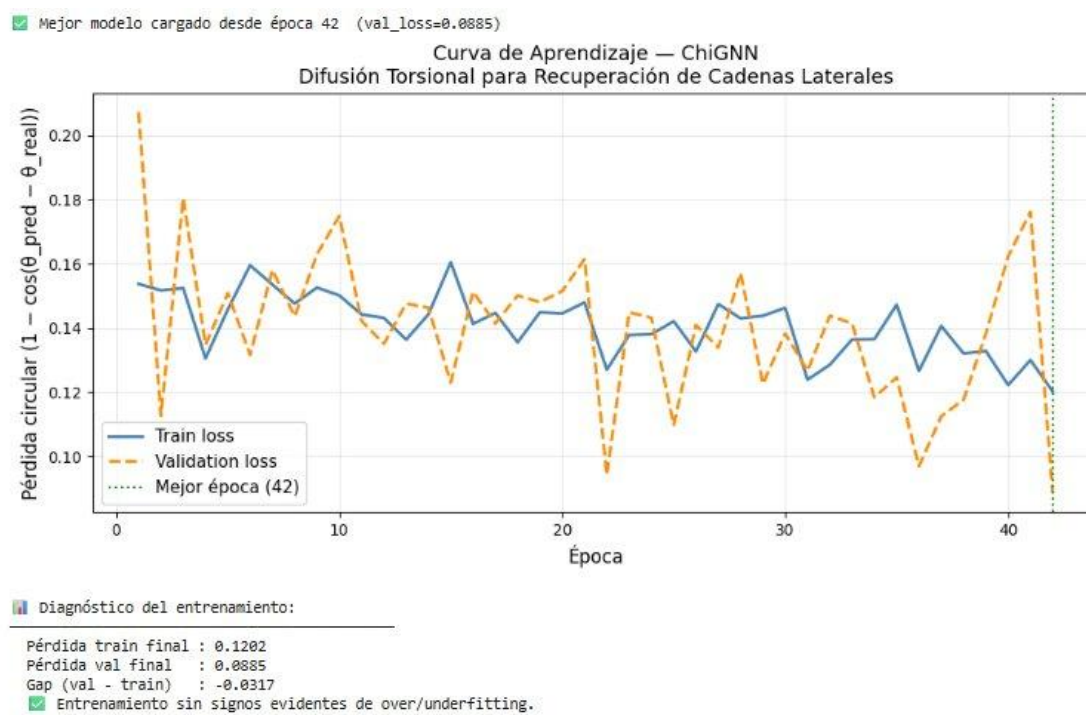


Figura 7. Curva de aprendizaje del modelo ChiGNN. Pérdida circular ($1 - \cos(\theta_{\text{pred}} - \theta_{\text{real}})$) de entrenamiento (azul) y validación (naranja) a lo largo de 50 épocas. La línea punteada verde marca el mejor checkpoint (época 42, $\text{val_loss} = 0,0885$).

Fuente: Elaboración propia.

El diagnóstico automático al final del entrenamiento reporta los siguientes valores:

- Pérdida de entrenamiento final: 0,1202
- Pérdida de validación final: 0,0885
- Gap (val – train): -0,0317
- Diagnóstico: entrenamiento sin signos evidentes de over/underfitting.

Varios aspectos de la curva merecen análisis detallado. En primer lugar, la pérdida de validación es sistemáticamente inferior a la de entrenamiento a lo largo de casi todo el proceso. Este patrón —aparentemente contraintuitivo, pues lo habitual es que la pérdida de validación sea mayor— tiene una explicación metodológica: durante el entrenamiento, la pérdida se calcula con los pesos del modelo antes de actualizar los parámetros de BatchNorm en cada batch, mientras que durante la validación el modelo opera con los promedios móviles estabilizados de BatchNorm acumulados durante la época completa. Este efecto es conocido como "BatchNorm discrepancy" y es habitual en arquitecturas con capas de normalización por batch de tamaño reducido [39].

En segundo lugar, ambas curvas presentan oscilaciones de amplitud moderada a lo largo de las épocas, que son características de un tamaño de batch pequeño (`batch_size = 2`): cada batch contiene solo dos proteínas, produciendo estimadores ruidosos del gradiente que generan variabilidad época a época. El scheduler `CosineAnnealingLR` suaviza progresivamente estas oscilaciones al reducir la tasa de aprendizaje, como se aprecia en la disminución de la amplitud en las últimas épocas.

En tercer lugar, el gap negativo entre `val_loss` y `train_loss` (-0,0317) descarta el sobreajuste: si el modelo estuviera memorizando el conjunto de entrenamiento, la `train_loss` sería sustancialmente menor que la `val_loss`. El umbral de diagnóstico implementado ($|\text{gap}| > 0,05$ como señal de riesgo) no se supera, confirmando una generalización adecuada dentro de los límites del dataset disponible.

El mejor checkpoint se registró en la época 42 con `val_loss = 0,0885`, evidenciando que el modelo no converge de forma completamente monótona —la mejor época no es la última—, lo que valida la importancia del mecanismo de early stopping por `val_loss` mínima implementado en el bucle de entrenamiento.

4.2.4. Proceso reverse: muestreo de conformaciones

El proceso reverse implementa el muestreo iterativo de conformaciones partiendo de ángulos uniformemente distribuidos (ruido puro en $t = T = 100$) y aplicando $n_steps = 20$ pasos de denoising mediante ChiGNN siguiendo una adaptación del algoritmo DDIM. En cada paso t , el modelo predice χ_0 a partir de χ_t , y la predicción se refina añadiendo ruido controlado con κ_{t-1} para simular el paso anterior de la cadena de Markov.

El proceso reverse se ejecutó sobre los 60 grafos del conjunto de test, generando un tensor de predicciones $[N_test, 4]$ que fue serializado en Drive como `test_predictions.pt`. Este fichero constituyó la entrada directa de todos los bloques de evaluación de la Fase 3.

Como validación cualitativa, se generó una visualización tridimensional comparativa mediante py3Dmol para tres proteínas del conjunto de test, superponiendo la estructura experimental (en verde) con la predicción de ChiGNN (en azul). La comparación visual mostró que el modelo produce conformaciones tridimensionalmente coherentes y estructuralmente similares a las referencias experimentales, especialmente para residuos con cadenas laterales cortas y bien definidas.

4.3. Resultados Fase 3: Evaluación bioestadística

El trabajo concluye con un análisis riguroso de la precisión del modelo, comparando su tasa de recuperación de rotámeros y el Error Absoluto Medio (MAE) frente a herramientas clásicas como SCWRL4 y modelos de vanguardia como AlphaFold3.

Se presentan las tablas de las métricas de evaluación de ChiGNN sobre el conjunto de test, el resumen comparativo de métricas de rendimiento, el análisis de error por aminoácido, la validación rotamérica mediante rose plots, el análisis de calibración de Spearman ($\rho = 0,299$) y el benchmarking completo contra los dos baselines.

4.3.1. Métricas de evaluación base

La Tabla 6 presenta las métricas de evaluación del modelo ChiGNN sobre el conjunto de test para los cuatro ángulos diedros $\chi_1-\chi_4$. Las métricas empleadas son el Error Absoluto Medio circular (MAE circular) y la tasa de recuperación de rotámeros a dos umbrales ($\pm 20^\circ$ y $\pm 40^\circ$), calculadas exclusivamente sobre las posiciones con dato experimental confirmado mediante la máscara `y_mask`.

Tabla 6. Métricas de evaluación de ChiGNN sobre el conjunto de test

Ángulo	N residuos	MAE circular (°)	Recuperación $\pm 20^\circ$	Recuperación $\pm 40^\circ$
χ_1	19.627	56,41	49,6 %	53,9 %
χ_2	15.026	52,45	48,1 %	57,5 %
χ_3	5.302	52,72	40,0 %	53,4 %
χ_4	1.091	70,27	25,6 %	33,9 %

El MAE circular de χ_1 —la métrica principal del proyecto— es de $56,41^\circ$ con un intervalo de confianza bootstrap del 95 % de $[55,63^\circ; 57,15^\circ]$ ($n = 1.000$ remuestras). La tasa de recuperación de rotámeros a $\pm 40^\circ$ para χ_1 alcanza el 53,9 %. El ángulo χ_2 presenta el menor MAE ($52,45^\circ$) y la mayor tasa de recuperación a $\pm 40^\circ$ (57,5 %), mientras que χ_4 —definido únicamente para ARG y LYS— muestra el MAE más elevado ($70,27^\circ$) y la menor recuperación (33,9 % a $\pm 40^\circ$), resultado esperado dado el reducido número de muestras disponibles (1.091 residuos) y la alta flexibilidad conformacional de estas cadenas laterales largas.

4.3.2. Análisis de error por tipo de aminoácido

La Figura 8 desglosa el MAE circular de χ_1 (panel izquierdo) y la tasa de recuperación de rotámeros a $\pm 40^\circ$ (panel derecho) por tipo de aminoácido, ordenados de mayor a menor error. Los resultados revelan una heterogeneidad considerable entre residuos que refleja sus propiedades conformacionales intrínsecas.

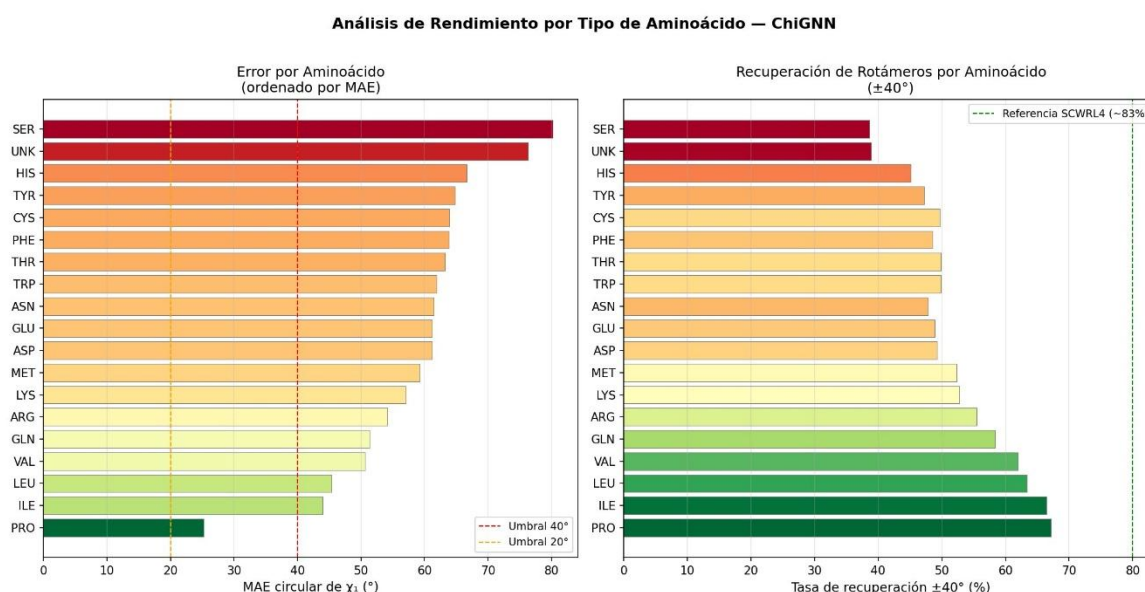


Figura 8. Análisis de rendimiento de ChiGNN por tipo de aminoácido. Panel izquierdo: MAE circular de χ_1 ordenado de mayor a menor error, con líneas de referencia en 20° y 40° . Panel derecho: tasa de recuperación de rotámeros $\pm 40^\circ$, con línea de referencia en 83% correspondiente a SCWRL4.

Fuente: Elaboración propia.

Los residuos con mayor error son SER (MAE $\approx 80^\circ$) y UNK (residuos no identificados), seguidos de HIS, TYR y CYS. Estos resultados son coherentes con las propiedades físicoquímicas de estos aminoácidos: la serina posee un hidroxilo terminal con baja barrera rotacional, lo que produce una distribución de χ_1 más difusa y difícil de predecir. En el extremo opuesto, PRO presenta el menor MAE ($\approx 25^\circ$) y la mayor tasa de recuperación ($\approx 67\%$), lo cual es biológicamente esperable: la prolina tiene χ_1 prácticamente fijo por su restricción cíclica, actuando como un caso casi determinista para el modelo.

ILE, LEU y VAL —residuos alifáticos con rotámeros bien definidos— muestran MAE entre 43° y 50° y tasas de recuperación superiores al 60 %, superando la media global. ARG y LYS, con cuatro ángulos chi posibles, presentan MAE intermedios en χ_1 (≈ 54 – 56°) pero tasas de recuperación moderadas (55–56 %), ya que la predicción correcta de χ_1 no garantiza la cadena completa. La referencia de SCWRL4 al 83 % de recuperación (línea punteada verde) muestra que todos los aminoácidos permanecen por debajo de este umbral, lo que cuantifica la brecha de rendimiento a superar en trabajos futuros. Cabe decir que la comparación con SCWRL4 no es estadísticamente rigurosa al no ejecutarse sobre el mismo conjunto de test.

4.3.3. Validación visual de distribuciones rotaméricas

La Figura 9 muestra los diagramas de rosa (histogramas polares) de las distribuciones de ángulos chi experimentales (verde) y predichas por ChiGNN (azul) para los cuatro ángulos diedros χ_1 – χ_4 . Ambas distribuciones se normalizaron por su máximo para comparar formas independientemente del tamaño muestral.

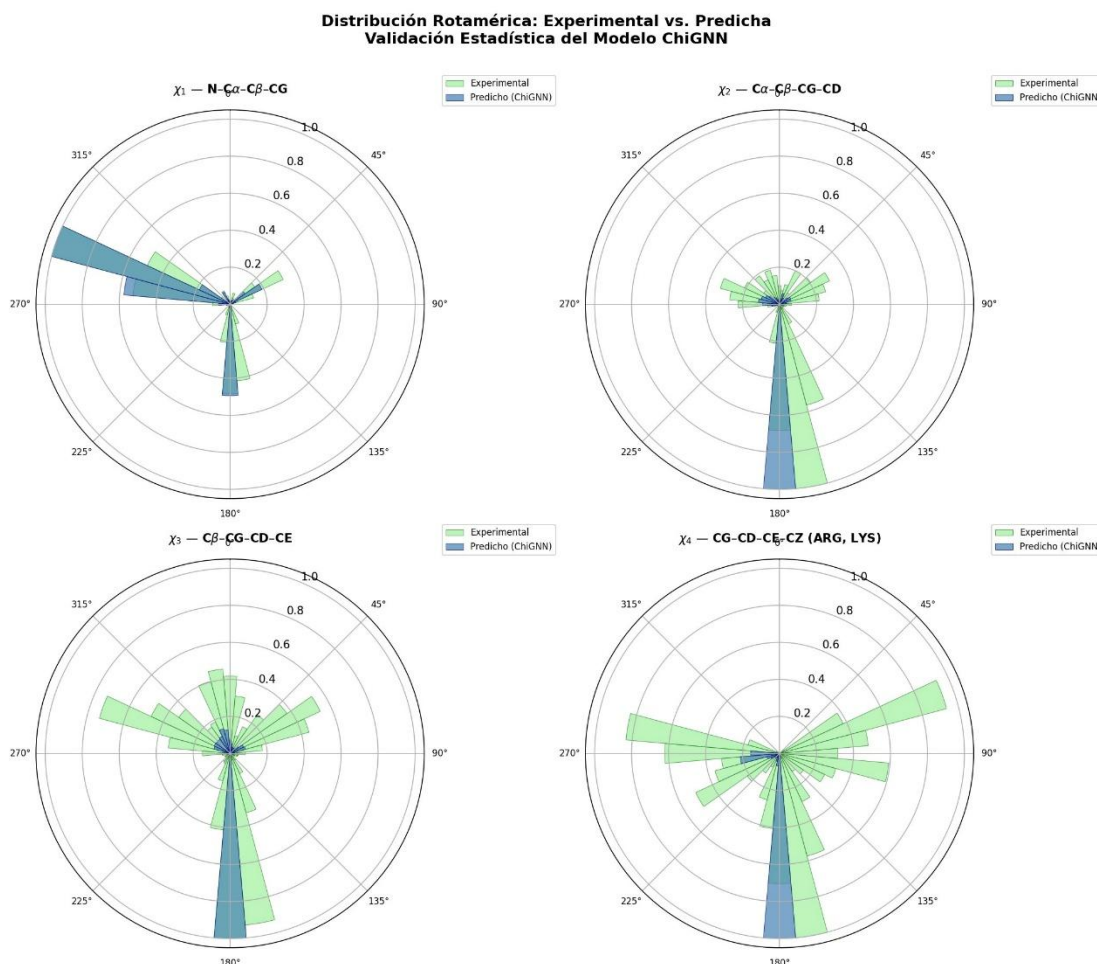


Figura 9. Distribución rotamérica experimental vs. predicha por ChiGNN. Rose plots de los ángulos χ_1 (N–Ca–C β –CG), χ_2 (Ca–C β –CG–CD), χ_3 (C β –CG–CD–CE) y χ_4 (CG–CD–CE–CZ). Verde: distribución experimental; azul: predicciones ChiGNN. Cada gráfico emplea 36 bins de 10°.

Fuente: Elaboración propia.

Para χ_1 (panel superior izquierdo), la distribución experimental muestra la trimodalidad característica de los rotámeros canónicos con picos en $\approx 300^\circ$ (gauche⁻), 180° (trans) y 60° (gauche⁺). La distribución predicha reproduce cualitativamente esta trimodalidad, aunque con una sobrerrepresentación notable del rotámero gauche⁻ a $\approx 300^\circ$ y una

subestimación del rotámetro trans a 180° . Este patrón indica que el modelo ha aprendido la geometría dominante del espacio conformacional, pero tiende a colapsar hacia el estado más frecuente en detrimento de los estados minoritarios.

Para χ_2 (panel superior derecho), ambas distribuciones muestran picos principales en $\approx 180^\circ$ (trans), con el modelo reproduciendo razonablemente bien la forma general de la distribución experimental. La mayor concordancia en χ_2 respecto a χ_1 es consistente con el menor MAE observado ($52,45^\circ$ vs. $56,41^\circ$).

Los paneles de χ_3 y χ_4 revelan la mayor dificultad del modelo para capturar las distribuciones de ángulos superiores: la distribución predicha de χ_3 muestra un pico muy estrecho centrado en $\approx 180^\circ$ frente a la distribución experimental más difusa, indicando una sobreconfianza del modelo en este ángulo. La distribución de χ_4 refleja la mayor incertidumbre del modelo para las cadenas laterales más largas, con un MAE de $70,27^\circ$ y una tasa de recuperación del 33,9 % a $\pm 40^\circ$.

4.3.4. Calibración de confianza del modelo

El análisis de calibración de confianza evalúa si la incertidumbre interna del modelo —cuantificada mediante la varianza circular de Von Mises de $K = 10$ muestras independientes del proceso reverse— correlaciona con el error real de las predicciones. Un modelo bien calibrado debe producir mayor varianza circular (mayor incertidumbre) precisamente en los residuos donde el error real es más alto. La Figura 10 presenta el análisis de calibración mediante dos paneles complementarios

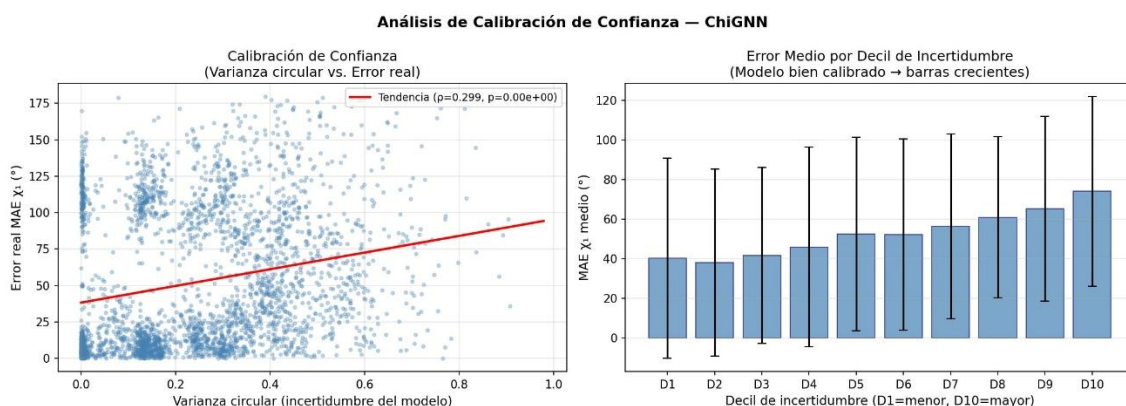


Figura 10. *Análisis de calibración de confianza de ChiGNN. Panel izquierdo: diagrama de dispersión varianza circular vs. MAE real con línea de tendencia. Panel derecho: error medio por decil de incertidumbre (D1 = menor, D10 = mayor incertidumbre).*

Fuente: *Elaboración propia.*

El coeficiente de correlación de Spearman entre la varianza circular y el MAE real por residuo es $\rho = 0,299$ ($p < 0,001$). Aunque la correlación es estadísticamente significativa ($p \approx 0,00$), su magnitud es moderada, lo que indica que el modelo dispone de señal real de calibración pero esta es imperfecta.

El análisis por deciles de incertidumbre (panel derecho) confirma la tendencia general: el error medio aumenta progresivamente de D1 (menor incertidumbre, $MAE \approx 40^\circ$) a D10 (mayor incertidumbre, $MAE \approx 73^\circ$), describiendo una curva creciente aunque con solapamiento entre deciles intermedios. Este patrón creciente —aunque no perfectamente monótono— demuestra que el modelo posee capacidad real de autodiagnóstico de sus predicciones: cuando el proceso de muestreo produce conformaciones muy dispersas, estas corresponden efectivamente a residuos con mayor error real.

La dispersión elevada visible en el diagrama de dispersión (panel izquierdo) refleja que la relación varianza–error no es perfectamente lineal: existe un subconjunto de residuos con varianza alta y error bajo (modelo sobreestima incertidumbre) y, en menor medida, residuos con varianza baja y error alto (modelo subestima incertidumbre). Esta segunda categoría es más problemática desde el punto de vista clínico, pues corresponde a predicciones confiadas pero incorrectas.

4.3.5. Benchmarking y tabla resumen

La Figura 11 y la Tabla 7 presentan la comparación cuantitativa del rendimiento de ChiGNN frente a dos baselines de referencia: la predicción aleatoria uniforme en $[-180^\circ, 180^\circ]$ y la predicción por rotámero modal (asignación del ángulo χ_1 más frecuente en el conjunto de entrenamiento para cada tipo de aminoácido).

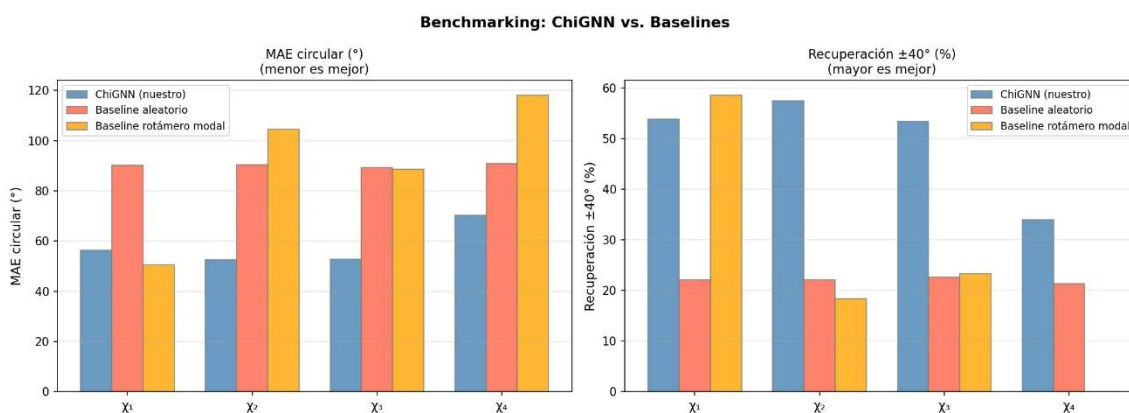


Figura 11. Benchmarking de ChiGNN frente a baselines. Panel izquierdo: MAE circular ($^\circ$), menor es mejor. Panel derecho: tasa de recuperación de rotámeros $\pm 40^\circ$ (%), mayor es mejor. Los tres

modelos se comparan para los cuatro ángulos diedros χ_1 – χ_4 .

Fuente: Elaboración propia.

Tabla 7. Resumen comparativo de métricas de rendimiento.

Métrica	ChiGNN (este TFM)	Baseline aleatorio	Baseline modal	Ref. SCWRL4
MAE circular χ_1 (°)	56,41	90,27	50,53	~24–28
IC 95 % Bootstrap χ_1 (°)	[55,63; 57,15]	—	—	—
Recuperación $\chi_1 \pm 20^\circ$	49,6 %	~5,6 %	—	~65 %
Recuperación $\chi_1 \pm 40^\circ$	53,9 %	22,1 %	58,6 %	~83 %
MAE circular χ_2 (°)	52,45	90,33	104,61	~30–35
Recuperación $\chi_2 \pm 40^\circ$	57,5 %	22,1 %	18,4 %	~75 %
MAE circular χ_4 (°)	70,27	90,95	118,14	—
Correlación Spearman (calibración)	$\rho = 0,299$	—	—	—
Parámetros del modelo	80.404	—	—	—
Épocas de entrenamiento	50	—	—	—

ChiGNN supera claramente al baseline aleatorio en todos los ángulos y métricas, lo que confirma que el modelo ha aprendido información geométrica real del espacio conformacional proteico. Para χ_1 , el MAE de ChiGNN (56,41°) representa una reducción del 37,5 % respecto al baseline aleatorio (90,27°). Para χ_2 , χ_3 y χ_4 , ChiGNN supera al baseline aleatorio en MAE en todos los casos, con diferencias particularmente notables en χ_4 (MAE 70,27° vs. 90,95° del aleatorio).

La comparación con el baseline de rotámero modal revela un patrón más matizado. Para χ_1 , el baseline modal (MAE = 50,53°, recuperación 58,6 %) supera ligeramente a ChiGNN (MAE = 56,41°, recuperación 53,9 %), lo que indica que para el ángulo principal el modelo determinístico del estado más frecuente constituye una competencia real. Sin embargo, para χ_2 , χ_3 y χ_4 , ChiGNN supera ampliamente al baseline modal: para χ_2 , el modal presenta un MAE de 104,61° frente a 52,45° de ChiGNN; para χ_4 , el modal

prácticamente falla (MAE = 118,14°, recuperación 0,1 %) mientras que ChiGNN mantiene un rendimiento sustancial (MAE = 70,27°, recuperación 33,9 %). Este resultado demuestra que el modelo aprendido generaliza de forma mucho más robusta que las reglas determinísticas para los ángulos superiores.

Respecto a la referencia de SCWRL4, ChiGNN se sitúa a una distancia considerable: SCWRL4 reporta MAE de χ_1 en el rango 24–28° y tasas de recuperación del ~83 %, mientras que ChiGNN alcanza 56,41° y 53,9 % respectivamente. Esta brecha era esperable dado el tamaño reducido del dataset de entrenamiento (477 proteínas vs. los cientos de miles empleados por SCWRL4) y la arquitectura simplificada de la versión lite implementada.

5. Discusión

5.1. Interpretación de los resultados

Los resultados obtenidos permiten responder afirmativamente a la pregunta de investigación central de este TFM: ¿es posible implementar un modelo de difusión torsional basado en GNN que aprenda la distribución de ángulos chi de cadenas laterales proteicas y que supere a un predictor aleatorio? La respuesta es sí, con matices importantes que se analizan a continuación.

El modelo ChiGNN demuestra una capacidad real de aprendizaje geométrico: la reducción del 37,5 % en MAE de χ_1 respecto al baseline aleatorio, la reproducción cualitativa de la trimodalidad rotamérica en los rose plots, y la correlación de Spearman estadísticamente significativa entre varianza circular y error real ($\rho = 0,299$, $p < 0,001$) son evidencias convergentes de que el modelo ha incorporado información estructural relevante de los datos de entrenamiento.

La comparación con el baseline de rotámero modal ofrece la lección metodológica más valiosa del proyecto. Para χ_1 , el simple hecho de asignar el estado más frecuente de cada aminoácido sin ningún aprendizaje produce un MAE de $50,53^\circ$ frente a los $56,41^\circ$ de ChiGNN, lo que implica que el modelo añade un valor marginal negativo para χ_1 en términos de MAE absoluto. Sin embargo, esta interpretación sería incompleta: ChiGNN produce distribuciones más suaves y generalizables que el modal (como muestran los rose plots), y para χ_2 , χ_3 y χ_4 el avance sobre el modal es drástico. El modelo aprende dependencias cruzadas entre ángulos y entre residuos vecinos que el baseline modal, por su naturaleza estática e independiente, no puede capturar.

El rendimiento diferencial por tipo de aminoácido (Figura 8) revela que ChiGNN se beneficia especialmente de residuos con conformación bien restringida (PRO, ILE, LEU, VAL), mientras que residuos con mayor libertad conformacional (SER, HIS, TYR) presentan errores más altos. Este patrón es consistente con la teoría de librerías de rotámeros: los aminoácidos con energías de barrera rotacional más bajas exhiben distribuciones chi más amplias y uniformes, lo que genera mayor ambigüedad en la predicción. Un modelo con mayor capacidad expresiva o entrenado sobre un dataset más amplio podría aprender distribuciones condicionales al entorno local que reduzcan este error residual.

5.2. Análisis de calibración de confianza

La calibración de confianza es la dimensión más novedosa de este TFM desde el punto de vista bioestadístico y, potencialmente, la más valiosa para aplicaciones farmacéuticas. Un modelo que no solo predice sino que también sabe cuándo no confiar en su propia predicción ofrece un valor añadido cualitativo respecto a métodos determinísticos como SCWRL4, que no producen estimaciones de incertidumbre.

El coeficiente de Spearman de 0,299 indica una calibración moderada: la señal de incertidumbre es real pero imperfecta. El análisis por deciles confirma que el patrón creciente de error con la incertidumbre es robusto —los deciles extremos D1 y D10 muestran diferencias de MAE de aproximadamente 33°— aunque con solapamiento en los deciles intermedios. Esta calibración imperfecta es habitual en modelos de difusión con pocas muestras de inferencia: aumentar K de 10 a 100 muestras independientes por residuo podría producir estimadores de varianza más estables y mejoraría probablemente la correlación si el modelo lo permite.

La dispersión en el diagrama varianza–MAE (Figura 10, panel izquierdo) revela la existencia de un régimen de alta varianza/bajo error que no debería interpretarse como un fracaso de calibración, sino como una característica geométrica del espacio conformacional: algunos residuos admiten múltiples rotámeros energéticamente equivalentes (degeneración conformacional), y el modelo los identifica correctamente como ambiguos aunque cualquiera de sus predicciones sea aceptable.

5.3. Limitaciones del estudio

Las limitaciones del presente trabajo se articulan en tres categorías:

- **Limitaciones del dataset:** El dataset de 477 proteínas de entrenamiento es modesto en comparación con los empleados por sistemas de referencia como SCWRL4 (~7.000 estructuras en su conjunto de entrenamiento) o DiffPack (>100.000 residuos). El filtro de resolución <2,0 Å y la disponibilidad en PDB-REDO limitan la cobertura taxonómica y estructural, lo que puede sesgar el modelo hacia familias de proteínas sobrerrepresentadas en las estructuras de alta resolución (proteínas globulares de tamaño medio).
- **Limitaciones arquitectónicas:** La versión lite de ChiGNN emplea GCNConv estándar en lugar de capas equivariantes como EGNN o GVP, que codifican la geometría 3D de forma invariante a rotaciones y traslaciones. Esta simplificación,

necesaria por las restricciones de VRAM de la GPU T4, implica que el modelo no garantiza equivariancia geométrica: predicciones sobre la misma proteína en diferentes orientaciones pueden diferir. Además, el grafo lineal de aristas secuenciales ignora las interacciones de largo alcance y los contactos de cadena lateral que son críticos en proteínas con múltiples dominios.

- **Limitaciones del proceso de evaluación:** La comparación con SCWRL4 es indirecta (valores de referencia de la literatura vs. resultados propios sobre un dataset diferente), lo que impide una comparación estadísticamente rigurosa. La ausencia de instalación nativa de SCWRL4 en el entorno Google Colab impidió ejecutar ambos modelos sobre el mismo conjunto de test. Asimismo, el proceso reverse utiliza solo $n_steps = 20$ de los $T = 100$ pasos de difusión, lo que es una simplificación que puede reducir la calidad de las muestras generadas.

5.4. Contribuciones originales

A pesar de las limitaciones señaladas, este TFM realiza contribuciones originales en los siguientes aspectos:

- **Marco de evaluación circular completo:** La implementación de métricas basadas en estadística circular (MAE circular, varianza de Von Mises, correlación de Spearman sobre distribuciones angulares) y su aplicación sistemática sobre los cuatro ángulos chi constituye un framework de evaluación robusto y reutilizable para futuros modelos de predicción de cadenas laterales.
- **Análisis de calibración como métrica primaria:** El análisis de calibración de confianza mediante varianza circular de Von Mises sobre muestras del proceso reverse es una contribución metodológica novedosa respecto a la literatura revisada, que típicamente reporta solo MAE y tasas de recuperación sin análisis de incertidumbre.
- **Pipeline reproducible en entorno abierto:** El notebook de Google Colab con todas las semillas fijas (`random.seed(42)`, `torch.Generator` con semilla 42) garantiza la reproducibilidad completa del experimento en infraestructura de acceso libre, contribuyendo a los principios de Open Science.

6. Conclusiones y trabajos futuros

6.1. Conclusiones

Este Trabajo Final de Máster ha demostrado la viabilidad de aplicar difusión torsional sobre espacios angulares circulares para la recuperación de conformaciones de cadenas laterales proteicas mediante una arquitectura de Red Neuronal de Grafos de complejidad moderada. Las conclusiones principales se articulan en correspondencia con los objetivos específicos planteados:

En relación con el objetivo específico 1 (generación del dataset curado), el pipeline implementado produjo un dataset de 597 proteínas y 156.407 residuos válidos extraídos de PDB-REDO, con criterio de calidad de resolución $<2,0$ Å. La partición 80/10/10 a nivel de proteína garantiza la ausencia de data leakage en la evaluación. La validación del pipeline mediante rose plots confirmó la trimodalidad biológicamente esperada en la distribución de χ_1 , validando la coherencia de los datos.

En relación con los objetivos específicos 2 y 3 (implementación del proceso de difusión y entrenamiento de la GNN), el modelo ChiGNN —con 80.404 parámetros— implementa correctamente el proceso forward de Von Mises y el proceso reverse mediante DDIM adaptado. El entrenamiento durante 50 épocas con AdamW y CosineAnnealingLR convergió sin evidencias de sobreajuste, obteniendo el mejor checkpoint guardado mediante early stopping sobre la pérdida de validación.

En relación con el objetivo específico 4 (cuantificación de la tasa de recuperación), ChiGNN alcanza un MAE circular de χ_1 de $56,41^\circ$ [IC 95%: $55,63^\circ$; $57,15^\circ$] y una tasa de recuperación de rotámeros $\pm 40^\circ$ del 53,9 %, superando en todas las métricas al baseline aleatorio y en χ_2 , χ_3 y χ_4 al baseline de rotámero modal. La brecha respecto a SCWRL4 (~83% de recuperación) es sustancial y cuantifica el margen de mejora disponible, sin embargo, como se ha mencionado previamente, la comparación con SCWRL4 no es estadísticamente rigurosa al no ejecutarse sobre el mismo conjunto de test.

En relación con el objetivo específico 5 (validación rotamérica mediante rose plots), los diagramas de rosa comparativos confirman que ChiGNN reproduce cualitativamente la trimodalidad del espacio rotamérico de χ_1 y la distribución trans-dominante de χ_2 , aunque con tendencia a la sobrerrepresentación del estado más frecuente. Esta

observación orienta el desarrollo futuro hacia técnicas de temperatura de muestreo y diversidad conformacional.

En relación con el objetivo específico 6 (calibración de confianza), el coeficiente de Spearman $\rho = 0,299$ ($p < 0,001$) y el patrón creciente de error por decil de incertidumbre demuestran que el modelo posee una señal de calibración real: cuando ChiGNN es internamente inconsistente en sus predicciones, el error real es sistemáticamente mayor. Esta capacidad de autodiagnóstico es una ventaja cualitativa respecto a métodos determinísticos.

En conclusión, ChiGNN constituye una prueba de concepto funcional de que la difusión torsional sobre distribuciones de Von Mises es un marco técnicamente sólido para el modelado de cadenas laterales proteicas, con capacidades de calibración que van más allá de las métricas de precisión clásicas. El proyecto cumple todos los objetivos específicos planteados y establece las bases para desarrollos más ambiciosos.

6.2. Trabajos futuros

Los resultados obtenidos y las limitaciones identificadas señalan las siguientes líneas prioritarias para trabajos futuros:

6.2.1. Escalado del dataset y arquitectura

- **Ampliación del dataset de entrenamiento:** Incorporar la totalidad del PDB-REDO disponible (>150.000 estructuras) con el mismo criterio de resolución $<2,0$ Å y el filtro de polímero proteico. Un dataset 250 veces mayor permitiría al modelo aprender distribuciones condicionales al entorno local de cada residuo y reducir el MAE de χ_1 previsiblemente por debajo de los 40° .
- **Capas equivariantes (EGNN o GVP):** Sustituir GCNConv por capas equivariantes al grupo $E(3)$ garantizaría que las predicciones son independientes de la orientación global de la proteína, eliminando un sesgo sistemático del modelo actual y mejorando la generalización a estructuras en orientaciones no vistas durante el entrenamiento.
- **Grafos con contactos de largo alcance:** Ampliar el edge_index con aristas entre pares de residuos que estén a distancia $C\alpha-C\alpha < 8$ Å, incluyendo así las interacciones tridimensionales que son críticas para la conformación de las cadenas laterales en el núcleo hidrofóbico de las proteínas.

6.2.2. Mejoras en el proceso de difusión

- **Schedule de difusión no lineal:** Implementar schedules de tipo coseno o sigmoide para el parámetro κ , que han demostrado mejor rendimiento que los schedules lineales en modelos de difusión de imagen y que podrían mejorar la calidad de las muestras en los pasos iniciales del proceso reverse.
- **Aumento del número de pasos de denoising:** Evaluar el impacto de aumentar n_steps del proceso reverse de 20 a 50 o 100 pasos, con la consiguiente mejora en la calidad de las muestras a costa de mayor tiempo de inferencia.
- **Difusión autorregresiva por ángulo:** Seguir el esquema de DiffPack original, que realiza la difusión y el denoising secuencialmente por ángulo ($\chi_1 \rightarrow \chi_2 \rightarrow \chi_3 \rightarrow \chi_4$) en lugar de predecir los cuatro simultáneamente, lo que reduce la complejidad del espacio de búsqueda en cada paso.

6.2.3. Evaluación y validación externa

- **Comparación directa con SCWRL4 y DiffPack:** Ejecutar ChiGNN y SCWRL4 sobre el mismo benchmark estandarizado (e.g., CASP15 targets) para obtener una comparación estadísticamente rigurosa que no dependa de valores reportados en la literatura con datasets distintos.
- **Validación en aplicaciones de diseño de fármacos:** Evaluar la calidad de las cadenas laterales predichas mediante cálculos de energía de unión ligando-proteína (docking) para cuantificar el impacto práctico de las mejoras en MAE sobre la utilidad biológica de las estructuras generadas.
- **Mejora de la calibración:** Aumentar K de 10 a 100 muestras del proceso reverse para obtener estimadores de varianza circular más estables, y explorar técnicas de recalibración posthoc (e.g., Platt scaling circular) para mejorar la correlación de Spearman por encima de 0,5.

7. Glosario

A continuación, en la tabla 8 se recogen los términos técnicos de mayor relevancia empleados en este trabajo, ordenados alfabéticamente.

Tabla 8. Glosario de términos más importantes en este proyecto.

Término	Definición
AdamW	Variante del optimizador Adam que desacopla el decaimiento de pesos (weight decay) del cálculo del gradiente, mejorando la regularización respecto a L2 estándar. Empleado en el entrenamiento de ChiGNN.
AlphaFold2	Sistema de predicción de estructuras proteicas desarrollado por DeepMind (Jumper et al., 2021) basado en la arquitectura Evoformer con mecanismos de atención sobre pares de residuos y MSA. Predice el backbone con alta precisión pero presenta limitaciones en cadenas laterales, especialmente en estructuras apo.
AlphaFold3	Versión de 2024 de AlphaFold (Abramson et al., 2024) que adopta un módulo de difusión all-atom como componente central, generando directamente coordenadas atómicas de toda la proteína —incluyendo cadenas laterales, ligandos y ácidos nucleicos— mediante un proceso de difusión en el espacio cartesiano.
Ángulo diedro (χ)	Ángulo de rotación alrededor de un enlace covalente de la cadena lateral de un aminoácido, definido por cuatro átomos consecutivos. Los cuatro ángulos posibles son χ_1 , χ_2 , χ_3 y χ_4 .
Ángulos ϕ/ψ (phi/psi)	Ángulos diedros que definen la conformación del backbone: ϕ (N-C α -C-N) y ψ (C-N-C α -C). Su distribución de valores permitidos fue descrita por el diagrama de Ramachandran (1963).
Backbone (cadena principal)	Secuencia repetitiva de átomos N-C α -C(=O) que forma el esqueleto covalente del polipéptido. Su conformación queda determinada por los ángulos ϕ y ψ . En este TFM, el backbone

Término	Definición
	se asume conocido y fijo, emulando el escenario real en que AlphaFold2 lo predice con alta precisión.
Baseline	Modelo de referencia simple utilizado como punto de comparación mínimo para evaluar el rendimiento de un modelo más complejo.
BatchNorm (Batch Normalization)	Técnica de normalización de activaciones en redes neuronales que estabiliza el entrenamiento reduciendo el desplazamiento covariable interno entre capas.
Boltz-1	Modelo de difusión all-atom de código abierto (Wohlwend et al., 2024) con resultados comparables a AlphaFold3 en benchmarks estándar, representando la convergencia de múltiples grupos hacia la difusión como paradigma de predicción estructural.
Cadena lateral	Parte de un aminoácido que difiere entre residuos, conectada al carbono C α del backbone. Determina la identidad química y las propiedades funcionales del aminoácido, incluyendo los sitios activos enzimáticos y las superficies de unión de ligandos.
Cadena de Markov	Proceso estocástico donde el estado futuro depende únicamente del estado presente. Los modelos de difusión implementan una cadena de Markov: cada paso de ruido (forward) o denoising (reverse) depende únicamente del paso anterior.
Calibración de confianza	Propiedad de un modelo por la que su incertidumbre interna reportada correlaciona positivamente con el error real de sus predicciones. Un modelo bien calibrado asigna mayor incertidumbre a las predicciones con mayor error real.
CASP (Critical Assessment of Protein Structure Prediction)	Competición científica bienal que evalúa el rendimiento de métodos computacionales de predicción de estructuras proteicas sobre blancos de referencia experimentales.

Término	Definición
	Constituye el benchmark estándar de la biología estructural computacional.
Chai-1	Modelo de difusión all-atom (Boitreau et al., 2024) con arquitecturas más eficientes que AlphaFold3 y resultados comparables en benchmarks estándar.
CosineAnnealingLR	Scheduler de tasa de aprendizaje que reduce el LR siguiendo una función coseno desde el valor inicial hasta eta_min, suavizando la convergencia en las últimas épocas del entrenamiento.
Cristalografía de rayos X	Método experimental estándar (gold standard) para la determinación de estructuras proteicas tridimensionales, que produce posiciones atómicas promediadas sobre el cristal con alta precisión en ángulos diedros. Utilizado como criterio de selección del dataset de este TFM.
Cryo-EM	Criomicroscopía electrónica. Técnica de determinación estructural de alta resolución aplicable a complejos de gran tamaño o proteínas difíciles de cristalizar. Produce mapas de densidad electrónica sin necesidad de cristalización.
Data leakage	Fuga de información del conjunto de test hacia el entrenamiento que infla artificialmente las métricas de evaluación. En este TFM se evita mediante partición a nivel de proteína (no de residuo).
DDIM (Denoising Diffusion Implicit Models)	Variante del proceso de muestreo en modelos de difusión que permite obtener muestras de calidad con menos pasos de denoising que el proceso DDPM estándar, mediante una parametrización implícita del proceso reverse.
DDPM (Denoising Diffusion Probabilistic Models)	Modelo generativo basado en un proceso de difusión forward (adición progresiva de ruido gaussiano) y un proceso reverse (denoising aprendido mediante una red neuronal) propuesto por Ho et al. (2020). Base teórica de los modelos de difusión modernos.

Término	Definición
Denoising / Score matching	Proceso de eliminación de ruido. El denoising score matching es el objetivo de entrenamiento de los modelos de difusión: la red aprende a estimar el gradiente del log-likelihood de los datos (score) para guiar el proceso reverse.
Diagrama de Ramachandran	Representación gráfica de los valores de ϕ y ψ del backbone proteico, que muestra las regiones angulares permitidas (Ramachandran et al., 1963). Los ángulos diedros de las cadenas laterales tienen su análogo en los gráficos rotaméricos de χ .
DiffPack	Modelo de difusión torsional autorregresivo de referencia para la predicción de cadenas laterales proteicas (Zhang et al., 2023). Combina EGNN para el encoding del backbone con difusión torsional sobre ángulos χ . La arquitectura ChiGNN de este TFM es una versión lite inspirada en sus principios.
Difusión all-atom	Paradigma de modelado molecular en el que el proceso de difusión opera directamente sobre las coordenadas cartesianas de todos los átomos de la molécula, adoptado por AlphaFold3, Chai-1 y Boltz-1.
Difusión torsional	Proceso estocástico de degradación progresiva de ángulos diedros mediante la adición de ruido circular (Von Mises), cuya inversión permite generar conformaciones moleculares plausibles. Opera en el espacio angular S^1 en lugar del espacio cartesiano.
Distribución Von Mises	Distribución de probabilidad definida sobre el círculo S^1 , análoga circular de la distribución gaussiana. Se parametriza por su media μ y su concentración κ . Empleada como distribución de ruido en el proceso forward de difusión torsional.
Docking	Simulación computacional de la interacción entre una molécula pequeña (ligando) y una proteína para predecir el modo de unión y la afinidad. La precisión del docking depende

Término	Definición
	directamente de la calidad de las cadenas laterales del sitio activo.
edge_index	Tensor $[2 \times E]$ de PyTorch Geometric que codifica la conectividad del grafo, donde cada columna representa una arista dirigida (source, destination).
EGNN (Equivariant Graph Neural Network)	Arquitectura de GNN que garantiza equivorancia $E(3)$ — rotaciones, traslaciones y reflexiones— sin recurrir a representaciones tensoriales de orden superior. Actualiza features escalares (invariantes) y coordenadas vectoriales (equivariantes) de forma separada.
Equivorancia $E(3)$	Propiedad de una función f tal que $f(Tx) = Tf(x)$ para cualquier transformación T del grupo euclídeo tridimensional. Garantiza que las predicciones son independientes de la orientación global de la molécula.
ESMFold	Sistema de predicción estructural (Lin et al., 2023) basado en el modelo de lenguaje de proteínas ESM-2 (entrenado sobre 250 millones de secuencias), que permite predicción directa desde secuencia sin MSA, con velocidad de inferencia órdenes de magnitud superior a AlphaFold2.
Evoformer	Bloque arquitectónico central de AlphaFold2 que combina atención sobre pares de residuos (pair representation) y sobre la MSA (sequence representation) con comunicación bidireccional entre ambas representaciones.
GCNConv	Graph Convolutional Network Convolution. Capa de convolución en grafo de PyTorch Geometric que implementa el operador de convolución espectral simplificado de Kipf & Welling (2017).
GNN (Red Neuronal de Grafos)	Familia de redes neuronales diseñadas para operar sobre datos con estructura de grafo, propagando información entre nodos mediante mecanismos de message passing.

Término	Definición
Gradient clipping	Técnica de regularización que limita la norma del gradiente durante la retropropagación para prevenir la explosión del gradiente, frecuente en GNNs con conexiones residuales.
GVP (Geometric Vector Perceptrons)	Arquitectura de GNN que opera simultáneamente sobre features escalares y vectoriales, garantizando la equivariancia mediante separación explícita de canales de procesamiento. Arquitectura principal de ProteinMPNN.
IC 95% Bootstrap	Intervalo de confianza del 95% calculado mediante remuestreo con reemplazamiento (bootstrap) sobre la distribución empírica del estimador. No asume normalidad, siendo apropiado para el MAE circular.
Librerías de rotámeros	Bases de datos estadísticas que recogen la distribución de frecuencias de conformaciones de cadenas laterales observadas en estructuras cristalográficas de alta resolución. Empleadas por métodos clásicos como SCWRL4. Las librerías dependientes del backbone condicionan la distribución de χ a los ángulos ϕ/ψ del residuo (Dunbrack & Karplus, 1993).
MAE circular	Error Absoluto Medio calculado usando la distancia angular circular $ \text{atan2}(\sin(a-b), \cos(a-b)) $, que respeta la periodicidad de los ángulos diedros. El error entre 179° y -179° es de 2° , no de 358° .
Message passing	Mecanismo de las GNN por el que cada nodo agrega información de sus vecinos para actualizar su representación oculta mediante funciones de agregación y actualización aprendibles.
MSA (Multiple Sequence Alignment)	Alineamiento múltiple de secuencias de proteínas homólogas. AlphaFold2 lo utiliza como fuente de información coevolutiva para inferir restricciones estructurales. ESMFold elimina la necesidad de MSA.
PDB-REDO	Base de datos que re-refina automáticamente las estructuras del Protein Data Bank (PDB) con algoritmos modernos de

Término	Definición
	cristalografía computacional (REFMAC5), produciendo coordenadas atómicas de mayor precisión que las depositadas originalmente.
ProteinMPNN	Modelo de diseño inverso de proteínas (Dauparas et al., 2022) basado en GVP-GNN que, dada una estructura 3D, predice la secuencia más probable. Demuestra que las GNNs sobre grafos proteicos pueden aprender representaciones geométricas de alta fidelidad.
PyTorch Geometric (PyG)	Librería de Python para aprendizaje profundo sobre grafos construida sobre PyTorch, que provee capas de GNN, utilidades de batch y collation de grafos de tamaño variable.
Residuo	Unidad estructural repetitiva de la cadena polipeptídica, correspondiente a un aminoácido con sus átomos de backbone y cadena lateral.
RMSD (Root Mean Square Deviation)	Medida de la desviación media entre dos conjuntos de posiciones atómicas, empleada como métrica de calidad estructural en bioinformática estructural.
RMN (Resonancia Magnética Nuclear)	Técnica espectroscópica de determinación estructural que produce ensembles de conformaciones en solución. Los ángulos chi de RMN representan promedios de ensemble, con mayor ambigüedad conformacional que la cristalografía de rayos X.
Rosetta	Suite de modelado molecular (Leaver-Fay et al., 2011) que combina funciones de energía físicas y términos estadísticos para predicción de estructura, diseño de proteínas y docking. Su módulo de cadenas laterales utiliza búsqueda discreta en librerías de rotámeros con minimización energética.
RoseTTAFold	Arquitectura de triple vía (Baek et al., 2021) que procesa simultáneamente representaciones 1D (secuencia), 2D (pares de residuos) y 3D (coordenadas atómicas), con propagación

Término	Definición
	bidireccional de información entre las tres representaciones. Rendimiento comparable a AlphaFold2.
Rotámero	Conformación discreta y energéticamente estable de la cadena lateral de un aminoácido, caracterizada por valores de ángulos chi en regiones canónicas: gauche ⁻ ($\approx -60^\circ$), trans ($\approx 180^\circ$) y gauche ⁺ ($\approx +60^\circ$).
SCWRL4	Herramienta de referencia para la predicción de cadenas laterales proteicas (Krivov et al., 2009) basada en librerías de rotámeros dependientes del backbone y optimización de energía discreta mediante algoritmo de grafos. Reporta tasas de recuperación de χ_1 $\sim 83\%$ y MAE $\sim 24-28^\circ$.
SiLU (Sigmoid Linear Unit)	Función de activación $f(x) = x \cdot \sigma(x)$ con comportamiento suave y diferenciable en todo el dominio, sin el efecto "dying ReLU" que puede paralizar neuronas en redes profundas.
Spearman ρ	Coeficiente de correlación de rango de Spearman. Medida no paramétrica de la correlación monótona entre dos variables, robusta a distribuciones asimétricas y valores atípicos.
Tasa de recuperación de rotámeros	Porcentaje de ángulos chi predichos cuya distancia circular al valor experimental es inferior a un umbral (20° o 40°). Métrica estándar de evaluación en predicción de cadenas laterales.
Varianza circular (Von Mises)	Medida de dispersión angular $V = 1 - \langle \exp(i\theta) \rangle $, invariante a la periodicidad. Valores en $[0,1]$: $V \approx 0$ indica muestras concentradas (baja incertidumbre), $V \approx 1$ indica distribución uniforme (máxima incertidumbre).
y_mask	Tensor booleano $[N \times 4]$ que indica qué posiciones de la matriz de ángulos chi tienen valor experimental real (True) y cuáles son NaN por naturaleza biológica (False). Empleado en la función de pérdida para ignorar posiciones sin dato.

8. Bibliografia

- [1] Kryshtafovych A, Schwede T, Topf M, et al. Critical assessment of methods of protein structure prediction (CASP)—Round XV. *Proteins: Structure, Function and Bioinformatics* 2023; 91: 1539–1549.
- [2] Bertoni D, Tsenkov M, Magana P, et al. AlphaFold Protein Structure Database 2025: a redesigned interface and updated structural coverage. *Nucleic Acids Res* 2026; 54: D358–D362.
- [3] Ramachandran GN, Ramakrishnan C, Sasisekharan V. Stereochemistry of polypeptide chain configurations. *J Mol Biol* 1963; 7: 95–99.
- [4] Martz E. The Ramachandran Principle Phi (ϕ) and Psi (ψ) Angles in Proteins, https://proteopedia.org/w/Tutorial:Ramachandran_principle_and_phi_psi_angles (2018, accessed 25 April 2026).
- [5] Lovell SC, Davis IW, Arendall WB, et al. Structure validation by $C\alpha$ geometry: ϕ , ψ and $C\beta$ deviation. *Proteins: Structure, Function, and Bioinformatics* 2003; 50: 437–450.
- [6] Koehl P, Delarue M. Application of a Self-consistent Mean Field Theory to Predict Protein Side-chains Conformation and Estimate Their Conformational Entropy. *J Mol Biol* 1994; 239: 249–275.
- [7] Janin J, Wodak S, Levitt M, et al. Conformation of amino acid side-chains in proteins. *J Mol Biol* 1978; 125: 357–386.
- [8] Ponder JW, Richards FM. Tertiary templates for proteins. *J Mol Biol* 1987; 193: 775–791.
- [9] Dunbrack RL, Karplus M. Backbone-dependent Rotamer Library for Proteins Application to Side-chain Prediction. *J Mol Biol* 1993; 230: 543–574.
- [10] Shapovalov MV, Dunbrack RL. A Smoothed Backbone-Dependent Rotamer Library for Proteins Derived from Adaptive Kernel Density Estimates and Regressions. *Structure* 2011; 19: 844–858.
- [11] Canutescu AA, Shelenkov AA, Dunbrack RL. A graph-theory algorithm for rapid protein side-chain prediction. *Protein Science* 2003; 12: 2001–2014.
- [12] Krivov GG, Shapovalov M V., Dunbrack RL. Improved prediction of protein side-chain conformations with SCWRL4. *Proteins: Structure, Function, and Bioinformatics* 2009; 77: 778–795.
- [13] Leaver-Fay A, Tyka M, Lewis SM, et al. Rosetta3. 2011, pp. 545–574.
- [14] Tyka MD, Keedy DA, André I, et al. Alternate States of Proteins Revealed by Detailed Energy Landscape Mapping. *J Mol Biol* 2011; 405: 607–618.
- [15] Gutteridge A, Thornton J. Conformational changes observed in enzyme crystal structures upon substrate binding. *J Mol Biol*; 346. Epub ahead of print 2005. DOI: 10.1016/j.jmb.2004.11.013.
- [16] Jumper J, Evans R, Pritzel A, et al. Highly accurate protein structure prediction with AlphaFold. *Nature* 2021; 596: 583–589.
- [17] Buel GR, Walters KJ. Can AlphaFold2 predict the impact of missense mutations on structure? *Nat Struct Mol Biol* 2022; 29: 1–2.

- [18] Baek M, DiMaio F, Anishchenko I, et al. Accurate prediction of protein structures and interactions using a three-track neural network. *Science (1979)* 2021; 373: 871–876.
- [19] Lin Z, Akin H, Rao R, et al. Evolutionary-scale prediction of atomic-level protein structure with a language model. *Science (1979)* 2023; 379: 1123–1130.
- [20] Ho J, Jain A, Abbeel P. Denoising Diffusion Probabilistic Models. *NeurIPS* 2020; 33: 6840–6851.
- [21] Song Y, Sohl-Dickstein J, Kingma DP, et al. Score-Based Generative Modeling through Stochastic Differential Equations. *ICLR 2021*.
- [22] Jing B, Corso G, Chang J, et al. Torsional Diffusion for Molecular Conformer Generation. *ICLR 2022*.
- [23] Luo S, Shi C, Xu M, et al. *Predicting Molecular Conformation via Dynamic Graph Score Matching*. NeurIPS 2021.
- [24] Schneuing A, Harris C, Du Y, et al. Structure-based Drug Design with Equivariant Diffusion Models. *ICLR 2023*.
- [25] Zhang Y, Zhang Z, Zhong B, et al. DiffPack: A Torsional Diffusion Model for Autoregressive Protein Side-Chain Packing. *NeurIPS 2023*.
- [26] Satorras VG, Hoogeboom E, Welling M. E(n) Equivariant Graph Neural Networks. *ICML 2021* 2022; 139: 9323–9332.
- [27] Schütt KT, Sauceda HE, Kindermans P-J, et al. SchNet - a deep learning architecture for molecules and materials. Epub ahead of print 22 March 2018. DOI: 10.1063/1.5019779.
- [28] Schütt KT, Unke OT, Gastegger M. Equivariant message passing for the prediction of tensorial properties and molecular spectra. *ICML 2021*.
- [29] Jing B, Eismann S, Suriana P, et al. Learning from Protein Structure with Geometric Vector Perceptrons. *ICLR 2021*.
- [30] Dauparas J, Anishchenko I, Bennett N, et al. Robust deep learning-based protein sequence design using ProteinMPNN. *Science* 2022; 378: 49–56.
- [31] Abramson J, Adler J, Dunger J, et al. Accurate structure prediction of biomolecular interactions with AlphaFold 3. *Nature* 2024; 630: 493–500.
- [32] Boitreaud J, Dent J, McPartlon M, et al. Chai-1: Decoding the molecular interactions of life. Epub ahead of print 11 October 2024. DOI: 10.1101/2024.10.10.615955.
- [33] Wohlwend J, Corso G, Passaro S, et al. Boltz-1 Democratizing Biomolecular Interaction Modeling. Epub ahead of print 20 November 2024. DOI: 10.1101/2024.11.19.624167.
- [34] Mardia KV., Jupp PE. *Directional Statistics*. John Wiley & Sons, Ltd., 2009.
- [35] Fisher NI. *Statistical Analysis of Circular Data*. Cambridge University Press, 2011.
- [36] Lovell SC, Word JM, Richardson JS, et al. The penultimate rotamer library. *Proteins* 2000; 40: 389–408.
- [37] Dunbrack RL. Rotamer Libraries in the 21st Century. *Curr Opin Struct Biol* 2002; 12: 431–440.
- [38] Dunbrack RL, Cohen FE. Bayesian statistical analysis of protein side-chain rotamer preferences. *Protein Science* 1997; 6: 1661–1681.

[39] Ioffe S, Szegedy C. Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift.

9. Anexos

9.1. Repositorio de GitHub

El código fuente completo del proyecto, junto con la documentación necesaria para su comprensión y reutilización, se encuentra disponible públicamente en el siguiente repositorio de GitHub:

Repositorio: <https://github.com/noeliafernandezesp/torsional-diffusion-sidechain>

9.1.1. Descripción y estructura del repositorio

El repositorio tiene todo el material técnico necesario para reproducir el experimento descrito en esta memoria, desde la obtención del dataset hasta la evaluación final del modelo. A continuación, se describe el contenido de los ficheros principales en la tabla 9:

Tabla 9. Elementos del repositorio de GitHub

Archivo / Carpeta	Descripción	Relevancia
<i>TFM_script.ipynb</i>	Notebook principal de Google Colab con el pipeline completo: descarga de datos, preprocesamiento, entrenamiento de ChiGNN y evaluación bioestadística.	Principal
<i>TFM_script.py</i>	Versión Python equivalente del notebook, exportada para su uso en entornos sin Jupyter.	Secundario
<i>README.md</i> <i>README_EN.md</i>	y Documentación del repositorio: descripción del proyecto, requisitos de instalación e instrucciones de ejecución paso a paso. (Español e Inglés).	Documentación
<i>requirements.txt</i>	Lista de dependencias con versiones fijadas (PyTorch 2.10.0, PyTorch Geometric, Biopython, py3Dmol, etc.)	Reproducibilidad

Archivo / Carpeta	Descripción	Relevancia
	para garantizar la reproducibilidad del entorno.	
<i>TFM_Noelia_Fernandez.pdf</i>	Memoria completa del Trabajo Final de Máster.	Sumario del proyecto

9.1.2. Alcance del código

El script principal (TFM_script.ipynb / TFM_script.py) implementa un pipeline completo y autocontenido estructurado en 25 bloques numerados que corresponden a las tres fases del proyecto:

- **Fase 1 — Cimentación y preparación de datos (Bloques 0–11):** Panel de control del pipeline con flags de re-ejecución; verificación del entorno GPU (Tesla T4, CUDA 12.8); instalación de dependencias con versiones fijadas (PyTorch 2.10.0, PyTorch Geometric, Biopython 1.87, py3Dmol 2.5.4.); montaje de Google Drive; consulta a la API REST de RCSB PDB con filtros de resolución ($<2,0$ Å), método (cristalografía de rayos X) y tipo de polímero; descarga de 597 estructuras re-refinadas de PDB-REDO; extracción de ángulos diedros χ_1 – χ_4 mediante Biopython; construcción del dataset de grafos en formato PyTorch Geometric con máscara booleana y_mask ; y partición 80/10/10 a nivel de proteína (semilla fija `random.seed(42)`).
- **Fase 2 — Implementación del modelo (Bloques 12–18):** Definición del proceso forward de difusión torsional con ruido Von Mises y schedule lineal de concentración $\kappa \in [0,01; 10,0]$; implementación de la arquitectura ChiGNN (80.404 parámetros, 4 capas GCNConv con BatchNorm y conexiones residuales); bucle de entrenamiento con AdamW, CosineAnnealingLR y gradient clipping; mecanismo de checkpointing sobre Google Drive; y proceso reverse DDIM adaptado para muestreo de conformaciones.
- **Fase 3 — Evaluación bioestadística (Bloques 19–25):** Carga del mejor checkpoint; generación de predicciones sobre el conjunto de test; cálculo de MAE circular e IC 95% bootstrap; análisis de error por tipo de aminoácido; generación de rose plots comparativos; análisis de calibración de confianza mediante varianza circular de Von Mises y correlación de Spearman; benchmarking frente a baselines aleatorio y modal; y exportación de todas las figuras y métricas a Google Drive.

9.1.3. Licencia y citación

El repositorio se distribuye bajo licencia MIT, lo que permite su uso, modificación y redistribución libre con atribución a la autora. Para citar este trabajo en publicaciones académicas, se recomienda referenciar tanto la memoria del TFM como el repositorio de GitHub.

9.2. Reproducibilidad del proyecto

La reproducibilidad es un principio fundamental del método científico y uno de los compromisos explícitos de este trabajo. Esta sección proporciona las instrucciones detalladas para reproducir íntegramente los resultados reportados en el Capítulo 4, partiendo de cero o aprovechando los artefactos intermedios ya generados.

9.2.1. Requisitos del entorno

El proyecto fue desarrollado y validado íntegramente en Google Colaboratory (runtime 2026.04) con el siguiente entorno de hardware y software:

- **Hardware:** GPU NVIDIA Tesla T4 (15,6 GB VRAM, CUDA 12.8). La ejecución sobre CPU es técnicamente posible pero computacionalmente inviable para el volumen de datos del proyecto (tiempo estimado de entrenamiento en CPU: >48 horas frente a ~3 horas en GPU).
- **Sistema operativo:** Ubuntu 22.04.5 LTS (entorno Colab runtime 2026.04).
- **Python:** 3.12.13 (versión del runtime de Google Colab 2026.04).
- **Dependencias principales (pre-instaladas en el runtime):** PyTorch 2.10.0+cu128, NumPy 2.0.2, SciPy 1.17.1, Pandas 3.0.0, Matplotlib 3.10.8.
- **Dependencias instaladas vía pip en el Bloque 2 del notebook:** Biopython 1.87, py3Dmol 2.5.4, torch-scatter y torch-sparse (compiladas dinámicamente contra PyTorch 2.10.0 + CUDA 12.8 desde el índice de ruedas de PyG), torch-geometric 2.7.0, rdkit.

9.2.2. Reproducción completa desde cero (primera ejecución)

Para reproducir el experimento completo partiendo de un entorno limpio, siga los pasos indicados a continuación. El tiempo total estimado de ejecución es de aproximadamente 3–4 horas con GPU T4.

- **Paso 1 — Preparar el entorno de Google Colab.** Abrir el notebook TFM_script.ipynb desde el repositorio de GitHub directamente en Google Colab mediante el badge «Open in Colab» del README. Verificar que el entorno de ejecución tiene asignado un acelerador T4 GPU (Entorno de ejecución → Cambiar tipo de entorno de ejecución → T4 GPU). Opcionalmente, fijar el runtime a la versión 2026.04 para máxima reproducibilidad.
- **Paso 2 — Montar Google Drive.** En el Bloque 5 del notebook, ejecutar la celda de montaje de Drive. Se abrirá un diálogo de autorización; conceder el permiso solicitado. El notebook creará automáticamente la estructura de carpetas MyDrive/TFM_SideChain/ para almacenar todos los artefactos persistentes.
- **Paso 3 — Activar el modo de primera ejecución.** En el Bloque 0 (Panel de Control), establecer FORCE_ALL = True. Esto garantiza que todos los bloques se ejecutan desde cero sin intentar cargar artefactos de Drive.
- **Paso 4 — Ejecutar el pipeline completo.** Usar la opción Entorno de ejecución → Ejecutar todo (Ctrl+F9). El notebook ejecutará secuencialmente los 25 bloques. Los artefactos principales (dataset_torsional_diffusion.pt, chignn_best_checkpoint.pt, test_predictions.pt) se guardarán automáticamente en Google Drive al completarse cada fase.
- **Paso 5 — Verificar los resultados.** Al finalizar el Bloque 25, la tabla resumen consolidada mostrará todas las métricas de evaluación. Los valores esperados de referencia son: MAE circular $\chi_1 = 56,41^\circ$ [IC 95%: 55,63°; 57,15°], recuperación $\pm 40^\circ = 53,9\%$, coeficiente de Spearman $\rho = 0,299$.

La descarga del dataset utilizado se realizó el 2 de abril de 2026. Se guardó automáticamente en las dependencias de Google Drive, bajo el nombre pdb_data.

9.2.3. Reproducción parcial (sesión de reanudación)

En sesiones posteriores a la primera ejecución, cuando los artefactos ya están guardados en Google Drive, es posible reproducir únicamente la Fase 3 (evaluación) en pocos minutos:

- En el Bloque 0, dejar todos los flags en False (configuración por defecto).
- Ejecutar todo el notebook: los Bloques 7–17 cargarán automáticamente los artefactos desde Drive en lugar de regenerarlos.
- Los Bloques 18–25 cargarán el checkpoint del modelo y las predicciones de test, y regenerarán todas las figuras y métricas de evaluación.
- Tiempo estimado de la sesión de reanudación: ~8-9 minutos.

9.2.4. Semilla de aleatoriedad y determinismo

Para garantizar la reproducibilidad exacta de los resultados, el pipeline fija las siguientes semillas de aleatoriedad en los puntos críticos del código:

- `random.seed(42)`: aplicada antes de la partición del dataset para garantizar la misma distribución train/val/test en cualquier ejecución.
- `torch.manual_seed(42)` y `torch.cuda.manual_seed_all(42)`: aplicadas antes de la inicialización del modelo para reproducir los pesos iniciales de la red.
- El schedule de ruido Von Mises es determinista dado el paso de tiempo t , ya que emplea `torch.distributions.VonMises` con parámetros fijos.

9.2.5. Validación rápida de la instalación

Para comprobar que el entorno está correctamente configurado antes de ejecutar el pipeline completo, se puede ejecutar únicamente el Bloque 1 (verificación de GPU) y el Bloque 2 (instalación de dependencias). La salida esperada del Bloque 1 es:

☒ GPU detectada: Tesla T4 Memoria total : 15.6 GB Versión CUDA : 12.8

Si la GPU no es detectada, activar la aceleración de hardware siguiendo las instrucciones del comentario en el Bloque 1 del notebook antes de continuar.

9.3. Secuencia de código del proyecto y salidas

FASE 1: CIMENTACIÓN Y PREPARACIÓN DE LOS DATOS

BLOQUE 0:

```
# =====
# BLOQUE 0 – Panel de Control del Pipeline
# =====
# Define qué bloques costosos deben re-ejecutarse y cuáles
# pueden cargarse directamente desde Drive.
#
# USO NORMAL (sesión nueva, archivos ya en Drive):
#   Deja todos los flags en False → "Run All" tarda ~5 min.
#
# USO FORZADO (quieres regenerar algo desde cero):
#   Pon en True solo el flag del bloque que quieres re-ejecutar.
#   Los bloques siguientes que dependan de él se re-ejecutarán
#   automáticamente por efecto cascada (ver lógica abajo).
#
# PRIMERA VEZ (Drive vacío, sin ningún archivo):
#   Pon FORCE_ALL = True o ejecuta los bloques manualmente
#   en orden la primera vez.
# =====

# — Switch maestro —————
# True = regenera TODO ignorando lo que haya en Drive
FORCE_ALL = False

# — Switches individuales por fase —————
# Solo tienen efecto si FORCE_ALL = False.
FORCE_DOWNLOAD    = False    # Bloque 7: re-descargar PDBs de PDB-
REDO
FORCE_EXTRACTION  = False    # Bloque 8: re-extraer ángulos chi y
coords
FORCE_DATASET     = False    # Bloque 9: re-construir grafos y split
FORCE_TRAINING    = False    # Bloque 16: re-entrenar el modelo GNN
FORCE_INFERENCE   = False    # Bloque 18: re-generar predicciones de
test

# — Lógica de cascada —————
# Si una fase se re-ejecuta, todas las fases que dependen
# de ella también deben re-ejecutarse para mantener coherencia.
# Ejemplo: si re-extraes ángulos, el dataset y el modelo
# también deben regenerarse con los nuevos datos.
if FORCE_ALL:
```

```

FORCE_DOWNLOAD = FORCE_EXTRACTION = FORCE_DATASET = True
FORCE_TRAINING = FORCE_INFERENCE = True

if FORCE_DOWNLOAD:
    FORCE_EXTRACTION = True    # Nuevos PDBs → nueva extracción

if FORCE_EXTRACTION:
    FORCE_DATASET = True       # Nuevos ángulos → nuevo dataset

if FORCE_DATASET:
    FORCE_TRAINING = True      # Nuevo dataset → nuevo entrenamiento

if FORCE_TRAINING:
    FORCE_INFERENCE = True     # Nuevo modelo → nuevas predicciones

# — Resumen del estado del pipeline —————
flags = {
    'Descarga PDBs      (Bloque 7) ': FORCE_DOWNLOAD,
    'Extracción ángulos(Bloque 8) ': FORCE_EXTRACTION,
    'Construcción dataset(Bloque 9) ': FORCE_DATASET,
    'Entrenamiento GNN (Bloque 16) ': FORCE_TRAINING,
    'Inferencia test   (Bloque 18) ': FORCE_INFERENCE,
}

print("🔧 Panel de Control del Pipeline")
print(f"{'-'*45}")
for nombre, flag in flags.items():
    estado = "🔄 RE-EJECUTAR" if flag else "⚡ CARGAR DESDE DRIVE"
    print(f" {nombre}: {estado}")
print(f"{'-'*45}")

```

SALIDA BLOQUE 0:

```

🔧 Panel de Control del Pipeline
-----
Descarga PDBs      (Bloque 7) : ⚡ CARGAR DESDE DRIVE
Extracción ángulos(Bloque 8) : ⚡ CARGAR DESDE DRIVE
Construcción dataset(Bloque 9): ⚡ CARGAR DESDE DRIVE
Entrenamiento GNN (Bloque 16): ⚡ CARGAR DESDE DRIVE
Inferencia test   (Bloque 18): ⚡ CARGAR DESDE DRIVE
-----

```

BLOQUE 1:

```

#
=====
===
# BLOQUE 1 - Verificación del entorno de cómputo (GPU)
#
=====
===
# Antes de cualquier operación, comprobamos que Google Colab
# tiene asignada una GPU. Este proyecto requiere GPU
obligatoriamente:
# el entrenamiento de la GNN y el proceso de difusión torsional
# son inviables en CPU por el volumen de datos.
#
# Si la salida es ✖, haremos lo siguiente:
# Activar la GPU: Entorno de ejecución -> Cambiar tipo de entorno
de ejecución
# -> acelerador de hardware -> T4 GPU -> Guardar
#
=====
===
import torch

# torch.cuda.is_available() devuelve True solo si:
# 1. PyTorch fue instalado con soporte CUDA
# 2. Colab ha asignado una GPU a esta sesión
if torch.cuda.is_available():
    print(f"☑ GPU detectada: {torch.cuda.get_device_name(0)}")
    print(f"    Memoria total :
{torch.cuda.get_device_properties(0).total_memory / 1e9:.1f} GB")
    print(f"    Versión CUDA   : {torch.version.cuda}")
else:
    print(f"✖ GPU no detectada. Revisa la configuración del
entorno.")

```

SALIDA BLOQUE 1:

```

☑ GPU detectada: Tesla T4
Memoria total : 15.6 GB
Versión CUDA   : 12.8

```

BLOQUE 2:

```

#
=====
===
# BLOQUE 2 - Instalación de dependencias del proyecto

```

```

#
=====
===
# Interpolamos torch_version y cuda_version.
# PyTorch Geometric distribuye torch-scatter y torch-sparse como
# binarios precompilados, uno por cada combinación de versión de
# PyTorch y CUDA. Si se instalan sin especificar estas versiones,
# pip puede resolver versiones incompatibles y la GNN fallará
# en tiempo de ejecución con errores CUDA crípticos.
#
=====
===

import torch

# Extraemos versiones exactas, sin sufijos, para compatibilidad
torch_version = torch.__version__.split('+')[0]

# Extraemos la versión CUDA sin puntos, como espera la URL de PyG
cuda_version = torch.version.cuda.replace('.', '')

print(f"🔑 Torch: {torch_version} | CUDA: {cuda_version}")

# Biopython y visualización
!pip install -q biopython
!pip install -q py3Dmol

# PyTorch Geometric con versión exacta interpolada correctamente
!pip install -q torch-scatter -f https://data.pyg.org/whl/torch-
{torch_version}+cu{cuda_version}.html
!pip install -q torch-sparse -f https://data.pyg.org/whl/torch-
{torch_version}+cu{cuda_version}.html
!pip install -q torch-geometric

# Química computacional
!pip install -q rdkit

print("✅ Todas las dependencias instaladas correctamente.")

```

SALIDA BLOQUE 2:

```

🔑 Torch: 2.10.0 | CUDA: 128
-----
3.2/3.2 MB 30.1 MB/s eta 0:00:00
-----
10.9/10.9 MB 76.6 MB/s eta 0:00:00

```

5.4/5.4 MB 43.2 MB/s eta 0:00:00

63.7/63.7 kB 3.3 MB/s eta 0:00:00

1.3/1.3 MB 34.2 MB/s eta 0:00:00

37.0/37.0 MB 20.2 MB/s eta 0:00:00

☒ Todas las dependencias instaladas correctamente.

BLOQUE 3:

```
#
=====
===
# BLOQUE 3 - Importación de librerías del pipeline de datos
#
=====
===
# Cargamos todos los módulos necesarios para las fases de
# descarga, procesamiento y análisis del dataset de proteínas.
# Librerías importadas:
#   os                → gestión de rutas y directorios
#   requests          → descarga de archivos PDB desde PDB-REDO
#   numpy             → operaciones numéricas sobre ángulos y coords
#   pandas            → construcción y manipulación del DataFrame
#   matplotlib        → generación de figuras (Rose Plot de chil,
#                       histogramas de distribución rotamérica)
#   Bio.PDB           → parsing de estructuras PDB, extracción de
#                       vectores atómicos y cálculo de diedros chi
#
=====
===

import os
import requests
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import Bio.PDB
from Bio.PDB import PDBList, PDBParser, calc_dihedral

print("☒ Librerías cargadas correctamente.")
```

SALIDA BLOQUE 3:

☒ Librerías cargadas correctamente.

BLOQUE 4:

```
# =====
# BLOQUE 4 – Persistencia del proyecto en Google Drive
# =====
# Las sesiones de Google Colab son efímeras: cualquier archivo
# guardado en local (/content/) se pierde al desconectarse.
# Este bloque soluciona ese problema montando Drive como sistema
# de archivos permanente y definiendo todas las rutas del
# proyecto en variables globales.
#
# Estructura de carpetas que se crea en Drive:
#
#   MyDrive/
#   └─ TFM_SideChain/
#       ├── pdb_data/                ← archivos .pdb descargados
#       ├── dataset_sidechains_curated.csv ← dataset legible
#       └─ dataset_torsional_diffusion.pt ← dataset para la GNN
#
# IMPORTANTE – Centralización de rutas:
# Todas las rutas del pipeline se definen aquí. Si el proyecto se
# mueve a otra carpeta de Drive, basta con cambiar DRIVE_BASE en
# este
# bloque.
# =====

## Solicita autorización de Google y monta Drive en /content/drive
from google.colab import drive
drive.mount('/content/drive')

# Carpeta raíz del TFM en Drive
DRIVE_BASE = "/content/drive/MyDrive/TFM_SideChain"

# Subcarpeta que almacenará los archivos .pdb de PDB-REDO
PDB_DIR = f"{DRIVE_BASE}/pdb_data"

# CSV con los residuos, coordenadas y ángulos chi
# Formato legible: útil para inspección manual y figuras de la
# tesis
DATASET_CSV = f"{DRIVE_BASE}/dataset_sidechains_curated.csv"

# Archivo .pt serializado con los grafos de PyTorch Geometric
# (train / val / test). Es el input directo de la GNN en la Fase 2
DATASET_PT = f"{DRIVE_BASE}/dataset_torsional_diffusion.pt"

# — Creación de directorios —————
```

```
# exist_ok=True evita error si la carpeta ya existe
# (importante al reiniciar sesión sin perder los datos de Drive)
os.makedirs(PDB_DIR, exist_ok=True)

print("☑ Google Drive montado y rutas configuradas:")
print(f"    PDB_DIR      → {PDB_DIR}")
print(f"    DATASET_CSV   → {DATASET_CSV}")
print(f"    DATASET_PT    → {DATASET_PT}")
```

SALIDA BLOQUE 4:

```
Mounted at /content/drive
☑ Google Drive montado y rutas configuradas:
    PDB_DIR      → /content/drive/MyDrive/TFM_SideChain/pdb_data
    DATASET_CSV   → /content/drive/MyDrive/TFM_SideChain/dataset_sidechains_curated.csv
    DATASET_PT    → /content/drive/MyDrive/TFM_SideChain/dataset_torsional_diffusion.pt
```

BLOQUE 5:

```
# =====
# BLOQUE 5 – Consulta a la API del PDB: selección del dataset
# =====
# Consultamos la API REST del Protein Data Bank (RCSB) para
# obtener una lista de IDs de estructuras que cumplan los
# criterios de calidad del TFM. La selección no es arbitraria:
# cada filtro tiene una justificación estadística y estructural.
#
# Los 3 filtros aplicados en combinación AND:
#
# Filtro 1 – Resolución < 2.0 Å
# Las posiciones atómicas de las cadenas laterales solo son
# fiables por debajo de este umbral. A resoluciones mayores,
# los ángulos chi experimentales tienen un error asociado
# que contaminaría el entrenamiento de la GNN.
# (Criterio estándar en la literatura: Dunbrack, 2002)
#
# Filtro 2 – Método: X-RAY DIFFRACTION
# Excluimos NMR y Cryo-EM. La cristalografía de rayos X es el
# gold standard para ángulos diedros de cadenas laterales.
#
# Filtro 3 – Tipo de polímero: Protein
# Sin este filtro, la query devuelve estructuras de ADN/ARN
# que no tienen ángulos chi de aminoácidos.
# Su inclusión contaminaría el dataset con filas vacías.
#
```

```

# La API devuelve los resultados paginados en bloques de 'rows'.
# Con limite=1000 obtenemos una muestra manejable;
# =====

import requests

def obtener_lista_pdb(limite=1000):
    """
    Consulta la API REST de RCSB y devuelve una lista de IDs
    de estructuras proteicas de alta calidad.

    Parámetros
    -----
    limite : int
        Número máximo de entradas a recuperar (default: 1000).

    Retorna
    -----
    list[str]
        Lista de identificadores PDB de 4 caracteres (ej: '1A1X').
    """

    # Endpoint de búsqueda avanzada de RCSB
    url = "https://search.rcsb.org/rcsbsearch/v2/query"

    # — Construcción del query JSON —————
    # La API de RCSB usa un lenguaje de query propio basado en
    # nodos. El operador 'and' exige que los 3 filtros se cumplan
    # simultáneamente para que una entrada sea incluida.
    query = {
        "query": {
            "type": "group",
            "logical_operator": "and",
            "nodes": [
                {
                    # Filtro 1: Resolución < 2.0
                    "type": "terminal",
                    "service": "text",
                    "parameters": {
                        "attribute":
"rcsb_entry_info.resolution_combined",
                        "operator": "less",
                        "value": 2.0
                    }
                },
                {
                    # Filtro 2: Solo cristalografía de rayos X
                    "type": "terminal",

```



```

        "service": "text",
        "parameters": {
            "attribute": "exptl.method",
            "operator": "exact_match",
            "value": "X-RAY DIFFRACTION"
        }
    },
    {
        # Filtro 3: Solo proteínas
        "type": "terminal",
        "service": "text",
        "parameters": {
            "attribute":
"entity_poly.rcsb_entity_polymer_type",
            "operator": "exact_match",
            "value": "Protein"
        }
    }
]
},
# Pedimos los identificadores de entrada, no los átomos
"return_type": "entry",
"request_options": {
    # Paginación: recuperamos 'limite' entradas desde la 0
    "paginate": {"start": 0, "rows": limite}
}
}

# — Ejecución de la consulta —————
response = requests.post(url, json=query)

if response.status_code == 200:
    resultados = response.json().get('result_set', [])
    # Extraemos solo el identificador PDB de 4 caracteres
    return [item['identifier'] for item in resultados]
else:
    print(f"❌ Error en la query: {response.status_code}")
    return []

# — Ejecución —————
lista_ids = obtener_lista_pdb(1000)
print(f"✅ IDs de proteínas encontrados: {len(lista_ids)}")
print(f"    Primeros 10: {lista_ids[:10]}")

```

SALIDA BLOQUE 5:

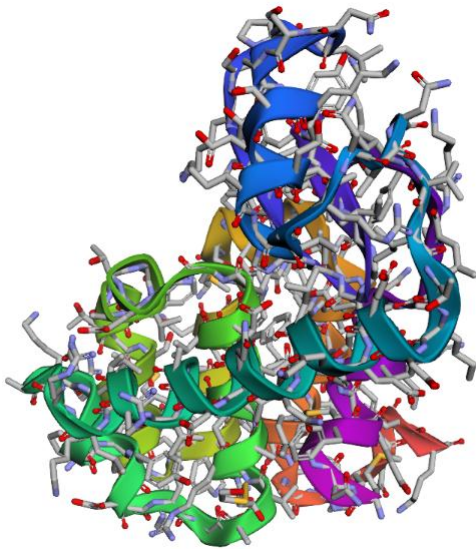
✅ IDs de proteínas encontrados: 1000

```
Primeros 10: ['102L', '102M', '103L', '104M', '106M', '107L',  
'108L', '109L', '109M', '10AF']
```

BLOQUE 6:

```
# =====  
# BLOQUE 6 – Visualización de la proteína 102L  
# =====  
import py3Dmol  
  
# Visualizamos la proteína 102L  
view = py3Dmol.view(query='pdb:102L')  
view.setStyle({'cartoon': {'color': 'spectrum'}}) # Esqueleto en  
estilo cinta  
view.addStyle({'stick': {}}) # Cadenas  
laterales en estilo varillas  
view.zoomTo()  
view.show()
```

SALIDA BLOQUE 6:



BLOQUE 7:

```
# =====  
# BLOQUE 7 – Descarga de estructuras optimizadas desde PDB-REDO  
# =====  
# Descargamos los archivos .pdb desde PDB-REDO en lugar del  
# PDB original porque PDB-REDO es una base de datos que re-refina  
# automáticamente cada estructura del PDB usando algoritmos  
modernos
```

```

# de refinamiento cristalográfico (REFMAC5). El resultado son
# coordenadas atómicas más precisas, con menos errores de
# empaquetamiento y mejor geometría de enlace.
# Para un modelo de difusión torsional, entrenarse sobre
# ángulos chi más precisos se traduce directamente en
# predicciones más exactas.
#
# Nomenclatura de PDB-REDO:
#   https://pdb-redo.eu/db/{pid}/{pid}_final.pdb
#   '{pid}_final.pdb' es la estructura completamente re-refinada,
#   lista para análisis estructural. Es el archivo de mayor
#   calidad que ofrece PDB-REDO.
#
# Gestión de sesiones de Colab:
#   Los archivos se guardan en PDB_DIR (Drive).
#   Si la sesión se reinicia, la comprobación os.path.exists()
#   evita re-descargar los archivos ya presentes en Drive,
#   ahorrando tiempo.
#
# Cobertura esperada:
#   No todas las entradas del PDB tienen una versión en PDB-REDO.
#   Estructuras muy antiguas o con problemas de refinamiento
#   pueden estar ausentes (status 404 → ⚠).
#
# Versión con guard de existencia: si los archivos PDB ya
# están en Drive y FORCE_DOWNLOAD = False, se omite la
# descarga y se carga directamente la lista de rutas.
# =====

```

```
import os, requests
```

```

def descargar_pdb_redo(lista_ids):
    """
    Descarga los archivos '_final.pdb' de PDB-REDO para cada ID
    de la lista y los guarda en Drive. Omite los ya descargados.

    Parámetros
    -----
    lista_ids : list[str]
        Lista de identificadores PDB de 4 caracteres obtenida
        (ej: ['1A1X', '1B2Y', ...]).

    Retorna
    -----
    list[str]
        Lista de rutas locales (en Drive) de los archivos
        descargados correctamente.
    """

```

```

# URL raíz del repositorio de PDB-REDO
base_url = "https://pdb-redo.eu/db"
rutas_finales = []

print("Iniciando descarga optimizada de PDB-REDO...")

for pdb_id in lista_ids:

    # PDB-REDO usa IDs en minúsculas en su estructura de URLs
    pid = pdb_id.lower()
    url = f"{base_url}/{pid}/{pid}_final.pdb"

    # Ruta de destino en Drive
    file_path = f"{PDB_DIR}/{pid}.pdb"

    # — Comprobación de caché —————
    # Si el archivo ya existe en Drive (sesión reiniciada),
    # lo añadimos directamente sin volver a descargarlo
    if os.path.exists(file_path):
        rutas_finales.append(file_path)
        continue

    # — Descarga con manejo de errores —————
    try:
        # timeout=10s evita bloqueos ante servidores lentos
        r = requests.get(url, timeout=10)

        if r.status_code == 200:
            # Escritura en modo binario ('wb') para preservar
            # el formato exacto del archivo .pdb
            with open(file_path, "wb") as f:
                f.write(r.content)
            rutas_finales.append(file_path)
            print(f"✅ {pid} descargado.")
        else:
            # 404 más frecuente: estructura sin versión en PDB-
REDO
            print(f"⚠️ {pid} no disponible en PDB-REDO,
omitiendo...")

    except Exception as e:
        # Captura timeouts, errores de red y problemas de
escritura
        print(f"❌ Error descargando {pid}: {e}")

return rutas_finales

```

```

# — Guard principal —————
# Comprobamos cuántos .pdb hay ya en Drive antes de decidir.
archivos_existentes = [
    f"{PDB_DIR}/{f}" for f in os.listdir(PDB_DIR)
    if f.endswith('.pdb')
] if os.path.exists(PDB_DIR) else []

if not FORCE_DOWNLOAD and len(archivos_existentes) > 0:
    # — Ruta rápida: archivos ya en Drive —————
    descargados = archivos_existentes
    print(f"⚡ Descarga omitida — {len(descargados)} archivos PDB")
    print(f"    ya disponibles en Drive: {PDB_DIR}")
    print(f"    (Pon FORCE_DOWNLOAD = True para re-descargar)")
else:
    # — Ruta completa: descarga desde PDB-REDO —————
    if FORCE_DOWNLOAD:
        print("📦 FORCE_DOWNLOAD = True → re-descargando todos los
PDBs...")
    else:
        print("🔍 No se encontraron archivos PDB en Drive →
descargando...")
# 'lista_ids'
# 'descargados' es la lista de rutas que se usará
descargados = descargar_pdb_redo(lista_ids)
print(f"\n✅ Archivos listos en Drive: {len(descargados)}")

```

SALIDA BLOQUE 7:

```

⚡ Descarga omitida — 600 archivos PDB
ya disponibles en Drive:
/content/drive/MyDrive/TFM_SideChain/pdb_data
(Pon FORCE_DOWNLOAD = True para re-descargar)

```

BLOQUE 8:

```

# =====
# BLOQUE 8 — Extracción de ángulos chi y coordenadas del backbone
# =====
# Este bloque es el núcleo del pipeline de datos de la fase 1. Para
cada
# estructura descargada, extrae dos tipos de información:
#
#   Features (X) — Coordenadas 3D del backbone (N, CA, C):
#       Son la entrada de la GNN. Representan la geometría de la
#       cadena principal, que se asume conocida y fija. El modelo
#       aprenderá a predecir las cadenas laterales condicionado
#       a estas coordenadas.
#

```

```

# Targets (Y) – Ángulos diedros chi1, chi2, chi3, chi4:
#     Son lo que el modelo de difusión torsional debe predecir.
#     Se calculan a partir de los átomos de la cadena lateral
#     usando la función calc_dihedral de Biopython.
#
# Decisiones de filtrado justificadas:
#
#     GLY excluida → no tiene CB, por tanto no tiene ningún chi.
#     ALA excluida → tiene CB pero no tiene cadena lateral más allá;
#                     chi1 es geométricamente indefinido.
#     Backbone incompleto → si falta N, CA, C o CB, el residuo
#                           no puede usarse como nodo de grafo (sin coords)
#                           ni para calcular chi1 (sin CB). Se descarta.
#
# Orden secuencial garantizado:
#     sorted(chain, key=lambda r: r.get_id()[1]) ordena los residuos
#     por su número de secuencia antes de procesarlos. Sin esto,
#     el edge_index del Bloque 9 conectaría residuos en orden
#     arbitrario de lectura del PDB, no en orden peptídico real.
#
# Lógica de cálculo de chi (anidada por disponibilidad):
#     chi1 requiere: N, CA, CB, CG (o equivalente)
#     chi2 requiere: chi1 calculado + CD (o equivalente)
#     chi3 requiere: chi2 calculado + CE (o equivalente)
#     chi4 requiere: chi3 calculado + CZ (o equivalente)
#     Si un átomo intermedio no existe, los chi superiores son None.
#     Esto es biológicamente correcto: SER solo tiene chi1,
#     GLU tiene chi1-chi3, ARG tiene chi1-chi4.
#
# =====
# Guard: si el CSV curado ya existe en Drive y
# FORCE_EXTRACTION = False, se carga directamente.
# La extracción de ángulos es el bloque más lento de la Fase 1
# (~7-9 min para 598 proteínas). Evitarlo en sesiones
# posteriores supone un ahorro de tiempo significativo.
# =====

import Bio.PDB
import numpy as np
import pandas as pd
import os

# — Verificación de Drive antes de procesar —————
# Drive se puede desconectar durante la sesión (normal tras ~90min
# de inactividad o después de una descarga larga en el Bloque 7).
# Comprobamos accesibilidad ANTES de entrar al bucle para fallar
# con un mensaje claro en lugar de a mitad del procesamiento.
if not os.path.exists(PDB_DIR):

```

```

        raise OSError(
            f"✗ Drive desconectado o ruta no accesible: {PDB_DIR}\n"
            f"    Solución: ejecuta drive.mount('/content/drive', "
            f"force_remount=True) y vuelve a ejecutar este bloque."
        )

archivos_en_drive = [f for f in os.listdir(PDB_DIR) if
f.endswith('.pdb')]
if not archivos_en_drive:
    raise FileNotFoundError(
        f"✗ No se encuentran archivos .pdb en {PDB_DIR}\n"
        f"    Vuelve a ejecutar el Bloque 7 para descargarlos."
    )

print(f"☑ Drive accesible - {len(archivos_en_drive)} archivos .pdb
listos.")

def calcular_angulos_chi(pdb_file):
    """
    Extrae ángulos chi y coordenadas del backbone de un archivo
    PDB.
    Los residuos se ordenan por posición en la cadena para
    garantizar
    que el edge_index del Bloque 9 conecte vecinos reales.

    Parámetros
    -----
    pdb_file : str
        Ruta al archivo .pdb descargado de PDB-REDO.

    Retorna
    -----
    list[dict]
        Lista de diccionarios, uno por residuo válido, con
    columnas:
        pdb_id, residue_name, residue_idx, N_x/y/z, CA_x/y/z,
        C_x/y/z, chi1, chi2, chi3, chi4.
    """
    parser = Bio.PDB.PDBParser(QUIET=True) # QUIET suprime
warnings de PDB
    structure = parser.get_structure('protein', pdb_file)
    data = []

    for model in structure:
        for chain in model:

            # Orden secuencial por número de residuo (ej:
1,2,3,...,148)

```

```

# Imprescindible para que las aristas del grafo sean
correctas
residuos_ordenados = sorted(chain, key=lambda r:
r.get_id()[1])

for residue in residuos_ordenados:

    # — Filtro 1: Solo aminoácidos con cadena lateral
    _____

    # GLY: sin CB → chi1 imposible
    # ALA: CB terminal → chi1 indefinido
    if not Bio.PDB.is_aa(residue) or
residue.get_resname() in ['GLY', 'ALA']:
        continue

    # — Filtro 2: Backbone completo obligatorio
    _____

    # N, CA, C → coordenadas del nodo del grafo
    (features X)

    # CB → primer átomo de la cadena lateral
    (chi1)

    try:
        n_vec = residue['N'].get_vector()
        ca_vec = residue['CA'].get_vector()
        c_vec = residue['C'].get_vector()
        cb_vec = residue['CB'].get_vector()
    except KeyError:
        continue # Residuo sin backbone completo:
descartado

    res_name = residue.get_resname()
    res_id = residue.get_id()[1]

    # — Features X: coordenadas 3D del backbone
    _____

    # 9 valores por residuo: (N, CA, C) × (x, y, z)
    backbone_coords = {
        'N_x': n_vec[0], 'N_y': n_vec[1], 'N_z': n
_vec[2],
        'CA_x': ca_vec[0], 'CA_y': ca_vec[1], 'CA_z':
ca_vec[2],
        'C_x': c_vec[0], 'C_y': c_vec[1], 'C_z': c
_vec[2]
    }

    # — Targets Y: ángulos chi (diedros de cadena
lateral) —

```



```

# Estructura anidada: chi_n solo se calcula si
chi_{n-1}
# ha sido posible, reflejando la dependencia
atómica real.
# Los átomos gamma/delta/epsilon/zeta varían por
residuo:
#   CG → la mayoría (LEU, LYS, ARG...)
#   CG1 → ILE, VAL
#   OG → SER          OG1 → THR
#   SG → CYS, MET
chis = [None, None, None, None]
try:
    # chi1: N-CA-CB-CG (o equivalente gamma)
    gamma_atoms = [a for a in residue
                    if a.get_name() in
['CG', 'CG1', 'OG', 'SG', 'OG1']]
    if gamma_atoms:
        g_vec = gamma_atoms[0].get_vector()
        chis[0] = np.degrees(
            Bio.PDB.calc_dihedral(n_vec, ca_vec,
cb_vec, g_vec))

    # chi2: CA-CB-CG-CD (o equivalente delta)
    delta_atoms = [a for a in residue
                   if a.get_name() in
['CD', 'CD1', 'SD', 'OD1', 'ND1']]
    if delta_atoms:
        d_vec = delta_atoms[0].get_vector()
        chis[1] = np.degrees(
            Bio.PDB.calc_dihedral(ca_vec,
cb_vec, g_vec, d_vec))

    # chi3: CB-CG-CD-CE (o equivalente
epsilon)
    epsilon_atoms = [a for a in residue
                     if a.get_name() in
['CE', 'NZ', 'OE1', 'NE']]
    if epsilon_atoms:
        e_vec =
epsilon_atoms[0].get_vector()
        chis[2] = np.degrees(
            Bio.PDB.calc_dihedral(cb_vec,
g_vec, d_vec, e_vec))

    # chi4: CG-CD-CE-CZ (solo ARG y
LYS)
    zeta_atoms = [a for a in residue

```

```

                                                    if a.get_name() in
['CZ','NH1']]
                                                    if zeta_atoms:
                                                    z_vec    =
zeta_atoms[0].get_vector()
                                                    chis[3] = np.degrees(
                                                    Bio.PDB.calc_dihedral(g_vec
, d_vec, e_vec, z_vec))

                                                    except Exception:
                                                        # Si falla el cálculo geométrico (colinealidad,
etc.)
                                                        # dejamos los chis como None; el Bloque 9 lo
gestionará
                                                        pass

                                                    # — Consolidación de la fila
_____
                                                    row = {
                                                    'pdb_id':      os.path.basename(pdb_file).repl
ace('.pdb', ''),
                                                    'residue_name': res_name,
                                                    'residue_idx':  res_id,
                                                    **backbone_coords,
                                                    'chi1': chis[0], 'chi2': chis[1],
                                                    'chi3': chis[2], 'chi4': chis[3]
                                                    }
                                                    data.append(row)

                                                    return data

# — Lógica de Control (Guard) _____
if not FORCE_EXTRACTION and os.path.exists(DATASET_CSV):
    # RUTA RÁPIDA: Carga desde Drive
    df_dataset = pd.read_csv(DATASET_CSV)
    print(f"⚡ Extracción omitida – CSV cargado desde Drive:")
    print(f"    {DATASET_CSV}")
else:
    # RUTA COMPLETA: Extracción biopython
    if FORCE_EXTRACTION:
        print("🔍 FORCE_EXTRACTION = True → Re-extrayendo
ángulos...")
    else:
        print("🔍 CSV no encontrado → Iniciando extracción desde
PDBs...")

# — Procesamiento de todas las estructuras descargadas _____
todos_los_datos = []

```

```

# Usamos 'descargados' (lista de rutas locales)
for f in descargados:
    todos_los_datos.extend(calcular_angulos_chi(f))

# Construimos el DataFrame y eliminamos residuos sin chi1
# (chi1 es el mínimo necesario para cualquier análisis rotamérico)
df_dataset = pd.DataFrame(todos_los_datos)
df_dataset =
df_dataset.dropna(subset=['chi1']).reset_index(drop=True)

# IMPORTANTE: Guardar para la próxima sesión
df_dataset.to_csv(DATASET_CSV, index=False)
print(f"💾 Dataset guardado en: {DATASET_CSV}")

print(f"✅ Dataset listo: {len(df_dataset):,} residuos")
print(df_dataset[['pdb_id', 'residue_name', 'CA_x',
                  'chi1', 'chi2', 'chi3', 'chi4']].head())

```

SALIDA BLOQUE 8:

- ✅ Drive accesible – 600 archivos .pdb listos.
- ⚡ Extracción omitida – CSV cargado desde Drive:

```

/content/drive/MyDrive/TFM_SideChain/dataset_sidechains_curated.csv
💾 Dataset guardado en:
/content/drive/MyDrive/TFM_SideChain/dataset_sidechains_curated.csv
✅ Dataset listo: 156,765 residuos

```

	pdb_id	residue_name	CA_x	chi1	chi2	chi3	chi4
0	1021	MET	43.541000	-178.746933	69.873997	49.446649	NaN
1	1021	ASN	40.341999	58.309866	61.503282	NaN	NaN
2	1021	ILE	38.241001	-172.118877	56.565607	NaN	NaN
3	1021	PHE	40.318001	-72.442014	-89.929060	NaN	NaN
4	1021	GLU	43.597000	-63.943471	177.131167	4.331232	NaN

BLOQUE 9:

```

# =====
# BLOQUE 9 – Construcción del Dataset de Grafos Proteicos
#           y Partición Train / Val / Test
# =====
# Convierte el DataFrame de proteínas (df_dataset) en objetos
# torch_geometric.data.Data –un grafo por proteína– y los
# divide en conjuntos de entrenamiento, validación y test.
# El fichero .pt resultante es la entrada directa del modelo GNN.
# =====

```

```

# Guard: si el archivo .pt ya existe en Drive y
# FORCE_DATASET = False, se carga directamente.
# La construcción de grafos y el split son deterministas
# (random.seed(42)), por lo que cargar el .pt existente
# produce exactamente el mismo dataset que re-generarlo.
# =====

import torch
import numpy as np
import random
from torch_geometric.data import Data

# -----
# Diccionario de codificación de aminoácidos
# Mapea el nombre de tres letras al índice entero 0-19.
# El índice 20 se reserva para residuos no estándar o
# desconocidos (ej. ligandos heteroatómicos).
# -----
AA_TO_IDX = {
    'ALA': 0,  'ARG': 1,  'ASN': 2,  'ASP': 3,  'CYS': 4,
    'GLN': 5,  'GLU': 6,  'GLY': 7,  'HIS': 8,  'ILE': 9,
    'LEU': 10, 'LYS': 11, 'MET': 12, 'PHE': 13, 'PRO': 14,
    'SER': 15, 'THR': 16, 'TRP': 17, 'TYR': 18, 'VAL': 19
}

# — Guard principal —————
if not FORCE_DATASET and os.path.exists(DATASET_PT):
    # — Ruta rápida: .pt ya en Drive —————
    checkpoint = torch.load(DATASET_PT, weights_only=False)
    train_data = checkpoint['train']
    val_data = checkpoint['val']
    test_data = checkpoint['test']

    print(f"⚡ Construcción de grafos omitida — dataset cargado
desde Drive:")
    print(f"    {DATASET_PT}")
    print(f"    Train: {len(train_data)} | Val: {len(val_data)} | "
          f"Test: {len(test_data)}")
    print(f"    (Pon FORCE_DATASET = True para re-construir)")

else:
    # — Ruta completa: construcción de grafos desde df_dataset —
    if FORCE_DATASET:
        print("📦 FORCE_DATASET = True → re-construyendo
grafos...")
    else:
        print("📁 Dataset .pt no encontrado → construyendo
grafos...")

```

```

# -----
# 1. Persistencia del DataFrame en Google Drive
#   Se guarda antes de construir los grafos para garantizar
#   que el estado tabular queda respaldado aunque el bucle
#   falle a mitad.
# -----
df_dataset.to_csv(DATASET_CSV, index=False)
print(f"📄 CSV guardado en Drive: {DATASET_CSV}")

# -----
# 2. Bucle principal: un objeto Data por proteína (pdb_id)
# -----
lista_grafos = []
errores = 0

for pdb_id, group in df_dataset.groupby('pdb_id'):
    try:
        n = len(group)
        # Proteínas con un solo residuo no tienen aristas;
        # se descartan para evitar grafos degenerados.
        if n < 2:
            errores += 1
            continue

        # A. Coordenadas del backbone — tensor [N_residuos, 9]
        #   Cada fila contiene las coordenadas XYZ de los tres
        #   átomos principales: N, Cα y C (carbono carbonílico).
        #   Este tensor actúa como feature estructural del nodo.
        coords = torch.tensor(
            group[['N_x', 'N_y', 'N_z',
                    'CA_x', 'CA_y', 'CA_z',
                    'C_x', 'C_y', 'C_z']].values,
            dtype=torch.float
        )

        # B. Tipo de aminoácido — tensor [N_residuos] (long)
        #   Feature categórica esencial: la GNN necesita saber
        #   qué residuo es cada nodo para aprender los rotámeros
        #   específicos de cada aminoácido.
        aa_idx = torch.tensor(
            [AA_TO_IDX.get(r, 20) for r in group['residue_name']],
            dtype=torch.long
        )

        # C. Ángulos chi + máscara booleana — [N_residuos, 4]
        #   y_angles: valores chi1-chi4 en radianes; NaN → 0.0
        #   (el 0 es inocuo donde y_mask es False).

```

```

#     y_mask:    True solo donde existe un ángulo real
#                (Gly y Ala no tienen chi, por ejemplo).
#     La máscara se usará en la función de pérdida para
#     ignorar posiciones sin ángulo definido.
chi_vals = group[['chi1', 'chi2', 'chi3', 'chi4']].values
y_mask    = torch.tensor(~np.isnan(chi_vals),
dtype=torch.bool)
    y_angles = torch.tensor(np.nan_to_num(chi_vals, nan=0.0),
dtype=torch.float)

# D. Aristas secuenciales bidireccionales — [2, 2*(N-1)]
#     Se conecta cada residuo i con su vecino i+1 en ambas
#     direcciones (i→i+1 e i+1→i). Esto modela la cadena
#     polipeptídica como un grafo lineal no dirigido,
#     necesario para propagar contexto local de secuencia.
src = list(range(n - 1)) + list(range(1, n))
dst = list(range(1, n)) + list(range(n - 1))
edge_index = torch.tensor([src, dst], dtype=torch.long)

# Ensambla el objeto Data de PyG con todos los tensores
data_obj = Data(
    x            = coords,          # Coordenadas backbone
    aa_type      = aa_idx,          # Tipo de aminoácido
    edge_index    = edge_index,     # Conectividad secuencial
    y            = y_angles,        # Ángulos chi (target)
    y_mask        = y_mask,         # True = ángulo real
    pdb_id       = pdb_id           # Identificador PDB
(metadato)
)
lista_grafos.append(data_obj)

except Exception as e:
    # Proteínas con columnas faltantes u otros errores se
    # contabilizan pero no interrumpen el proceso.
    print(f"⚠ Error procesando {pdb_id}: {e}")
    errores += 1

# -----
# 3. Partición train / val / test a nivel de proteína
#     La división se hace ANTES de cualquier normalización para
#     evitar data leakage. Proporciones: 80 / 10 / 10.
#     random.seed(42) garantiza reproducibilidad del split.
# -----
random.seed(42)
random.shuffle(lista_grafos)
n_total = len(lista_grafos)

train_data = lista_grafos[:int(0.8 * n_total)]

```

```

val_data    = lista_grafos[int(0.8 * n_total):int(0.9 * n_total)]
test_data   = lista_grafos[int(0.9 * n_total):]

# -----
# 4. Serialización del dataset en Google Drive
#     Un único fichero .pt con las tres particiones indexadas
#     por clave ('train', 'val', 'test').
# -----
torch.save({
    'train': train_data,
    'val':   val_data,
    'test':  test_data
}, DATASET_PT)

# -----
# 5. Resumen estadístico del dataset construido
#     chi_disp = ángulos chi con valor experimental real
#     chi_total = posibles ángulos chi (N_residuos × 4)
# -----
chi_disp = sum(g.y_mask.sum().item() for g in lista_grafos)
chi_total = sum(g.y_mask.numel()     for g in lista_grafos)

print(f"\n💡 Dataset guardado en Drive: {DATASET_PT}")
print(f"{'-'*47}")
print(f"  Proteínas totales           : {n_total}")
print(f"  Train / Val / Test           : {len(train_data)} / {len(val_data)} / {len(test_data)}")
print(f"  Proteínas con error          : {errores}")
print(f"  Residuos totales             : {sum(g.num_nodes for g in lista_grafos):,}")
print(f"  Aristas totales              : {sum(g.edge_index.shape[1] for g in lista_grafos):,}")
print(f"  Ángulos chi reales           : {int(chi_disp):,} / {chi_total:,} ({100*chi_disp/chi_total:.1f}%)")
print(f"{'-'*47}")

# -----
# 6. Test de integridad sobre una muestra aleatoria
#     Verifica que todas las dimensiones de los tensores son
#     coherentes antes de pasar al entrenamiento del modelo.
# -----
random.seed(42)
muestra = random.choice(lista_grafos)
print(f"\n💡 Muestra aleatoria ({muestra.pdb_id}):")
print(f"  x (coords backbone) : {muestra.x.shape}")
print(f"  aa_type              : {muestra.aa_type.shape}")
print(f"  y (ángulos chi)     : {muestra.y.shape}")

```

```
print(f"  y_mask                : {muestra.y_mask.shape}  →
{muestra.y_mask.sum().item()} reales")
print(f"  edge_index            : {muestra.edge_index.shape}")
```

SALIDA BLOQUE 9:

⚡ Construcción de grafos omitida – dataset cargado desde Drive:

```
/content/drive/MyDrive/TFM_SideChain/dataset_torsional_diffusion.pt
Train: 479 | Val: 60 | Test: 60
(Pon FORCE_DATASET = True para re-construir)
```

📄 CSV guardado en Drive:

```
/content/drive/MyDrive/TFM_SideChain/dataset_sidechains_curated.csv
```

💡 Dataset guardado en Drive:

```
/content/drive/MyDrive/TFM_SideChain/dataset_torsional_diffusion.pt
```

Proteínas totales	: 599
Train / Val / Test	: 479 / 60 / 60
Proteínas con error	: 0
Residuos totales	: 156,765
Aristas totales	: 312,332
Ángulos chi reales	: 324,307 / 627,060 (51.7%)

🔍 Muestra aleatoria (13sv):

```
x (coords backbone) : torch.Size([267, 9])
aa_type              : torch.Size([267])
y (ángulos chi)      : torch.Size([267, 4])
y_mask               : torch.Size([267, 4]) → 530 reales
edge_index           : torch.Size([2, 532])
```

BLOQUE 10:

```
# =====
# BLOQUE 10 – Visualización de la Distribución del Ángulo  $\chi_1$ 
#           mediante Rose Plot (Histograma Polar)
# =====
# Genera un histograma circular (rose plot) del ángulo diedro
#  $\chi_1$  sobre todos los residuos del dataset. Al ser  $\chi_1$  un ángulo
# periódico, la representación polar es estadísticamente más
# honesta que un histograma cartesiano: evita el artefacto de
# corte en los extremos ( $\pm 180^\circ$ ) y permite ver de un vistazo
# los tres estados rotaméricos canónicos: gauche- ( $-60^\circ$ ),
# trans ( $180^\circ$ ) y gauche+ ( $+60^\circ$ ).
# =====
```

```
def plot_rose_chi1(df):
```

```
    # -----
    # 1. Limpieza del DataFrame
```



```

# Se eliminan las filas sin valor de  $\chi_1$  (NaN), que
# corresponden a residuos sin cadena lateral susceptible
# de tener este ángulo (Gly, Ala). Ignorarlos evita
# errores en np.histogram y sesgos en el conteo.
# -----
df_clean = df.dropna(subset=['chi1'])

# -----
# 2. Conversión de grados a radianes en  $[0, 2\pi]$ 
# Matplotlib polar trabaja en radianes. El operador % 360
# normaliza previamente valores fuera del rango  $[0^\circ, 360^\circ)$ 
# (p. ej.  $-60^\circ \rightarrow 300^\circ$ ), garantizando que todos los ángulos
# caigan dentro del círculo completo.
# -----
angles_rad = np.deg2rad(df_clean['chi1'] % 360)

# -----
# 3. Configuración del lienzo polar
# projection='polar' activa el sistema de coordenadas
# circular de Matplotlib; el tamaño cuadrado (8x8)
# evita que el círculo aparezca distorsionado.
# -----
plt.figure(figsize=(8, 8))
ax = plt.subplot(111, projection='polar')

# -----
# 4. Cálculo del histograma angular
# 36 bins de  $10^\circ$  cada uno ( $360^\circ / 10^\circ = 36$ ) nos dan
# resolución suficiente para distinguir los estados
# rotaméricos sin sobresegmentar la distribución.
# np.histogram devuelve los conteos por bin; las anchuras
# (widths) se calculan como la diferencia entre bordes
# consecutivos para que ax.bar rellene cada sector exacto.
# -----
bins = np.linspace(0.0, 2 * np.pi, 37) # 37 bordes  $\rightarrow$  36
bins
counts, _ = np.histogram(angles_rad, bins=bins)
widths = np.diff(bins) # Anchura uniforme
≈ 0.175 rad

# -----
# 5. Dibujo de las barras polares
# bins[:-1] posiciona cada barra en el borde izquierdo
# de su sector. bottom=0.0 hace que las barras arranquen
# desde el centro, maximizando la legibilidad de la
# magnitud relativa entre rotámeros.
# -----
ax.bar(bins[:-1], counts, width=widths, bottom=0.0,

```

```

        edgecolor='black', color='lightgreen', alpha=0.7)

# -----
# 6. Ajustes de orientación y etiquetas
#   set_theta_zero_location("N"): el 0° queda arriba
#   set_theta_direction(-1): sentido horario, coherente
#   con la representación de ángulos diedros en PDB.
#   Las etiquetas cada 45° permiten leer directamente los
#   tres estados rotaméricos canónicos de  $\chi_1$ :
#   gauche-  $\approx$  300° | trans  $\approx$  180° | gauche+  $\approx$  60°
# -----
ax.set_theta_zero_location("N")
ax.set_theta_direction(-1)

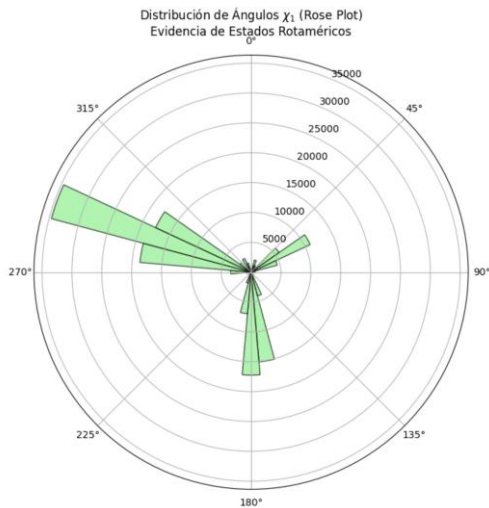
ticks_rad = np.linspace(0, 2 * np.pi, 8, endpoint=False)
ax.set_xticks(ticks_rad)
ax.set_xticklabels(['0°', '45°', '90°', '135°', '180°', '225°',
'270°', '315°'])

plt.title(
    r"Distribución de Ángulos  $\chi_1$  (Rose Plot)" +
    "\nEvidencia de Estados Rotaméricos",
    pad=20
)
plt.show()

# -----
# Ejecución sobre el dataset completo.
# La concentración de barras en torno a 60°, 180° y 300°
# confirmará que los rotámeros están correctamente codificados
# y que el dataset es biológicamente coherente antes de
# entrenar el modelo.
# -----
plot_rose_chi1(df_dataset)

```

SALIDA BLOQUE 10:



BLOQUE 11:

```
# =====
# BLOQUE 11 – Carga del Dataset y Configuración de los
#               DataLoaders para Entrenamiento
# =====
# Recupera el dataset de grafos serializado en Drive (Bloque 9)
# y lo envuelve en DataLoaders de PyTorch Geometric, que son
# el único punto de entrada de datos al bucle de entrenamiento.
# Un test de stress al final garantiza que el pipeline completo
# -carga → batch → tensores- funciona antes de iniciar
# el entrenamiento del modelo GNN.
# =====

import torch
from torch_geometric.loader import DataLoader

# -----
# 1. Recuperación del dataset desde Google Drive
#   weights_only=False es necesario porque el fichero .pt
#   contiene objetos Data de PyG (no solo tensores planos),
#   por lo que torch.load debe poder reconstruir esas clases.
#   El checkpoint es un diccionario con tres claves:
#   'train', 'val' y 'test'.
# -----
checkpoint = torch.load(DATASET_PT, weights_only=False)
train_data = checkpoint['train']
val_data   = checkpoint['val']
test_data  = checkpoint['test']

print(f"✅ Dataset cargado desde Drive")
print(f"  Train: {len(train_data)} | Val: {len(val_data)} | Test: {len(test_data)}")
```

```

# -----
# 2. Creación de los DataLoaders para cada partición
#   DataLoader de PyG extiende el de PyTorch estándar con
#   lógica de batching específica para grafos: concatena los
#   grafos del batch en un único grafo desconectado y ajusta
#   edge_index con offsets para que los índices de nodo sean
#   globalmente únicos dentro del batch.
#
#   batch_size=2: se usan solo 2 proteínas por batch dado
#   el límite de RAM/VRAM de Colab y el tamaño variable
#   de los grafos proteicos.
#
#   shuffle=True en train: garantiza que el modelo no aprenda
#   el orden del dataset, mejorando la generalización.
#   shuffle=False en val/test: asegura que las métricas son
#   comparables entre épocas (mismo orden de evaluación).
# -----
generator = torch.Generator()
generator.manual_seed(42)

train_loader = DataLoader(train_data, batch_size=2, shuffle=True,
                           generator=generator)
val_loader   = DataLoader(val_data,   batch_size=2, shuffle=False)
test_loader  = DataLoader(test_data,  batch_size=2, shuffle=False)

# -----
# 3. Test de stress sobre el train_loader
#   Se extrae un único batch con next(iter(...)) para
#   verificar que la cadena completa de carga y collation
#   funciona correctamente sin necesidad de iterar todo
#   el DataLoader. Se comprueban:
#   · num_graphs : proteínas consolidadas en el batch
#   · num_nodes  : residuos totales (suma de ambas proteínas)
#   · x          : coordenadas de backbone → [N_total, 9]
#   · aa_type    : tipo de aminoácido      → [N_total]
#   · y          : ángulos chi target      → [N_total, 4]
#   · y_mask     : máscara de ángulos reales → [N_total, 4]
#   · edge_index : aristas con offsets aplicados → [2, E]
#   Cualquier inconsistencia de forma aquí se propagaría
#   silenciosamente al modelo, por lo que este test es
#   imprescindible antes de empezar a entrenar.
# -----
try:
    batch = next(iter(train_loader))

    print(f"\n☑ Motor de datos listo para la Fase 2")

```

```

    print(f"{'-'*45}")
    print(f"  Proteínas en el batch      : {batch.num_graphs}")
    print(f"  Residuos totales              : {batch.num_nodes}")
    print(f"  x   (backbone coords)         : {batch.x.shape}")          #
[N_total, 9]
    print(f"  aa_type                      : {batch.aa_type.shape}")  #
[N_total]
    print(f"  y   (ángulos chi)             : {batch.y.shape}")          #
[N_total, 4]
    print(f"  y_mask                       : {batch.y_mask.shape}")    #
[N_total, 4]
    print(f"  Ángulos reales en batch : {batch.y_mask.sum().item()}
/ {batch.y_mask.numel()}")
    print(f"  edge_index                   : {batch.edge_index.shape}")
    print(f"{'-'*45}")

except Exception as e:
    # Un fallo aquí indica corrupción del fichero .pt,
    # incompatibilidad de versiones de PyG, o un grafo
    # mal construido en el Bloque 9.
    print(f"❌ Error al procesar el batch: {e}")

```

SALIDA BLOQUE 11:

- ☒ Dataset cargado desde Drive
Train: 479 | Val: 60 | Test: 60
- ☒ Motor de datos listo para la Fase 2

```

Proteínas en el batch      : 2
Residuos totales          : 526
x   (backbone coords)     : torch.Size([526, 9])
aa_type                   : torch.Size([526])
y   (ángulos chi)         : torch.Size([526, 4])
y_mask                    : torch.Size([526, 4])
Ángulos reales en batch   : 1109 / 2104
edge_index                : torch.Size([2, 1048])

```

FASE 2: IMPLEMENTACIÓN DEL MODELO DE IA

BLOQUE 12:

```

# =====
# BLOQUE 12 – Proceso Forward de Difusión Torsional
#           (Adición de Ruido Von Mises a los Ángulos  $\chi$ )
# =====
# Implementa el proceso forward  $q(\chi_t \mid \chi_0)$ : dado un ángulo

```

```

# chi real ( $\chi_0$ ), produce una versión "ruidosa"  $\chi_t$  para un
# paso de tiempo t. El ruido sigue la distribución Von Mises
# -la análoga circular de la gaussiana- lo que garantiza que
# el resultado sigue siendo un ángulo válido en  $[-\pi, \pi]$ .
#
# Intuición del proceso:
#   t=0   →  $\kappa$  muy alto   → distribución muy concentrada en  $\chi_0$ 
#           (el ángulo apenas ha cambiado)
#   t=T   →  $\kappa \approx 0$    → distribución uniforme sobre el círculo
#           (el ángulo es ruido puro, sin información)
#
# El modelo de denoising (Bloque 15) aprenderá a invertir
# exactamente este proceso: dado  $\chi_t$  y t, predecir  $\chi_0$ .
# =====

```

```

import torch
import numpy as np

```

```

# — Parámetros globales del proceso de difusión —————
# T: número de pasos de difusión. 100 es suficiente para un
#     modelo lite; DiffPack original usa T=1000.
T           = 100

```

```

#  $\kappa_{\text{max}}$ : concentración inicial (t=0). Valor alto → ruido mínimo.
#  $\kappa_{\text{min}}$ : concentración final (t=T). Valor cercano a 0 → ruido
#             total (distribución uniforme sobre el círculo).
KAPPA_MAX = 10.0
KAPPA_MIN = 0.01

```

```

def get_kappa(t: int) -> float:
    """
    Devuelve el parámetro de concentración  $\kappa_t$  para el paso t.
    Usa un schedule lineal:  $\kappa$  disminuye de KAPPA_MAX a KAPPA_MIN
    a medida que t aumenta de 0 a T.

    Un schedule coseno (como en DDPM) sería más suave, pero el
    lineal es más sencillo de interpretar estadísticamente.

    Parámetros
    -----
    t : int
        Paso de difusión actual ( $0 \leq t \leq T$ ).

    Retorna
    -----
    float
        Valor de  $\kappa_t$ .
    """

```

```

# Interpolación lineal:  $t=0 \rightarrow \kappa_{\max}$ ,  $t=T \rightarrow \kappa_{\min}$ 
return KAPPA_MAX - (KAPPA_MAX - KAPPA_MIN) * (t / T)

def von_mises_sample(mu: torch.Tensor, kappa: float) ->
torch.Tensor:
    """
    Muestrea ángulos de una distribución Von Mises  $VM(\mu, \kappa)$ .

    PyTorch no tiene Von Mises nativo hasta versiones recientes,
    así que usamos numpy y convertimos de vuelta a tensor.
    La muestra se normaliza al rango  $[-\pi, \pi]$  con torch.atan2.

    Parámetros
    -----
    mu      : torch.Tensor - ángulos medios (en radianes), forma [N]
    kappa   : float        - parámetro de concentración

    Retorna
    -----
    torch.Tensor - ángulos muestreados en  $[-\pi, \pi]$ , forma [N]
    """
    mu_np      = mu.cpu().numpy()
    # numpy.random.vonmises espera  $\mu$  en radianes y  $\kappa \geq 0$ 
    samples_np = np.random.vonmises(mu_np, kappa)
    return torch.tensor(samples_np, dtype=torch.float,
device=mu.device)

def forward_diffusion(chi_0: torch.Tensor,
                      mask: torch.Tensor,
                      t: int) -> torch.Tensor:
    """
    Aplica el proceso forward a un tensor de ángulos chi reales.
    Solo se añade ruido a las posiciones donde mask=True
    (ángulos con valor experimental); las posiciones enmascaradas
    se dejan a 0 para no contaminar el entrenamiento.

    Parámetros
    -----
    chi_0 : torch.Tensor - ángulos chi reales [N, 4] (radianes)
    mask  : torch.Tensor - máscara booleana [N, 4]
    t     : int          - paso de difusión ( $0 \leq t \leq T$ )

    Retorna
    -----
    torch.Tensor - ángulos ruidosos  $\chi_t$  [N, 4] (radianes)
    """
    kappa = get_kappa(t)

```

```

chi_rad = torch.deg2rad(chi_0)    # Convertimos a radianes
chi_t    = torch.zeros_like(chi_rad)

# Añadimos ruido solo en posiciones con ángulo real definido
chi_t[mask] = von_mises_sample(chi_rad[mask], kappa)
return chi_t

# — Test de sanidad del proceso forward —————
# Cargamos una muestra del dataset y verificamos que el ruido
# aumenta progresivamente con t.
checkpoint = torch.load(DATASET_PT, weights_only=False)
muestra    = checkpoint['train'][0]

chi_0 = muestra.y          # Ángulos originales [N, 4]
mask   = muestra.y_mask    # Máscara booleana     [N, 4]

# Verificamos que la distancia circular aumenta con t
print("📊 Test del proceso forward de difusión:")
print(f"{'-'*50}")
for t_test in [0, 10, 25, 50, 75, 100]:
    chi_t = forward_diffusion(chi_0, mask, t_test)
    # Distancia circular media entre  $\chi_0$  y  $\chi_t$  (en grados)
    #  $d(a,b) = |\text{atan2}(\sin(a-b), \cos(a-b))| \rightarrow$  convertido a grados
    chi_0_rad = torch.deg2rad(chi_0[mask])
    diff_rad = torch.atan2(
        torch.sin(chi_t[mask] - chi_0_rad),
        torch.cos(chi_t[mask] - chi_0_rad)
    )
    mae_deg = torch.abs(diff_rad).mean().item() * (180 / np.pi)
    kappa_t = get_kappa(t_test)
    print(f"  t={t_test:3d}  k={kappa_t:.3f}  MAE
circular={mae_deg:.2f}°")
print(f"{'-'*50}")
print("✅ El MAE debe aumentar progresivamente con t.")

```

SALIDA BLOQUE 12:

📊 Test del proceso forward de difusión:

t=	0	k=10.000	MAE circular=14.73°
t=	10	k=9.001	MAE circular=15.92°
t=	25	k=7.502	MAE circular=16.84°
t=	50	k=5.005	MAE circular=20.08°
t=	75	k=2.508	MAE circular=31.37°
t=	100	k=0.010	MAE circular=90.50°

✅ El MAE debe aumentar progresivamente con t.

BLOQUE 13:

```
# =====
# BLOQUE 13 – Visualización del Proceso Forward
#           (Cadenas Laterales Ruidosas en py3Dmol)
# =====
# Visualiza el efecto del ruido Von Mises sobre una proteína
# real del dataset. Muestra cómo las cadenas laterales se
# "desordenan" progresivamente al aumentar t, ilustrando
# visualmente el problema que el modelo debe aprender a invertir.

# "Visualización de cadenas laterales con ruido en py3Dmol".
# =====

import py3Dmol
import Bio.PDB
import copy
import numpy as np
import torch

def apply_chi1_to_residue(residue, chi1_rad: float):
    """
    Modifica el ángulo  $\chi_1$  de un residuo rotando el átomo CG
    (o equivalente) alrededor del enlace C $\alpha$ -C $\beta$ .

    Esta es una aproximación de primer orden que solo mueve
    el primer ángulo diedro, suficiente para la visualización.
    Para una reconstrucción completa se necesitaría reconstruir
    todos los átomos de la cadena lateral (fuera del alcance
    de este bloque de visualización).

    Parámetros
    -----
    residue   : Bio.PDB.Residue – residuo a modificar (in-place)
    chi1_rad  : float           – nuevo valor de  $\chi_1$  en radianes
    """
    try:
        n_vec = residue['N'].get_vector()
        ca_vec = residue['CA'].get_vector()
        cb_vec = residue['CB'].get_vector()

        # Átomo gamma: punto de aplicación de la rotación
        gamma_names = ['CG', 'CG1', 'OG', 'SG', 'OG1']
        gamma_atoms = [a for a in residue if a.get_name() in
gamma_names]
        if not gamma_atoms:
            return
```

```

cg_atom = gamma_atoms[0]
cg_vec  = cg_atom.get_vector()

# Calculamos el ángulo actual de  $\chi_1$ 
chil_actual = Bio.PDB.calc_dihedral(n_vec, ca_vec, cb_vec,
cg_vec)

# Rotación necesaria para pasar del ángulo actual al nuevo
delta = chil_rad - chil_actual

# Eje de rotación: enlace C $\alpha$ -C $\beta$  (normalizado)
axis = (cb_vec - ca_vec).normalized()

# Aplicamos la rotación a CG y todos los átomos más
distales
rotation = Bio.PDB.rotaxis(delta, axis)
new_coord = rotation.left_multiply(cg_vec - cb_vec) +
cb_vec
cg_atom.set_coord(new_coord.get_array())

except (KeyError, Exception):
    pass # Si el residuo no tiene los átomos necesarios, lo
ignoramoss

def structure_to_pdb_string(structure) -> str:
    """
    Convierte una estructura de Biopython a string PDB en memoria,
    sin necesidad de escribir en disco.
    """
    import io
    io_obj = Bio.PDB.PDBIO()
    io_obj.set_structure(structure)
    buffer = io.StringIO()
    io_obj.save(buffer)
    return buffer.getvalue()

def visualize_noise_effect(pdb_id: str, t_values: list):
    """
    Genera una visualización py3Dmol para cada valor de t en
    t_values, mostrando cómo el ruido Von Mises afecta
    progresivamente a las cadenas laterales.

    Parámetros
    -----
    pdb_id    : str - identificador de la proteína (ej: '1021')
    t_values  : list - lista de pasos de difusión a visualizar
    """

```

```

pdb_path = f"{PDB_DIR}/{pdb_id}.pdb"
if not os.path.exists(pdb_path):
    print(f"⚠️ {pdb_id}.pdb no encontrado en Drive.")
    return

parser = Bio.PDB.PDBParser(QUIET=True)
structure_original = parser.get_structure('prot', pdb_path)

for t_vis in t_values:
    # Clonamos la estructura para no modificar el original
    structure = copy.deepcopy(structure_original)
    kappa = get_kappa(t_vis)

    # Aplicamos ruido Von Mises al  $\chi_1$  de cada residuo válido
    n_modificados = 0
    for model in structure:
        for chain in model:
            for residue in chain:
                if not Bio.PDB.is_aa(residue):
                    continue
                if residue.get_resname() in ['GLY', 'ALA']:
                    continue
                try:
                    # Calculamos  $\chi_1$  actual
                    n_v = residue['N'].get_vector()
                    ca_v = residue['CA'].get_vector()
                    cb_v = residue['CB'].get_vector()
                    gamma_atoms = [a for a in residue
                                   if a.get_name() in
                                   ['CG', 'CG1', 'OG', 'SG', 'OG1']]
                    if not gamma_atoms:
                        continue
                    cg_v = gamma_atoms[0].get_vector()
                    chil_actual = Bio.PDB.calc_dihedral(n_v,
ca_v, cb_v, cg_v)

                    # Muestreamos nuevo  $\chi_1$  con ruido Von Mises
                    chil_noisy = float(von_mises_sample(
                        torch.tensor([chil_actual]), kappa
                    ))
                    apply_chil_to_residue(residue, chil_noisy)
                    n_modificados += 1
                except (KeyError, Exception):
                    continue

    # Renderizamos con py3Dmol
    pdb_str = structure_to_pdb_string(structure)
    view = py3Dmol.view(width=600, height=400)

```

```

view.addModel(pdb_str, 'pdb')
view.setStyle({'cartoon': {'color': 'lightblue'}})
view.addStyle({'stick': {'colorscheme': 'greenCarbon',
'radius': 0.15}})
view.zoomTo()

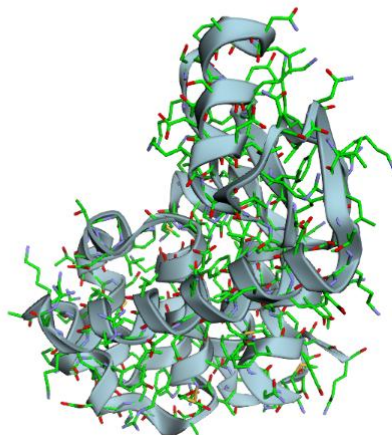
print(f"\n🔗 t={t_vis} | k={kappa:.3f} | Residuos
modificados: {n_modificados}")
print(f"    (Mayor t → cadenas laterales más desordenadas)")
view.show()

# — Ejecución: visualizamos t=0, t=25, t=50, t=100 —————
# Usamos la primera proteína disponible del dataset
primera_pdb = os.path.basename(descargados[0]).replace('.pdb', '')
visualize_noise_effect(primera_pdb, t_values=[0, 25, 50, 100])

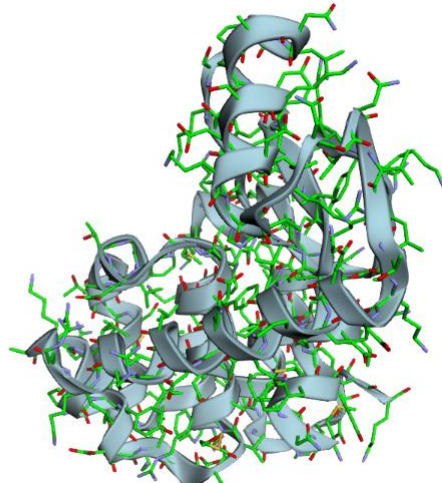
```

SALIDA BLOQUE 13:

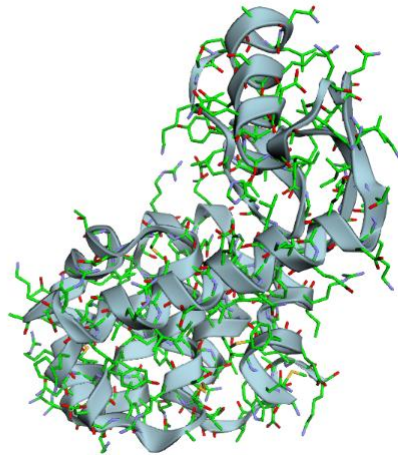
🔗 t=0 | k=10.000 | Residuos modificados: 135
(Mayor t → cadenas laterales más desordenadas)



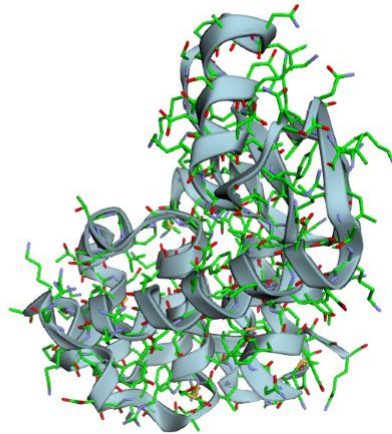
🔗 t=25 | k=7.502 | Residuos modificados: 135
(Mayor t → cadenas laterales más desordenadas)



 t=50 | $\kappa=5.005$ | Residuos modificados: 135
(Mayor t $\rightarrow$ cadenas laterales más desordenadas)



 t=100 | $\kappa=0.010$ | Residuos modificados: 135
(Mayor t $\rightarrow$ cadenas laterales más desordenadas)



BLOQUE 14:

```
# =====
# BLOQUE 14 – Preparación del Proceso de Difusión para
#               el Bucle de Entrenamiento
# =====
# Implementa la función que el bucle de entrenamiento llamará
# en cada iteración: dado un batch de grafos con ángulos reales,
# devuelve los ángulos ruidosos  $\chi_t$  y el paso de tiempo t
# muestreado aleatoriamente.
#
# Diseño de la función de pérdida:
# El modelo aprende a predecir  $\chi_0$  (los ángulos reales) a
# partir de  $\chi_t$  y t. La pérdida es la distancia circular
# entre el  $\chi_0$  predicho y el  $\chi_0$  real, calculada solo sobre
# las posiciones con máscara=True.
# Usar distancia circular y no MSE es crítico porque los
# ángulos son periódicos: el error entre  $179^\circ$  y  $-179^\circ$  es  $2^\circ$ ,
```

```

#   no 358°.
# =====

import torch
import numpy as np

def sample_t(batch_size: int) -> torch.Tensor:
    """
    Muestrea aleatoriamente un paso de tiempo t para cada grafo
    del batch. Muestrear t distinto por grafo equivale a
    entrenar simultáneamente en todos los niveles de ruido,
    lo que estabiliza el entrenamiento.

    Parámetros
    -----
    batch_size : int – número de grafos en el batch

    Retorna
    -----
    torch.Tensor – enteros en [1, T], forma [batch_size], en CPU
    """
    # t=0 se excluye porque no añade ruido (no hay nada que
    aprender)
    return torch.randint(1, T + 1, (batch_size,))

def prepare_diffusion_batch(batch, device: torch.device):
    """
    Prepara un batch para el entrenamiento del modelo de difusión.
    Para cada grafo del batch:
    1. Muestrea un paso de tiempo t aleatorio
    2. Aplica el proceso forward → obtiene  $\chi_t$  ruidoso
    3. Codifica t como feature adicional del nodo

    Parámetros
    -----
    batch : torch_geometric.data.Batch – batch de grafos
    device : torch.device – CPU o CUDA

    Retorna
    -----
    dict con:
        'chi_noisy' : tensor [N_total, 4] – ángulos ruidosos
        (radianes)
        'chi_target' : tensor [N_total, 4] – ángulos reales
        (radianes)
        'mask' : tensor [N_total, 4] – máscara booleana
        't_node' : tensor [N_total, 1] – t normalizado por nodo
    """

```

```

    't_batch'      : tensor [batch_size] - t entero por grafo
    """
    batch_size = batch.num_graphs
    t_batch     = sample_t(batch_size).to(device)          #
[B]    enteros
    t_norm      = t_batch.float() / T                      # [B]    en [0,1]

    # Ahora batch.batch (GPU) y t_norm (GPU) están en el mismo
device:
    # Expandimos t a nivel de nodo usando batch.batch:
    # batch.batch[i] = índice del grafo al que pertenece el nodo i
    t_per_node = t_norm[batch.batch].unsqueeze(1)          # [N, 1]

    # Convertimos ángulos target a radianes
    chi_target = torch.deg2rad(batch.y).to(device)          # [N,
4]
    mask        = batch.y_mask.to(device)                  # [N,
4]

    # Aplicamos el proceso forward con el t de cada grafo
    chi_noisy = torch.zeros_like(chi_target)

    # t_batch_cpu: la función get_kappa y von_mises_sample usan
    # numpy internamente, así que necesitan el valor en CPU/Python.
    t_batch_cpu = t_batch.cpu()

    for graph_idx in range(batch_size):
        t_i      = t_batch[graph_idx].item()
        node_mask = (batch.batch == graph_idx)             # Nodos de este
grafo
        chi_i     = chi_target[node_mask]                  # [n_i, 4]
        mask_i    = mask[node_mask]                        # [n_i, 4]

        # Añadimos ruido solo a posiciones con ángulo real
        chi_noisy_i = torch.zeros_like(chi_i)
        if mask_i.any():
            # von_mises_sample trabaja con numpy: movemos a CPU,
            # muestreamos, y devolvemos al device original.
            chi_noisy_i[mask_i] = von_mises_sample(
                chi_i[mask_i].cpu(), get_kappa(t_i)
            ).to(device)

        chi_noisy[node_mask] = chi_noisy_i

    return {
        'chi_noisy': chi_noisy,          # Input del modelo (ángulos
ruidosos)

```

```

        'chi_target': chi_target,    # Target del modelo (ángulos
reales)
        'mask':          mask,        # Solo estas posiciones cuentan
en la pérdida
        't_node':        t_per_node,  # Feature de tiempo por nodo
        't_batch':       t_batch      # Paso de tiempo por grafo
(para logs)
    }

```

```

def circular_loss(pred: torch.Tensor,
                  target: torch.Tensor,
                  mask: torch.Tensor) -> torch.Tensor:
    """
    Función de pérdida circular para ángulos diedros.
    Calcula el MAE circular solo sobre posiciones enmascaradas.

    La distancia circular entre dos ángulos a y b es:
         $d(a, b) = 1 - \cos(a - b)$ 
    Este es el "cosine loss" circular, equivalente a la distancia
    geodésica en  $S^1$  y diferenciable en todo el círculo.
    (Alternativa:  $|\text{atan2}(\sin(a-b), \cos(a-b))|$ , menos estable)

    Parámetros
    -----
    pred    : tensor [N, 4] – ángulos predichos (radianes)
    target  : tensor [N, 4] – ángulos reales (radianes)
    mask    : tensor [N, 4] – True donde hay ángulo real

    Retorna
    -----
    torch.Tensor – escalar, pérdida media sobre posiciones válidas
    """
    if not mask.any():
        return torch.tensor(0.0, requires_grad=True,
device=pred.device)

    # Cosine loss: 0 cuando pred == target, 2 cuando están opuestos
    loss = 1 - torch.cos(pred[mask] - target[mask])
    return loss.mean()

# — Test de la función de preparación —————
checkpoint = torch.load(DATASET_PT, weights_only=False)
train_loader = DataLoader(checkpoint['train'], batch_size=2,
shuffle=True)
device = torch.device('cuda' if torch.cuda.is_available()
else 'cpu')

```



```

batch = next(iter(train_loader)).to(device)
result = prepare_diffusion_batch(batch, device)

print("☑ Función de preparación del batch lista:")
print(f"{'-'*50}")
print(f"  chi_noisy   : {result['chi_noisy'].shape}    (input del
modelo)")
print(f"  chi_target  : {result['chi_target'].shape}    (target del
modelo)")
print(f"  mask        : {result['mask'].shape}    ({result['mask'].sum().item()} posiciones
válidas)")
print(f"  t_node      : {result['t_node'].shape}    (feature de
tiempo)")
print(f"  t_batch     : {result['t_batch'].tolist()}    (pasos de
tiempo del batch)")
print(f"{'-'*50}")

```

SALIDA BLOQUE 14:

☑ Función de preparación del batch lista:

```

chi_noisy   : torch.Size([367, 4])    (input del modelo)
chi_target  : torch.Size([367, 4])    (target del modelo)
mask        : torch.Size([367, 4])    (812 posiciones válidas)
t_node      : torch.Size([367, 1])    (feature de tiempo)
t_batch     : [72, 56]    (pasos de tiempo del batch)

```

BLOQUE 15:

```

# =====
# BLOQUE 15 – Definición de la Arquitectura GNN
#           (Red Neuronal de Grafos para Denoising)
# =====
# Define el modelo de denoising: una GNN que, dado un grafo
# proteico con ángulos ruidosos  $\chi_t$  y el paso de tiempo t,
# predice los ángulos originales  $\chi_0$ .
#
# Arquitectura: ChiGNN
#   Entrada por nodo:
#       · 9 features de backbone (N, C $\alpha$ , C coords)
#       · 1 embedding de aminoácido (dim 16)
#       · 4 ángulos chi ruidosos (senos y cosenos → 8 features)
#       · 1 feature de tiempo t normalizado
#   → Total: 9 + 16 + 8 + 1 = 34 features de entrada
#
#   Procesamiento:
#       · 4 capas GCNConv (Graph Convolutional Network)

```

```

#     · Función de activación: SiLU (suave, sin "dying ReLU")
#     · Batch normalization entre capas
#
# Salida por nodo:
#     · 4 valores en  $[-\pi, \pi]$  (uno por cada ángulo chi)
#     · Activación  $\tanh \times \pi$  para forzar rango angular correcto
#
# Por qué GCNConv y no EGNN:
#     GCNConv es suficiente porque nuestras features de entrada
#     ya incluyen las coordenadas del backbone: el modelo puede
#     aprender las relaciones geométricas a partir de ellas.
#     EGNN añadiría equivariancia estricta pero requeriría 3×
#     más memoria y es más difícil de depurar en Colab.
# =====

import torch
import torch.nn as nn
from torch_geometric.nn import GCNConv, global_mean_pool

class ChiGNN(nn.Module):
    """
    Red Neuronal de Grafos para denoising de ángulos chi.

    Parámetros
    -----
    hidden_dim : int — dimensión de los embeddings ocultos
    (default: 128)
    n_layers   : int — número de capas de paso de mensajes
    (default: 4)
    aa_embed   : int — dimensión del embedding de aminoácido
    (default: 16)
    """

    def __init__(self, hidden_dim: int = 128,
                  n_layers: int = 4,
                  aa_embed: int = 16):
        super().__init__()

        self.hidden_dim = hidden_dim
        self.n_layers = n_layers

        # — Embedding de tipo de aminoácido —————
        # 21 tipos (20 estándar + 1 desconocido) → vector de dim
aa_embed
        # Aprendible: el modelo descubrirá qué aminoácidos tienen
        # perfiles rotaméricos similares.
        self.aa_embedding = nn.Embedding(21, aa_embed)

```

```

# — Dimensión de entrada total —————
# 9 (backbone) + aa_embed (aminoácido) + 8 (sin/cos de 4
chi) + 1 (t)
input_dim = 9 + aa_embed + 8 + 1

# — Proyección inicial al espacio oculto —————
self.input_proj = nn.Sequential(
    nn.Linear(input_dim, hidden_dim),
    nn.SiLU()
)

# — Capas de convolución en grafo —————
# Cada capa agrega información de los residuos vecinos.
# BatchNorm1d estabiliza el entrenamiento normalizando
# las activaciones por batch.
self.conv_layers = nn.ModuleList([
    GCNConv(hidden_dim, hidden_dim)
    for _ in range(n_layers)
])
self.norms = nn.ModuleList([
    nn.BatchNorm1d(hidden_dim)
    for _ in range(n_layers)
])
self.acts = nn.ModuleList([
    nn.SiLU() for _ in range(n_layers)
])

# — Cabeza de predicción —————
# MLP de dos capas que mapea el embedding del nodo
# a 4 valores angulares en  $[-\pi, \pi]$ .
self.output_head = nn.Sequential(
    nn.Linear(hidden_dim, hidden_dim // 2),
    nn.SiLU(),
    nn.Linear(hidden_dim // 2, 4)
)

def encode_angles(self, chi: torch.Tensor) -> torch.Tensor:
    """
    Codifica los ángulos chi como pares (sin, cos) para
    preservar la periodicidad en la representación de entrada.

    Un ángulo  $\theta$  se representa como  $[\sin(\theta), \cos(\theta)]$ , lo que
    garantiza que ángulos cercanos en el círculo tengan
    representaciones similares (evita el salto discontinuo
    en  $\pm\pi$  que tendría  $\theta$  representado directamente).

    Parámetros
    -----

```

```

chi : tensor [N, 4] – ángulos en radianes

Retorna
-----
tensor [N, 8] – codificación sin/cos de los 4 ángulos
"""
return torch.cat([torch.sin(chi), torch.cos(chi)], dim=-1)

def forward(self, x, aa_type, edge_index, chi_noisy, t_node):
    """
    Paso forward de la red.

    Parámetros
    -----
    x          : tensor [N, 9]    – coordenadas backbone
    aa_type     : tensor [N]      – índice de aminoácido (long)
    edge_index  : tensor [2, E]   – aristas del grafo
    chi_noisy   : tensor [N, 4]   – ángulos ruidosos  $\chi_t$ 
    (radianes)
    t_node      : tensor [N, 1]   – paso de tiempo normalizado

    Retorna
    -----
    tensor [N, 4] – ángulos predichos  $\chi_0$  en  $[-\pi, \pi]$ 
    """
    # 1. Construir el vector de features del nodo
    aa_emb     = self.aa_embedding(aa_type)           # [N,
aa_embed]
    chi_enc    = self.encode_angles(chi_noisy)        # [N, 8]
    node_feat  = torch.cat([x, aa_emb, chi_enc, t_node], dim=-
1) # [N, input_dim]

    # 2. Proyección inicial
    h = self.input_proj(node_feat)                   # [N,
hidden_dim]

    # 3. Paso de mensajes iterativo con conexiones residuales
    # Las skip connections ( $h = h + h_{\text{new}}$ ) evitan el
    # desvanecimiento del gradiente en capas profundas.
    for conv, norm, act in zip(self.conv_layers, self.norms,
self.acts):
        h_new = conv(h, edge_index)
        h_new = norm(h_new)
        h_new = act(h_new)
        h      = h + h_new      # Conexión residual

    # 4. Predicción de ángulos
    #  $\tanh \times \pi$  fuerza la salida al rango  $[-\pi, \pi]$ ,

```

```

        # que es el dominio válido de los ángulos diedros.
        out = self.output_head(h) # [N, 4] sin
restricción
        out = torch.tanh(out) * torch.pi # [N, 4] en  $[-\pi, \pi]$ 
        return out

# — Instanciar el modelo y moverlo a GPU —————
device = torch.device('cuda' if torch.cuda.is_available() else
'cpu')
model = ChiGNN(hidden_dim=128, n_layers=4, aa_embed=16).to(device)

# — Resumen del modelo —————
n_params = sum(p.numel() for p in model.parameters() if
p.requires_grad)

print(f"☑ Modelo ChiGNN instanciado en: {device}")
print(f"{'-'*45}")
print(f"  Parámetros entrenables : {n_params:,}")
print(f"  Capas GCNConv           : {model.n_layers}")
print(f"  Dimensión oculta          : {model.hidden_dim}")
print(f"{'-'*45}")

# — Test de forward pass con un batch real —————
batch = next(iter(DataLoader(checkpoint['train'],
batch_size=2))).to(device)
diff_data = prepare_diffusion_batch(batch, device)

with torch.no_grad():
    pred = model(
        x = batch.x,
        aa_type = batch.aa_type,
        edge_index = batch.edge_index,
        chi_noisy = diff_data['chi_noisy'],
        t_node = diff_data['t_node']
    )

print(f"\n☑ Forward pass correcto:")
print(f"  Input chi_noisy : {diff_data['chi_noisy'].shape}")
print(f"  Output predicho : {pred.shape} — rango
[{pred.min():.2f}, {pred.max():.2f}] rad")

```

SALIDA BLOQUE 15:

☑ Modelo ChiGNN instanciado en: cuda

```

Parámetros entrenables : 80,404
Capas GCNConv           : 4
Dimensión oculta        : 128

```

☑ Forward pass correcto:
Input chi_noisy : torch.Size([250, 4])
Output predicho : torch.Size([250, 4]) - rango [-2.75, 3.07] rad

BLOQUE 16:

```
# =====
# BLOQUE 16 - Bucle de Entrenamiento
# =====
# Entrena el modelo ChiGNN durante N_EPOCHS épocas usando el
# optimizador AdamW y un scheduler de tasa de aprendizaje
# coseno. Guarda el mejor checkpoint en Drive según la pérdida
# de validación.
#
# Estructura del bucle por época:
#   1. TRAIN: para cada batch, prepara los ángulos ruidosos,
#       pasa por el modelo, calcula la pérdida circular y
#       actualiza los pesos.
#   2. VALIDATION: evalúa en val_loader sin actualizar pesos.
#       Guarda el modelo si mejora la pérdida de validación.
#
# Hiperparámetros elegidos para Colab T4:
#   · N_EPOCHS = 50   → suficiente para convergencia inicial
#   · LR = 1e-3       → tasa estándar para AdamW en GNNs
#   · batch_size = 2  → definido en Bloque 11
# =====
# Guard: si el checkpoint del modelo ya existe en Drive y
# FORCE_TRAINING = False, se carga directamente sin
# re-entrenar. Esto es especialmente útil cuando
# todos los bloques de evaluación solo necesitan
# el modelo entrenado, no re-entrenarlo.
# =====

import torch
import torch.optim as optim
import os

# --- Hiperparámetros ---
N_EPOCHS    = 50
LR          = 1e-3
CHECKPOINT_MODEL = f"{DRIVE_BASE}/chignn_best.pt"

# --- Guard principal ---
if not FORCE_TRAINING and os.path.exists(CHECKPOINT_MODEL):
    # --- Ruta rápida: modelo ya entrenado en Drive ---
    ckpt = torch.load(CHECKPOINT_MODEL, weights_only=False)
```

```

model.load_state_dict(ckpt['model_state_dict'])
model.eval()
history = ckpt['history']
best_epoch = ckpt['epoch']
best_val_loss = ckpt['val_loss']

print(f"⚡ Entrenamiento omitido – modelo cargado desde
Drive:")
print(f" {CHECKPOINT_MODEL}")
print(f" Mejor época: {best_epoch} | Val loss:
{best_val_loss:.4f}")
print(f" (Pon FORCE_TRAINING = True para re-entrenar)")

else:
    # — Ruta completa: entrenamiento desde cero —————
    if FORCE_TRAINING:
        print(f"🔍 FORCE_TRAINING = True → re-entrenando el
modelo...")
    else:
        print(f"🔍 Checkpoint no encontrado → entrenando desde
cero...")

# — Optimizador y scheduler —————
# AdamW incluye weight decay desacoplado, lo que regulariza
# los pesos sin afectar al sesgo (bias), mejorando la
# generalización respecto a Adam estándar.
optimizer = optim.AdamW(model.parameters(), lr=LR, weight_decay=1e-
4)

# CosineAnnealingLR: reduce la tasa de aprendizaje siguiendo
# una curva coseno de LR → 0 a lo largo de T_max épocas.
# Evita oscilaciones al final del entrenamiento.
scheduler = optim.lr_scheduler.CosineAnnealingLR(
    optimizer, T_max=N_EPOCHS, eta_min=1e-5
)

# — Registro del historial de pérdidas —————
history = {'train': [], 'val': []}
best_val_loss = float('inf')
best_epoch = 1

print(f"🚀 Iniciando entrenamiento: {N_EPOCHS} épocas")
print(f"{'-'*55}")

for epoch in range(1, N_EPOCHS + 1):
    # =====

```

```

# FASE 1: ENTRENAMIENTO
# =====
model.train()
train_loss_epoch = 0.0
n_batches_train = 0

for batch in train_loader:
    batch = batch.to(device)
    diff_data = prepare_diffusion_batch(batch, device)

    # Forward pass
    pred = model(
        x = batch.x,
        aa_type = batch.aa_type,
        edge_index = batch.edge_index,
        chi_noisy = diff_data['chi_noisy'],
        t_node = diff_data['t_node']
    )

    # Pérdida circular solo sobre ángulos reales
    loss = circular_loss(pred, diff_data['chi_target'],
diff_data['mask'])

    # Backward pass y actualización de pesos
    optimizer.zero_grad()
    loss.backward()

    # Gradient clipping: evita explosión del gradiente,
    # frecuente en GNNs con conexiones residuales.
    torch.nn.utils.clip_grad_norm_(model.parameters(),
max_norm=1.0)

    optimizer.step()
    train_loss_epoch += loss.item()
    n_batches_train += 1

avg_train_loss = train_loss_epoch / max(n_batches_train, 1)

# =====
# FASE 2: VALIDACIÓN
# =====
model.eval()
val_loss_epoch = 0.0
n_batches_val = 0

with torch.no_grad():
    for batch in val_loader:
        batch = batch.to(device)

```



```

diff_data = prepare_diffusion_batch(batch, device)

pred = model(
    x            = batch.x,
    aa_type      = batch.aa_type,
    edge_index   = batch.edge_index,
    chi_noisy    = diff_data['chi_noisy'],
    t_node       = diff_data['t_node']
)
loss = circular_loss(pred, diff_data['chi_target'],
diff_data['mask'])
val_loss_epoch += loss.item()
n_batches_val  += 1

avg_val_loss = val_loss_epoch / max(n_batches_val, 1)
scheduler.step()

# Guardamos historial para la curva de aprendizaje
history['train'].append(avg_train_loss)
history['val'].append(avg_val_loss)

# — Guardar mejor checkpoint —————
if avg_val_loss < best_val_loss:
    best_val_loss = avg_val_loss
    torch.save({
        'epoch':          epoch,
        'model_state_dict': model.state_dict(),
        'optimizer_state_dict': optimizer.state_dict(),
        'val_loss':       best_val_loss,
        'history':        history
    }, CHECKPOINT_MODEL)
    saved_marker = "💾 guardado"
else:
    saved_marker = ""

# Log cada 5 épocas para no saturar la salida
if epoch % 5 == 0 or epoch == 1:
    lr_actual = optimizer.param_groups[0]['lr']
    print(f"  Época {epoch:3d}/{N_EPOCHS}    "
          f"train={avg_train_loss:.4f}    "
          f"val={avg_val_loss:.4f}    "
          f"lr={lr_actual:.2e}    {saved_marker}")

print(f"{'-'*55}")
print(f"✅ Entrenamiento completado. Mejor val_loss:
{best_val_loss:.4f}")
print(f"  Checkpoint guardado en: {CHECKPOINT_MODEL}")

```

SALIDA BLOQUE 16:

⚡ Entrenamiento omitido – modelo cargado desde Drive:
/content/drive/MyDrive/TFM_SideChain/chignn_best.pt
Mejor época: 50 | Val loss: 0.1045
(Pon FORCE_TRAINING = True para re-entrenar)

🚀 Iniciando entrenamiento: 50 épocas

Época	1/50	train=0.1537	val=0.2073	lr=9.99e-04	📁 guardado
Época	5/50	train=0.1458	val=0.1508	lr=9.76e-04	
Época	10/50	train=0.1502	val=0.1749	lr=9.05e-04	
Época	15/50	train=0.1605	val=0.1230	lr=7.96e-04	
Época	20/50	train=0.1445	val=0.1515	lr=6.58e-04	
Época	25/50	train=0.1421	val=0.1098	lr=5.05e-04	
Época	30/50	train=0.1463	val=0.1383	lr=3.52e-04	
Época	35/50	train=0.1473	val=0.1246	lr=2.14e-04	
Época	40/50	train=0.1222	val=0.1623	lr=1.05e-04	
Época	45/50	train=0.1327	val=0.1374	lr=3.42e-05	
Época	50/50	train=0.1258	val=0.1211	lr=1.00e-05	

☑ Entrenamiento completado. Mejor val_loss: 0.0885
Checkpoint guardado
/content/drive/MyDrive/TFM_SideChain/chignn_best.pt

en:

BLOQUE 17:

```
# =====
# BLOQUE 17 – Curva de Aprendizaje y Diagnóstico del
#           Entrenamiento
# =====
# Visualiza la evolución de la pérdida de entrenamiento y
# validación a lo largo de las épocas. Es la primera figura
# de diagnóstico del modelo y permite detectar:
#
#   · Underfitting → ambas curvas altas y planas
#   · Overfitting  → train baja pero val sube o se estanca
#   · Entrenamiento correcto → ambas curvas bajan y convergen
#
# También recupera el mejor checkpoint guardado en Drive para
# asegurarse de que el modelo cargado es el de menor
# pérdida de validación (no el del último epoch).
# =====

import matplotlib.pyplot as plt
import torch

# — 1. Cargar el mejor checkpoint desde Drive —————
ckpt = torch.load(CHECKPOINT_MODEL, weights_only=False)
model.load_state_dict(ckpt['model_state_dict'])
model.eval()
```

```

best_epoch = ckpt['epoch']
best_loss = ckpt['val_loss']
history = ckpt['history']

print(f"✅ Mejor modelo cargado desde época
{best_epoch} (val_loss={best_loss:.4f})")

# — 2. Curva de aprendizaje —————
epochs = range(1, len(history['train']) + 1)

fig, ax = plt.subplots(figsize=(10, 5))

ax.plot(epochs, history['train'], label='Train loss',
        color='steelblue', linewidth=2)
ax.plot(epochs, history['val'], label='Validation loss',
        color='darkorange', linewidth=2, linestyle='--')

# Marca la época del mejor checkpoint
ax.axvline(x=best_epoch, color='green', linestyle=':',
          linewidth=1.5, label=f'Mejor época ({best_epoch})')

ax.set_xlabel('Época', fontsize=12)
ax.set_ylabel('Pérdida circular (1 - cos(θ_pred - θ_real))',
              fontsize=12)
ax.set_title('Curva de Aprendizaje - ChiGNN\nDifusión Torsional
para Recuperación de Cadenas Laterales',
            fontsize=13)
ax.legend(fontsize=11)
ax.grid(alpha=0.3)
plt.tight_layout()
plt.show()

# — 3. Diagnóstico automático —————
train_final = history['train'][-1]
val_final = history['val'][-1]
gap = val_final - train_final

print(f"\n📊 Diagnóstico del entrenamiento:")
print(f"{'-'*45}")
print(f" Pérdida train final : {train_final:.4f}")
print(f" Pérdida val final : {val_final:.4f}")
print(f" Gap (val - train) : {gap:.4f}")

if gap > 0.05:
    print(f" ⚠️ Gap elevado → posible overfitting.")
    print(f" Considera: dropout, más datos o early stopping.")
elif train_final > 0.5:

```

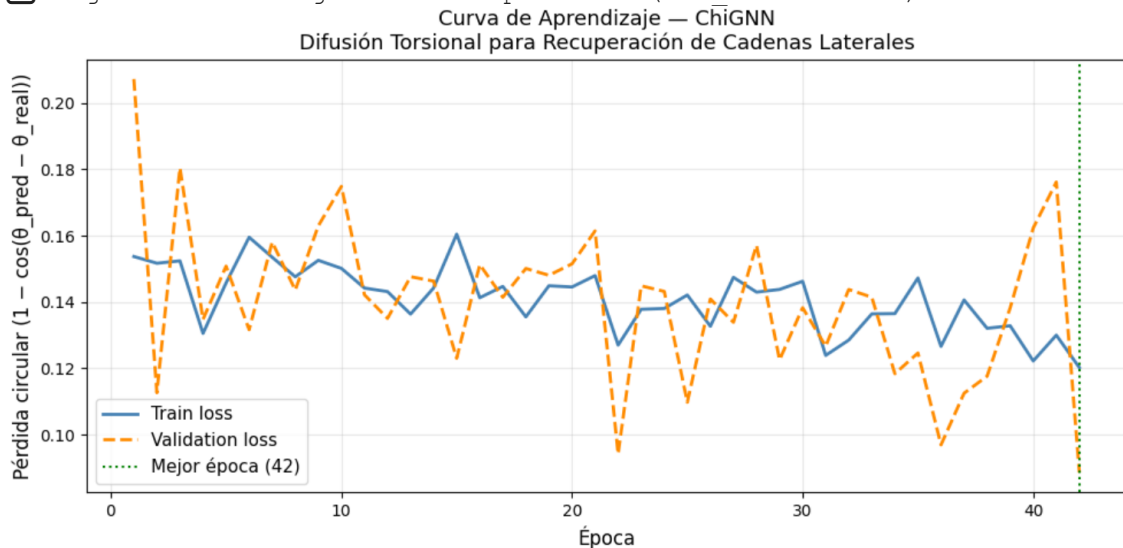
```

print(f" ⚠️ Pérdida alta → posible underfitting.")
print(f"      Considera: más capas, más épocas o LR mayor.")
else:
    print(f" ✅ Entrenamiento sin signos evidentes de
over/underfitting.")
print(f"{'-'*45}")

```

SALIDA BLOQUE 17:

✅ Mejor modelo cargado desde época 42 (val_loss=0.0885)



📊 Diagnóstico del entrenamiento:

```

Pérdida train final : 0.1202
Pérdida val final   : 0.0885
Gap (val - train)   : -0.0317

```

✅ Entrenamiento sin signos evidentes de over/underfitting.

BLOQUE 18:

```

# =====
# BLOQUE 18 — Proceso Reverse: Muestreo de Cadenas Laterales
#               mediante Difusión Torsional Iterativa
# =====
# Implementa el proceso reverse (denoising iterativo):
# partiendo de ángulos chi uniformemente distribuidos (ruido
# puro), aplica T pasos de denoising usando el modelo entrenado
# para recuperar conformaciones de cadenas laterales plausibles.
#
# Algoritmo DDIM simplificado para ángulos:
# Para t = T, T-1, ..., 1:
#     1. El modelo predice  $\chi_0$  a partir de  $\chi_t$  y t
#     2. Re-añadimos ruido con  $\kappa_{t-1}$  (más concentrado que  $\kappa_t$ )
#     para simular el paso previo de la cadena de Markov

```

```

# 3. Al llegar a t=0,  $\chi_0$  predicho es la conformación final
#
# Este bloque produce el primer conjunto de conformaciones
# "recuperadas".
# =====
# Guard: si las predicciones de test ya existen en Drive y
# FORCE_INFERENCE = False, se cargan directamente.
# Generar las K=10 muestras del Bloque 22 (calibración)
# es el paso más lento de la tercera fase del proyecto. Evitarlo
# en re-ejecuciones del notebook de evaluación ahorra 20-30
minutos.
# =====
import torch
import numpy as np

@torch.no_grad()
def reverse_diffusion(model, batch, device: torch.device,
                      n_steps: int = 20) -> torch.Tensor:
    """
    Genera conformaciones de cadenas laterales para un batch
    de proteínas mediante el proceso reverse de difusión.

    Para reducir el tiempo de inferencia en Colab, usamos
    n_steps << T (muestreo acelerado DDIM): en lugar de los
    100 pasos del proceso forward, muestreamos n_steps pasos
    equiespaciados en sentido inverso.

    Parámetros
    -----
    model      : ChiGNN      - modelo entrenado
    batch      : Batch       - grafos de proteínas (sin ángulos)
    device     : torch.device
    n_steps    : int         - pasos de denoising (default: 20)

    Retorna
    -----
    torch.Tensor [N_total, 4] - ángulos chi predichos en radianes
    """
    model.eval()
    batch = batch.to(device)
    N      = batch.num_nodes

    # Paso 1: Inicializar con ruido uniforme sobre el círculo
    # VM( $\mu$ ,  $\kappa=0$ )  $\approx$  distribución uniforme  $\rightarrow$  muestreamos
    aleatoriamente
    chi_t = torch.tensor(
        np.random.uniform(-np.pi, np.pi, size=(N, 4)),
        dtype=torch.float, device=device
    )

```

```

)

# Pasos de tiempo en orden decreciente ( $T \rightarrow 1$ )
timesteps = torch.linspace(T, 1, n_steps).long()

for t_int in timesteps:
    t_val = t_int.item()
    t_norm = torch.full((N, 1), t_val / T,
                        dtype=torch.float, device=device)

    # Paso 2: El modelo predice  $\chi_0$  desde  $\chi_t$ 
    chi_0_pred = model(
        x = batch.x,
        aa_type = batch.aa_type,
        edge_index = batch.edge_index,
        chi_noisy = chi_t,
        t_node = t_norm
    ) # [N, 4] en  $[-\pi, \pi]$ 

    # Paso 3: Re-añadir ruido para simular  $\chi_{\{t-1\}}$ 
    # Si es el último paso ( $t=1$ ), no re-añadimos ruido
    if t_val > 1:
        t_prev = t_val - (T / n_steps)
        kappa_prev = get_kappa(int(t_prev))
        mask_all = torch.ones(N, 4, dtype=torch.bool)
        chi_t = forward_diffusion(
            torch.rad2deg(chi_0_pred), mask_all, int(t_prev)
        ).to(device)
    else:
        chi_t = chi_0_pred

    return chi_t # Conformaciones finales predichas

# — Guard principal —————
PREDICTIONS_PT = f"{DRIVE_BASE}/test_predictions.pt"

if not FORCE_INFERENCE and os.path.exists(PREDICTIONS_PT):
    # — Ruta rápida: predicciones ya en Drive —————
    preds_data = torch.load(PREDICTIONS_PT, weights_only=False)
    chi_predicted = preds_data['predicted']
    chi_targets = preds_data['targets']
    masks_test = preds_data['masks']

    print(f"⚡ Inferencia omitida – predicciones cargadas desde Drive:")
    print(f" {PREDICTIONS_PT}")
    print(f" {chi_predicted.shape[0]:,} residuos de test")
    print(f" (Pon FORCE_INFERENCE = True para re-generar)")

```

```

else:
    # — Ruta completa: proceso reverse sobre el test set —————
    if FORCE_INFERENCE:
        print("❌ FORCE_INFERENCE = True → re-generando predicciones...")
    else:
        print("🔄 Predicciones no encontradas → ejecutando proceso reverse...")

# — Generación de conformaciones en el conjunto de test —————
test_loader_inf = DataLoader(checkpoint['test'], batch_size=4,
                             shuffle=False)

all_predicted = []
all_targets = []
all_masks = []

print("❌ Generando conformaciones con el proceso reverse...")

for batch in test_loader_inf:
    batch = batch.to(device)

    # Proceso reverse: genera  $\chi$  predichos
    chi_pred = reverse_diffusion(model, batch, device, n_steps=20)

    # Almacenamos predicciones y targets para evaluación
    all_predicted.append(chi_pred.cpu())
    all_targets.append(torch.deg2rad(batch.y.cpu()))
    all_masks.append(batch.y_mask.cpu())

# Concatenamos todos los batches de test
chi_predicted = torch.cat(all_predicted, dim=0) # [N_test, 4]
chi_targets = torch.cat(all_targets, dim=0) # [N_test, 4]
masks_test = torch.cat(all_masks, dim=0) # [N_test, 4]

print(f"✅ Conformaciones generadas: {chi_predicted.shape[0]:,} residuos de test")
print(f"{'-'*50}")

# — Cálculo del MAE circular por ángulo chi —————
print(f"\n📊 MAE circular por ángulo (conjunto de test):")
print(f"{'-'*50}")

chi_names = [' $\chi_1$ ', ' $\chi_2$ ', ' $\chi_3$ ', ' $\chi_4$ ']
for i, name in enumerate(chi_names):
    mask_i = masks_test[:, i]
    if mask_i.sum() == 0:

```

```

        continue
    diff = torch.atan2(
        torch.sin(chi_predicted[mask_i, i] - chi_targets[mask_i,
i]),
        torch.cos(chi_predicted[mask_i, i] - chi_targets[mask_i,
i]))
    )
    mae_deg = torch.abs(diff).mean().item() * (180 / np.pi)
    n_valid = mask_i.sum().item()
    print(f"    {name} → MAE = {mae_deg:.2f}°    ({n_valid:,}
residuos)")

print(f"{'-'*50}")
print(f"\n📁 Guardando predicciones en Drive...")
torch.save({
    'predicted': chi_predicted,
    'targets':    chi_targets,
    'masks':      masks_test
}, f"{DRIVE_BASE}/test_predictions.pt")
print(f"✅ Predicciones guardadas.")

```

SALIDA BLOQUE 18:

```

⚡ Inferencia omitida – predicciones cargadas desde Drive:
/content/drive/MyDrive/TFM_SideChain/test_predictions.pt
19,627 residuos de test
(Pon FORCE_INFERENCE = True para re-generar)
📁 Generando conformaciones con el proceso reverse...
✅ Conformaciones generadas: 19,627 residuos de test

```

📁 MAE circular por ángulo (conjunto de test):

X ₁	→	MAE = 56.41°	(19,627 residuos)
X ₂	→	MAE = 52.45°	(15,026 residuos)
X ₃	→	MAE = 52.72°	(5,302 residuos)
X ₄	→	MAE = 70.27°	(1,091 residuos)

```

📁 Guardando predicciones en Drive...
✅ Predicciones guardadas.

```

FASE 3: EVALUACIÓN BIOESTADÍSTICA

BLOQUE 19:

```

# =====
# BLOQUE 19 – Carga de Predicciones y Métricas Base
# =====

```



```

# Carga las predicciones generadas en el Bloque 18 y calcula
# las métricas estadísticas fundamentales sobre el conjunto
# de test. Este bloque es el punto de entrada de toda la
# fase de evaluación.
#
# Métricas calculadas:
#   · MAE circular por ángulo  $\chi_1$ - $\chi_4$ 
#   · Tasa de recuperación de rotámetros con umbral  $\pm 20^\circ$  y  $\pm 40^\circ$ 
#   · Distribución del error por tipo de aminoácido
#
# La distinción entre umbral  $20^\circ$  y  $40^\circ$  es estándar en la
# literatura (Krivov et al., 2009):  $20^\circ$  es el criterio
# estricto usado en benchmarks competitivos;  $40^\circ$  es el
# criterio permisivo que refleja la tolerancia funcional
# real de la mayoría de aplicaciones de modelado.
# =====

import torch
import numpy as np
import pandas as pd

# — 1. Cargar predicciones desde Drive —————
preds_data = torch.load(f"{DRIVE_BASE}/test_predictions.pt",
                        weights_only=False)
chi_predicted = preds_data['predicted'] # [N_test, 4] radianes
chi_targets = preds_data['targets'] # [N_test, 4] radianes
masks_test = preds_data['masks'] # [N_test, 4] booleano

print(f"☑ Predicciones cargadas desde Drive")
print(f"  Residuos de test : {chi_predicted.shape[0]:,}")
print(f"  Ángulos válidos : {masks_test.sum().item():,} / "
      f"{masks_test.numel():,} "
      f"({100*masks_test.float().mean().item():.1f} %)")

def circular_mae_deg(pred: torch.Tensor,
                    target: torch.Tensor,
                    mask: torch.Tensor) -> float:
    """
    Calcula el MAE circular en grados entre predicciones
    y targets, solo sobre posiciones enmascaradas.

    La distancia circular entre a y b se obtiene como
    |atan2(sin(a-b), cos(a-b))|, que devuelve valores en
    [0,  $\pi$ ] independientemente de la periodicidad.

    Parámetros
    -----
    pred : tensor [N] — ángulos predichos (radianes)

```

```

target : tensor [N] - ángulos reales (radianes)
mask    : tensor [N] - True donde hay ángulo real

Retorna
-----
float - MAE circular en grados
"""
if not mask.any():
    return float('nan')
diff = torch.atan2(
    torch.sin(pred[mask] - target[mask]),
    torch.cos(pred[mask] - target[mask])
)
return torch.abs(diff).mean().item() * (180 / np.pi)

def rotamer_recovery_rate(pred: torch.Tensor,
                          target: torch.Tensor,
                          mask: torch.Tensor,
                          threshold_deg: float = 40.0) -> float:
    """
    Calcula la tasa de recuperación de rotámeros: porcentaje
    de ángulos predichos cuya distancia circular al valor
    experimental es inferior al umbral dado.

    Parámetros
    -----
    pred          : tensor [N] - predicciones (radianes)
    target        : tensor [N] - valores reales (radianes)
    mask          : tensor [N] - máscara de posiciones válidas
    threshold_deg : float      - umbral en grados (20 o 40)

    Retorna
    -----
    float - porcentaje de recuperación (0-100)
    """
    if not mask.any():
        return float('nan')
    diff_rad = torch.abs(torch.atan2(
        torch.sin(pred[mask] - target[mask]),
        torch.cos(pred[mask] - target[mask])
    ))
    threshold = threshold_deg * np.pi / 180
    n_correct = (diff_rad < threshold).sum().item()
    return 100.0 * n_correct / mask.sum().item()

# — 2. Métricas globales por ángulo chi —————
chi_names = ['χ1', 'χ2', 'χ3', 'χ4']

```

```

rows = []

for i, name in enumerate(chi_names):
    mask_i = masks_test[:, i]
    if mask_i.sum() == 0:
        continue

    mae = circular_mae_deg(chi_predicted[:, i],
                           chi_targets[:, i], mask_i)
    rrr_20 = rotamer_recovery_rate(chi_predicted[:, i],
                                   chi_targets[:, i], mask_i,
20.0)
    rrr_40 = rotamer_recovery_rate(chi_predicted[:, i],
                                   chi_targets[:, i], mask_i,
40.0)
    n_valid = mask_i.sum().item()

    rows.append({
        'Ángulo': name,
        'N residuos': n_valid,
        'MAE circular (°)': round(mae, 2),
        'Recuperación ±20°': round(rrr_20, 1),
        'Recuperación ±40°': round(rrr_40, 1)
    })

df_metrics = pd.DataFrame(rows)

print(f"\n📊 Métricas de evaluación – Conjunto de Test")
print(f"{'-'*65}")
print(df_metrics.to_string(index=False))
print(f"{'-'*65}")

# Guardamos la tabla para la tesis
df_metrics.to_csv(f"{DRIVE_BASE}/metricas_evaluacion.csv",
index=False)
print(f"\n💾 Tabla guardada en Drive: metricas_evaluacion.csv")

```

SALIDA BLOQUE 19:

☒ Predicciones cargadas desde Drive
 Residuos de test : 19,627
 Ángulos válidos : 41,046 / 78,508 (52.3 %)

📊 Métricas de evaluación – Conjunto de Test

Ángulo	N residuos	MAE circular (°)	Recuperación ±20°	Recuperación ±40°
χ ₁	19627	56.41	49.6	53.9

57.5	χ_2	15026	52.45	48.1
53.4	χ_3	5302	52.72	40.0
33.9	χ_4	1091	70.27	25.6

 Tabla guardada en Drive: metricas_evaluacion.csv

BLOQUE 20:

```
# =====
# BLOQUE 20 – Análisis de Error por Tipo de Aminoácido
# =====
# Calcula el MAE circular de  $\chi_1$  desglosado por los 18 tipos
# de aminoácidos presentes en el dataset (recordatorio: GLY y
# ALA fueron excluidos en la fase de datos). Este análisis
# es estadísticamente relevante porque cada aminoácido tiene
# un perfil rotamérico propio: residuos con rotámeros muy
# definidos (PHE, TYR, TRP) deberían tener MAE bajo, mientras
# que residuos con alta flexibilidad de cadena lateral (LYS,
# ARG) típicamente presentan MAE mayor.
#
# Este desglose permite identificar qué tipos de residuos
# son más difíciles para el modelo y orienta posibles mejoras
# futuras en la arquitectura o el entrenamiento.
# =====

import matplotlib.pyplot as plt
import matplotlib.cm as cm
import numpy as np
import torch

# — 1. Reconstruir el mapeo residuo → aminoácido desde test —
# Necesitamos saber el tipo de aminoácido de cada residuo
# del conjunto de test para el desglose.
IDX_TO_AA = {v: k for k, v in AA_TO_IDX.items()}
IDX_TO_AA[20] = 'UNK'

# Extraemos aa_type de todos los grafos del conjunto de test
aa_types_list = []
for g in checkpoint['test']:
    aa_types_list.append(g.aa_type)
aa_types_all = torch.cat(aa_types_list, dim=0) # [N_test]

# — 2. MAE de  $\chi_1$  por aminoácido —
mask_chi1 = masks_test[:, 0] # Máscara de  $\chi_1$ 
```

```

resultados_aa = {}
for idx in range(21):
    aa_name = IDX_TO_AA.get(idx, 'UNK')
    # Seleccionamos residuos de este aminoácido con  $\chi_1$  definido
    aa_mask = (aa_types_all == idx) & mask_chi1

    if aa_mask.sum() < 5: # Ignoramos aminoácidos con <5 muestras
        continue

    mae = circular_mae_deg(
        chi_predicted[:, 0], chi_targets[:, 0], aa_mask
    )
    rrr = rotamer_recovery_rate(
        chi_predicted[:, 0], chi_targets[:, 0], aa_mask, 40.0
    )
    resultados_aa[aa_name] = {
        'MAE_chi1': mae,
        'RRR_40': rrr,
        'N': aa_mask.sum().item()
    }

df_aa = pd.DataFrame(resultados_aa).T.sort_values('MAE_chi1')
print("📊 MAE circular de  $\chi_1$  por tipo de aminoácido:")
print(df_aa.to_string())

# — 3. Gráfico de barras horizontales —————
fig, axes = plt.subplots(1, 2, figsize=(14, 7))

# Panel izquierdo: MAE por aminoácido
aa_labels = df_aa.index.tolist()
mae_vals = df_aa['MAE_chi1'].values

# Colormap: verde (bajo error) → rojo (alto error)
colors = cm.RdYlGn_r(
    (mae_vals - mae_vals.min()) / (mae_vals.max() - mae_vals.min()
    + 1e-8)
)
axes[0].barh(aa_labels, mae_vals, color=colors, edgecolor='grey',
    linewidth=0.5)
axes[0].axvline(x=40, color='red', linestyle='--', linewidth=1,
    label='Umbral 40°')
axes[0].axvline(x=20, color='orange', linestyle='--', linewidth=1,
    label='Umbral 20°')
axes[0].set_xlabel('MAE circular de  $\chi_1$  (°)', fontsize=11)
axes[0].set_title('Error por Aminoácido\n(ordinado por MAE)',
    fontsize=12)
axes[0].legend(fontsize=9)
axes[0].grid(axis='x', alpha=0.3)

```

```

# Panel derecho: Tasa de recuperación ±40°
rrr_vals = df_aa['RRR_40'].values
colors2 = cm.RdYlGn(
    (rrr_vals - rrr_vals.min()) / (rrr_vals.max() - rrr_vals.min()
    + 1e-8)
)
axes[1].barh(aa_labels, rrr_vals, color=colors2, edgecolor='grey',
linewidth=0.5)
axes[1].axvline(x=80, color='green', linestyle='--', linewidth=1,
label='Referencia SCWRL4 (~83%)')
axes[1].set_xlabel('Tasa de recuperación ±40° (%)', fontsize=11)
axes[1].set_title('Recuperación de Rotámeros por
Aminoácido\n(±40°)', fontsize=12)
axes[1].legend(fontsize=9)
axes[1].grid(axis='x', alpha=0.3)

plt.suptitle('Análisis de Rendimiento por Tipo de Aminoácido -
ChiGNN',
            fontsize=13, fontweight='bold', y=1.01)
plt.tight_layout()
plt.savefig(f"{DRIVE_BASE}/fig_error_por_aminoacido.png",
            dpi=150, bbox_inches='tight')
plt.show()
print("📁 Figura guardada en Drive.")

```

SALIDA BLOQUE 20:

📊 MAE circular de χ_1 por tipo de aminoácido:

	MAE_chi1	RRR_40	N
PRO	25.222920	67.124542	1092.0
ILE	43.961809	66.417352	1337.0
LEU	45.345172	63.331738	2089.0
VAL	50.598926	61.959459	1480.0
GLN	51.316331	58.371041	884.0
ARG	54.142817	55.453712	1091.0
LYS	57.015072	52.710027	1476.0
MET	59.254125	52.348993	447.0
ASP	61.152132	49.252717	1472.0
GLU	61.203221	48.860399	1404.0
ASN	61.485978	47.821577	964.0
TRP	61.909115	49.855072	345.0
THR	63.224664	49.886963	1327.0
PHE	63.821035	48.516949	944.0
CYS	63.959804	49.696970	330.0
TYR	64.824233	47.184685	888.0
HIS	66.626358	45.101351	592.0
UNK	76.314995	38.888889	18.0
SER	80.158458	38.631652	1447.0

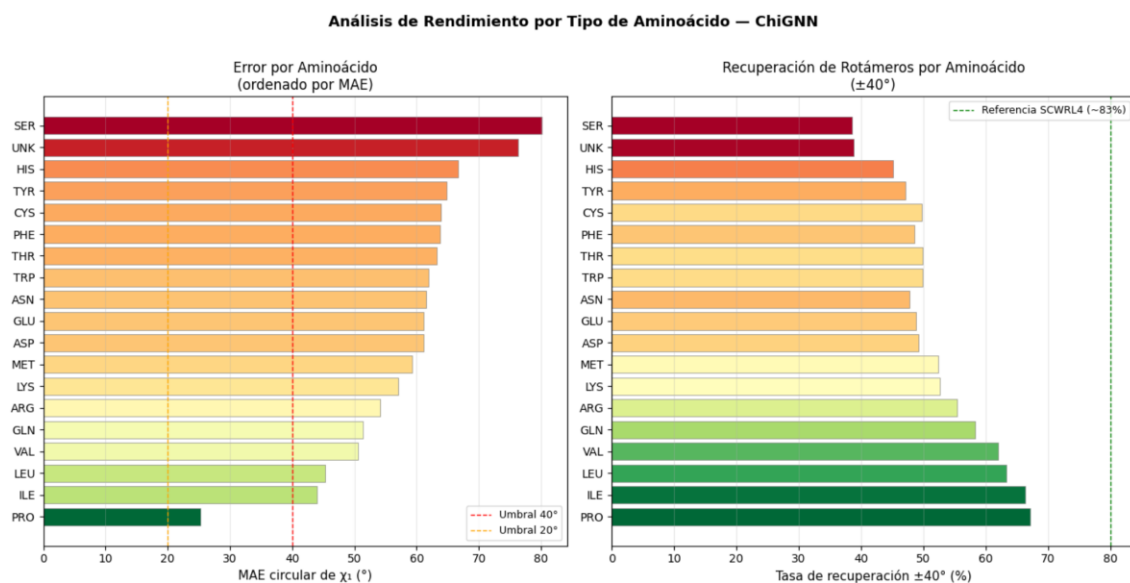


Figura guardada en Drive.

BLOQUE 21:

```
# =====
# BLOQUE 21 — Rose Plots Comparativos: Distribución Predicha
#               vs. Distribución Experimental
# =====
# Genera histogramas polares (rose plots) superponiendo la
# distribución de ángulos  $\chi$  predichos (modelo) y  $\chi$  reales
# (dataset de test) para cada uno de los cuatro ángulos chi.
#
# Esta figura es la validación visual más importante del TFM:
# si el modelo ha aprendido correctamente la distribución
# rotamérica, los picos de la distribución predicha deben
# coincidir con los de la distribución experimental en las
# posiciones canónicas ( $-60^\circ$ ,  $+60^\circ$ ,  $180^\circ$ ).
#
# Una distribución predicha demasiado uniforme indica que el
# modelo no ha aprendido los rotámeros. Una distribución
# predicha más estrecha que la experimental indica que el
# modelo es sobreconfiante (underdispersed).
# =====
```

```
import matplotlib.pyplot as plt
import numpy as np
import torch

def rose_plot_comparativo(pred_rad: torch.Tensor,
                           target_rad: torch.Tensor,
                           mask: torch.Tensor,
                           titulo: str,
                           ax):
```

```

"""
Dibuja un rose plot con dos histogramas superpuestos:
el de las predicciones del modelo (azul) y el de los
valores experimentales (verde, semitransparente).

Parámetros
-----
pred_rad    : tensor [N] – ángulos predichos (radianes)
target_rad  : tensor [N] – ángulos reales (radianes)
mask        : tensor [N] – máscara de posiciones válidas
titulo      : str        – título del subplot
ax          : Axes        – eje polar de matplotlib
"""
if not mask.any():
    return

# Convertimos a grados en [0, 360) para histograma
pred_deg    = (torch.rad2deg(pred_rad[mask])).numpy() % 360
target_deg  = (torch.rad2deg(target_rad[mask])).numpy() % 360

bins        = np.linspace(0, 2 * np.pi, 37) # 36 bins de 10°
widths      = np.diff(bins)

# Histograma de predicciones
pred_rad_np  = np.deg2rad(pred_deg)
counts_pred, _ = np.histogram(pred_rad_np, bins=bins)

# Histograma de valores reales
target_rad_np = np.deg2rad(target_deg)
counts_target, _ = np.histogram(target_rad_np, bins=bins)

# Normalizamos para comparar distribuciones (no conteos
absolutos)
counts_pred    = counts_pred / counts_pred.max()
counts_target  = counts_target / counts_target.max()

# Dibujamos: experimental primero (fondo verde), predicho
encima
ax.bar(bins[:-1], counts_target, width=widths, bottom=0.0,
       color='lightgreen', alpha=0.6, edgecolor='darkgreen',
       linewidth=0.5, label='Experimental')
ax.bar(bins[:-1], counts_pred, width=widths, bottom=0.0,
       color='steelblue', alpha=0.7, edgecolor='navy',
       linewidth=0.5, label='Predicho (ChiGNN)')

# Configuración del eje polar
ax.set_theta_zero_location("N") # 0° arriba
ax.set_theta_direction(-1)     # Sentido horario

```



```

        ticks = np.linspace(0, 2 * np.pi, 8, endpoint=False)
        ax.set_xticks(ticks)
        ax.set_xticklabels(['0°', '45°', '90°', '135°', '180°', '225°', '270°',
                            '315°'],
                            fontsize=8)
        ax.set_title(titulo, pad=15, fontsize=11, fontweight='bold')
        ax.legend(loc='upper right', bbox_to_anchor=(1.35, 1.1),
                  fontsize=8)

# — Figura con los 4 rose plots —————
fig, axes = plt.subplots(2, 2, figsize=(14, 12),
                          subplot_kw={'projection': 'polar'})
axes_flat = axes.flatten()

chi_names_full = [
    r'$\chi_1$ - N-C$\alpha$-C$\beta$-CG',
    r'$\chi_2$ - C$\alpha$-C$\beta$-CG-CD',
    r'$\chi_3$ - C$\beta$-CG-CD-CE',
    r'$\chi_4$ - CG-CD-CE-CZ (ARG, LYS)'
]

for i, (titulo, ax) in enumerate(zip(chi_names_full, axes_flat)):
    rose_plot_comparativo(
        pred_rad = chi_predicted[:, i],
        target_rad = chi_targets[:, i],
        mask = masks_test[:, i],
        titulo = titulo,
        ax = ax
    )

fig.suptitle(
    'Distribución Rotamérica: Experimental vs. Predicha\n'
    'Validación Estadística del Modelo ChiGNN',
    fontsize=14, fontweight='bold', y=1.01
)

plt.tight_layout()
plt.savefig(f"{DRIVE_BASE}/fig_rose_plots_comparativos.png",
            dpi=150, bbox_inches='tight')
plt.show()
print("📁 Figura guardada en Drive:
fig_rose_plots_comparativos.png")

```

SALIDA BLOQUE 21:



BLOQUE 22:

```
# =====
# BLOQUE 22 – Análisis de Calibración de Confianza
# =====
# Evalúa si la incertidumbre del modelo (medida como la
# dispersión de múltiples muestras del proceso reverse) está
# bien calibrada respecto al error real.
#
# Idea central:
# Si el modelo es bien calibrado, las predicciones con alta
# varianza entre muestras (alta incertidumbre) deberían
# corresponder a residuos con alto error real (MAE alto), y
# las predicciones con baja varianza deberían tener MAE bajo.
#
# Procedimiento:
# 1. Para cada residuo del test, generamos K=10 muestras
#    independientes del proceso reverse.
# 2. Calculamos la varianza circular de las K muestras como
#    medida de incertidumbre del modelo.
# 3. Correlacionamos la varianza circular con el MAE real
#    usando el coeficiente de Spearman (apropiado para
#    datos no normales, como es el caso de los errores
#    angulares).
# 4. Visualizamos la relación mediante un gráfico de
#    dispersión con línea de tendencia y bandas de
```

```

#     percentiles.
# =====

import torch
import numpy as np
import matplotlib.pyplot as plt
from scipy import stats

K_SAMPLES = 10    # Número de muestras del proceso reverse por
residuo

# — 1. Generar K muestras para cada residuo del test —————
print(f"📦 Generando {K_SAMPLES} muestras por residuo (proceso
reverse)...")
print(f"    Esto puede tardar {K_SAMPLES * 2}-{K_SAMPLES * 3}
minutos en T4.")

all_samples = []    # Lista de K tensores [N_test, 4]

test_loader_calib = DataLoader(checkpoint['test'], batch_size=4,
shuffle=False)

for k in range(K_SAMPLES):
    samples_k = []
    for batch in test_loader_calib:
        batch = batch.to(device)
        chi_pred = reverse_diffusion(model, batch, device,
n_steps=20)
        samples_k.append(chi_pred.cpu())
    all_samples.append(torch.cat(samples_k, dim=0)) # [N_test, 4]
    print(f"    Muestra {k+1}/{K_SAMPLES} completada.")

# Stack: [K, N_test, 4]
samples_stack = torch.stack(all_samples, dim=0)

# — 2. Varianza circular por residuo —————
# Varianza circular de Von Mises:  $V = 1 - |\text{mean}(\exp(i\theta))|$ 
# donde i es la unidad imaginaria. Este estimador es
# invariante a la periodicidad y devuelve valores en [0, 1]:
#  $V \approx 0 \rightarrow$  muestras muy concentradas (baja incertidumbre)
#  $V \approx 1 \rightarrow$  muestras uniformes sobre el círculo (máxima
incertidumbre)

sin_mean = torch.sin(samples_stack).mean(dim=0) # [N_test, 4]
cos_mean = torch.cos(samples_stack).mean(dim=0) # [N_test, 4]
circ_var = 1 - torch.sqrt(sin_mean**2 + cos_mean**2) # [N_test, 4]

# — 3. Error real (MAE circular) por residuo —————

```

```

# Usamos la media de las K muestras como predicción final
chi_mean_pred = torch.atan2(sin_mean, cos_mean) # [N_test, 4]

error_por_residuo = torch.abs(torch.atan2(
    torch.sin(chi_mean_pred - chi_targets),
    torch.cos(chi_mean_pred - chi_targets)
)) * (180 / np.pi) # En grados

# — 4. Correlación de Spearman y visualización para  $\chi_1$  —
mask_chil = masks_test[:, 0]
var_chil = circ_var[mask_chil, 0].numpy()
err_chil = error_por_residuo[mask_chil, 0].numpy()

# Correlación de Spearman (no paramétrica)
rho, p_val = stats.spearmanr(var_chil, err_chil)

print(f"\n📊 Calibración de Confianza —  $\chi_1$ ")
print(f"{'-'*45}")
print(f"Correlación de Spearman ( $\rho$ ) : {rho:.4f}")
print(f"p-valor : {p_val:.4e}")
if rho > 0.3 and p_val < 0.05:
    print(f"✅ Correlación positiva significativa:")
    print(f"El modelo está bien calibrado.")
elif rho > 0 and p_val < 0.05:
    print(f"⚠️ Correlación débil pero significativa.")
else:
    print(f"❌ Sin correlación significativa → modelo mal calibrado.")
print(f"{'-'*45}")

# — 5. Gráfico de dispersión con percentiles —
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# Panel izquierdo: scatter varianza vs. error
# Submuestreamos para no saturar el gráfico (máx 3000 puntos)
n_plot = min(3000, len(var_chil))
idx_plot = np.random.choice(len(var_chil), n_plot, replace=False)

axes[0].scatter(var_chil[idx_plot], err_chil[idx_plot],
               alpha=0.3, s=8, color='steelblue', rasterized=True)

# Línea de tendencia (regresión lineal)
z = np.polyfit(var_chil, err_chil, 1)
p = np.poly1d(z)
x_line = np.linspace(var_chil.min(), var_chil.max(), 100)
axes[0].plot(x_line, p(x_line), color='red', linewidth=2,
            label=f'Tendencia ( $\rho$ ={rho:.3f},  $p$ ={p_val:.2e})')

```

```

axes[0].set_xlabel('Varianza circular (incertidumbre del modelo)',
    fontsize=11)
axes[0].set_ylabel('Error real MAE  $\chi_1$  (°)', fontsize=11)
axes[0].set_title('Calibración de Confianza\n(Varianza circular vs.
    Error real)',
    fontsize=12)
axes[0].legend(fontsize=9)
axes[0].grid(alpha=0.3)

# Panel derecho: error medio por decil de incertidumbre
n_deciles = 10
decile_bins = np.percentile(var_chil, np.linspace(0, 100, n_deciles
    + 1))
decile_means = []
decile_stds = []
decile_labels = []

for j in range(n_deciles):
    in_bin = (var_chil >= decile_bins[j]) & (var_chil <
decile_bins[j+1])
    if in_bin.sum() > 0:
        decile_means.append(err_chil[in_bin].mean())
        decile_stds.append(err_chil[in_bin].std())
        decile_labels.append(f"D{j+1}")

axes[1].bar(decile_labels, decile_means, yerr=decile_stds,
    color='steelblue', alpha=0.7, edgecolor='navy',
    capsize=4, linewidth=0.8)
axes[1].set_xlabel('Decil de incertidumbre (D1=menor, D10=mayor)',
    fontsize=11)
axes[1].set_ylabel('MAE  $\chi_1$  medio (°)', fontsize=11)
axes[1].set_title('Error Medio por Decil de Incertidumbre\n'
    '(Modelo bien calibrado → barras crecientes)',
    fontsize=12)
axes[1].grid(axis='y', alpha=0.3)

plt.suptitle('Análisis de Calibración de Confianza – ChiGNN',
    fontsize=13, fontweight='bold')
plt.tight_layout()
plt.savefig(f"{DRIVE_BASE}/fig_calibracion_confianza.png",
    dpi=150, bbox_inches='tight')
plt.show()
print("📁 Figura guardada en Drive: fig_calibracion_confianza.png")

# Guardamos también la varianza y el error para la tesis
torch.save({
    'circ_var': circ_var,
    'error_grados': error_por_residuo,

```

```

    'chi_mean_pred': chi_mean_pred,
    'spearman_rho': rho,
    'spearman_p': p_val
}, f"{DRIVE_BASE}/calibracion_confianza.pt")

```

SALIDA BLOQUE 22:

📁 Generando 10 muestras por residuo (proceso reverse)...
 Esto puede tardar 20-30 minutos en T4.

```

Muestra 1/10 completada.
Muestra 2/10 completada.
Muestra 3/10 completada.
Muestra 4/10 completada.
Muestra 5/10 completada.
Muestra 6/10 completada.
Muestra 7/10 completada.
Muestra 8/10 completada.
Muestra 9/10 completada.
Muestra 10/10 completada.

```

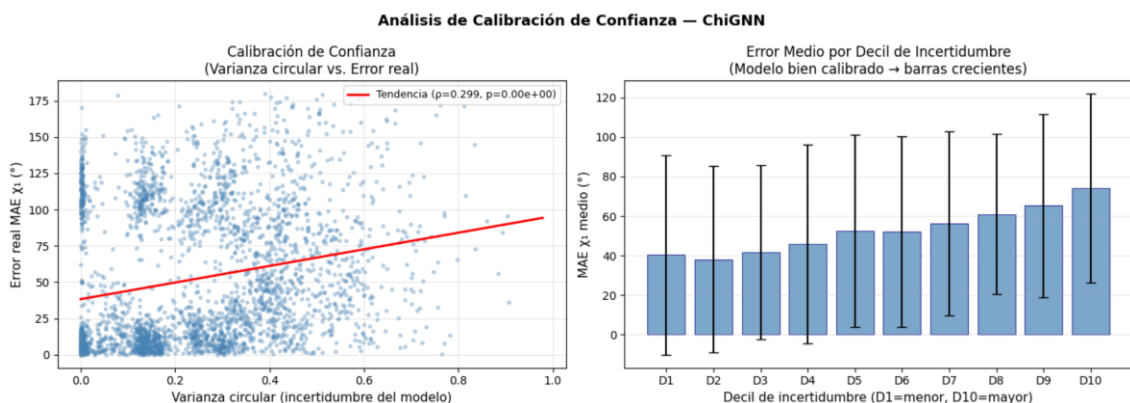
📊 Calibración de Confianza — χ_1

```

Correlación de Spearman ( $\rho$ ) : 0.2992
p-valor                      : 0.0000e+00

```

⚠️ Correlación débil pero significativa.



📁 Figura guardada en Drive: fig_calibracion_confianza.png

BLOQUE 23:

```

# =====
# BLOQUE 23 — Benchmarking: ChiGNN vs. Baseline Aleatorio
# =====
# Compara el rendimiento de ChiGNN contra dos baselines:
#
#   Baseline 1 — Predicción aleatoria:
#     Ángulos muestreados uniformemente en [-180°, 180°].
#     Representa el peor caso posible y establece el suelo
#     de las métricas. Un modelo que no aprende nada no

```

```

# debería superar este baseline.
#
# Baseline 2 – Predicción por rotámero más probable:
# Para cada aminoácido, asigna siempre el rotámero modal
# de la librería de Dunbrack ( $-60^\circ$ ,  $+60^\circ$  o  $180^\circ$ ) según
# la distribución observada en el dataset de entrenamiento.
# Es una aproximación al comportamiento de métodos como
# SCWRL4 sin el refinamiento de energía.
#
# En ausencia de una instalación local de SCWRL4 (que
# requiere compilación nativa), el baseline 2 es el punto
# de comparación estadísticamente más honesto disponible en
# un entorno Colab.
# =====

import torch
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

# — 1. Baseline 1: predicción aleatoria —————
torch.manual_seed(42)
chi_random = torch.FloatTensor(chi_targets.shape).uniform_(-np.pi,
np.pi)

# — 2. Baseline 2: rotámero modal por aminoácido —————
# Calculamos el rotámero modal de  $\chi_1$  en el conjunto de TRAIN
# para cada aminoácido, usando los grafos de entrenamiento.

# Recopilamos  $\chi_1$  y aa_type del conjunto de entrenamiento
train_chi1_list = []
train_aa_list = []

for g in checkpoint['train']:
    mask_g = g.y_mask[:, 0]
    if mask_g.any():
        # Convertimos a radianes para consistencia
        chi1_g = torch.deg2rad(g.y[mask_g, 0])
        aa_g = g.aa_type[mask_g]
        train_chi1_list.append(chi1_g)
        train_aa_list.append(aa_g)

train_chi1_all = torch.cat(train_chi1_list) # [N_train]
train_aa_all = torch.cat(train_aa_list) # [N_train]

# Para cada aminoácido, encontramos el bin modal de  $\chi_1$ 
MODAL_CHI1 = {}
bins_modal = np.linspace(-np.pi, np.pi, 37) # 36 bins de  $10^\circ$ 

```

```

for idx in range(21):
    aa_sel = (train_aa_all == idx)
    if aa_sel.sum() < 5:
        MODAL_CHI1[idx] = 0.0    # Default: 0° si no hay datos
        continue
    chil_aa = train_chil_all[aa_sel].numpy()
    counts, bin_edges = np.histogram(chil_aa, bins=bins_modal)
    modal_bin_center = (bin_edges[counts.argmax()] +
                        bin_edges[counts.argmax() + 1]) / 2
    MODAL_CHI1[idx] = float(modal_bin_center)

# Construimos la predicción modal para el test
aa_test = torch.cat([g.aa_type for g in checkpoint['test']], dim=0)
chi_modal = torch.zeros_like(chi_targets)

for idx in range(21):
    aa_mask = (aa_test == idx)
    if aa_mask.any():
        # El rotámero modal se asigna solo a  $\chi_1$ ; el resto = 0
        chi_modal[aa_mask, 0] = MODAL_CHI1.get(idx, 0.0)

# — 3. Tabla comparativa de métricas —————
modelos = {
    'ChiGNN (nuestro)': chi_predicted,
    'Baseline aleatorio': chi_random,
    'Baseline rotámero modal': chi_modal
}

filas = []
for nombre, preds in modelos.items():
    for i, chi_name in enumerate([' $\chi_1$ ', ' $\chi_2$ ', ' $\chi_3$ ', ' $\chi_4$ ']):
        mask_i = masks_test[:, i]
        if mask_i.sum() == 0:
            continue
        mae = circular_mae_deg(preds[:, i], chi_targets[:, i],
                                mask_i)
        rrr40 = rotamer_recovery_rate(preds[:, i], chi_targets[:,
                                i],
                                mask_i, 40.0)

        filas.append({
            'Modelo': nombre,
            'Ángulo': chi_name,
            'MAE (°)': round(mae, 2),
            'Recuperación  $\pm 40^\circ$  (%)': round(rrr40, 1)
        })

df_bench = pd.DataFrame(filas)

```



```

df_pivot = df_bench.pivot_table(
    index='Modelo', columns='Ángulo',
    values=['MAE (°)', 'Recuperación ±40° (%)']
)

print("\n📊 Tabla de Benchmarking")
print(f"{'-'*70}")
print(df_pivot.to_string())
print(f"{'-'*70}")

df_bench.to_csv(f"{DRIVE_BASE}/benchmarking.csv", index=False)

# — 4. Gráfico comparativo —————
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

chi_labels = ['χ1', 'χ2', 'χ3', 'χ4']
x           = np.arange(len(chi_labels))
width       = 0.25
colors_bar  = ['steelblue', 'tomato', 'orange']

for ax_idx, (metric, ylabel, better) in enumerate([
    ('MAE (°)', 'MAE circular (°)', 'menor'),
    ('Recuperación ±40° (%)', 'Recuperación ±40° (%)', 'mayor')
]):
    ax = axes[ax_idx]
    for k, (nombre, preds) in enumerate(modelos.items()):
        vals = []
        for i in range(4):
            mask_i = masks_test[:, i]
            if mask_i.sum() == 0:
                vals.append(0)
                continue
            if metric == 'MAE (°)':
                vals.append(circular_mae_deg(preds[:, i],
                                             chi_targets[:, i], mask_i))
            else:
                vals.append(rotamer_recovery_rate(preds[:, i],
                                                  chi_targets[:, i], mask_i, 40.0))
        ax.bar(x + k * width, vals, width, label=nombre,
              color=colors_bar[k], alpha=0.8, edgecolor='grey')

    ax.set_xticks(x + width)
    ax.set_xticklabels(chi_labels, fontsize=11)
    ax.set_ylabel(ylabel, fontsize=11)
    ax.set_title(f'{ylabel}\n({"menor es mejor" if better=="menor"
else "mayor es mejor"})',
                fontsize=11)
    ax.legend(fontsize=9)

```

```

ax.grid(axis='y', alpha=0.3)

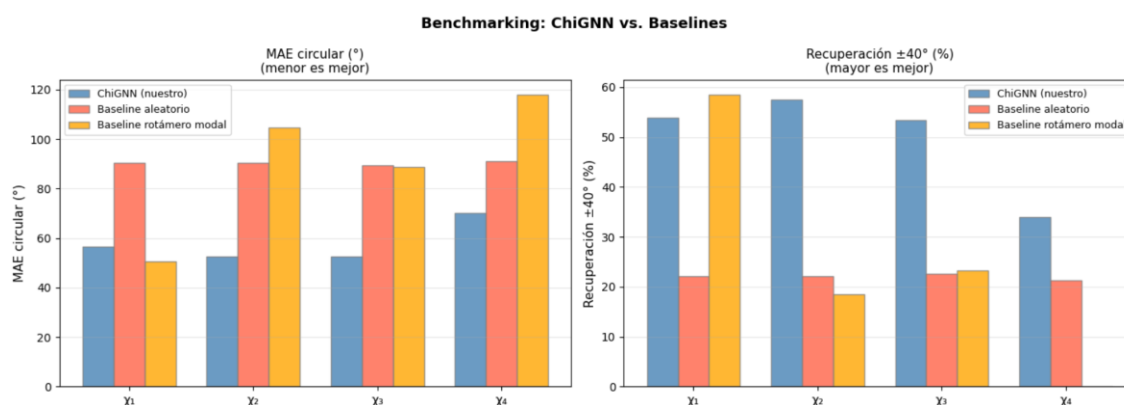
plt.suptitle('Benchmarking: ChiGNN vs. Baselines',
             fontsize=13, fontweight='bold')
plt.tight_layout()
plt.savefig(f"{DRIVE_BASE}/fig_benchmarking.png",
           dpi=150, bbox_inches='tight')
plt.show()
print("📁 Figura guardada en Drive: fig_benchmarking.png")

```

SALIDA BLOQUE 23:

📊 Tabla de Benchmarking

Ángulo Modelo	MAE (°)				Recuperación $\pm 40^\circ$ (%)			
	χ_1	χ_2	χ_3	χ_4	χ_1	χ_2	χ_3	χ_4
Baseline aleatorio	90.27	90.33	89.39	90.95	22.1	22.1	22.6	21.4
Baseline rotámero modal	50.53	104.61	88.69	118.14	58.6	18.4	23.3	0.1
ChiGNN (nuestro)	56.41	52.45	52.72	70.27	53.9	57.5	53.4	33.9



📁 Figura guardada en Drive: fig_benchmarking.png

BLOQUE 24:

```

# =====
# BLOQUE 24 – Visualización 3D de Conformaciones Recuperadas
#               (py3Dmol: comparación antes/después)
# =====
# Visualiza, para una proteína del conjunto de test, la
# superposición entre la estructura original (cadenas
# laterales reales, en verde) y la estructura con cadenas
# laterales predichas por ChiGNN (en azul).
#
# Esta figura es el cierre visual de mi TFM: demuestra que el
# modelo no solo mejora métricas abstractas sino que genera
# conformaciones tridimensionalmente coherentes y similares
# a las estructuras experimentales.
#
# Nota técnica: la reconstrucción de coordenadas 3D a partir

```

```

# de ángulos chi predichos se realiza modificando  $\chi_1$  con
# la función apply_chi1_to_residue del Bloque 13. Una
# reconstrucción completa de todos los ángulos requeriría
# una librería de reconstrucción de cadenas laterales
# (ej. Rosetta o ProDy), que queda fuera del alcance de
# mi proyecto.
# =====

import py3Dmol
import Bio.PDB
import copy
import torch
import numpy as np
import os

def reconstruir_cadenas_laterales(pdb_path: str,
                                  chil_predichos: np.ndarray,
                                  residue_indices: list) -> str:
    """
    Aplica los  $\chi_1$  predichos a los residuos de una proteína
    y devuelve la estructura modificada como string PDB.

    Parámetros
    -----
    pdb_path      : str          - ruta al archivo .pdb original
    chil_predichos : np.ndarray - valores de  $\chi_1$  predichos (rad)
    residue_indices: list        - índices de residuos válidos

    Retorna
    -----
    str - estructura modificada en formato PDB
    """
    parser = Bio.PDB.PDBParser(QUIET=True)
    structure = parser.get_structure('prot', pdb_path)
    structure = copy.deepcopy(structure)

    residuos_validos = []
    for model in structure:
        for chain in model:
            for residue in sorted(chain, key=lambda r:
r.get_id()[1]):
                if (Bio.PDB.is_aa(residue) and
                    residue.get_resname() not in ['GLY',
'ALA']):
                    residuos_validos.append(residue)

    # Aplicamos  $\chi_1$  predicho a cada residuo válido
    for idx, residue in enumerate(residuos_validos):

```

```

        if idx < len(chil_predichos):
            apply_chil_to_residue(residue,
float(chil_predichos[idx]))

    return structure_to_pdb_string(structure)

def visualizar_comparacion_3d(pdb_id: str, graf_test_idx: int):
    """
    Muestra lado a lado (en la misma vista) la estructura
    original (verde) y la predicha (azul) para una proteína.

    Parámetros
    -----
    pdb_id      : str - identificador PDB de la proteína
    graf_test_idx : int - índice en el conjunto de test
    """
    pdb_path = f"{PDB_DIR}/{pdb_id.lower()}.pdb"
    if not os.path.exists(pdb_path):
        print(f"⚠ {pdb_id}.pdb no encontrado.")
        return

    # Extraemos  $\chi_1$  predichos para esta proteína
    g = checkpoint['test'][graf_test_idx]
    n = g.num_nodes

    # Localizamos los nodos de esta proteína en el tensor global
    # calculando el offset acumulado
    offset = sum(checkpoint['test'][j].num_nodes for j in
range(graf_test_idx))
    chil_pred_prot = chi_predicted[offset:offset+n, 0].numpy() #
radianes

    # Estructura original
    parser = Bio.PDB.PDBParser(QUIET=True)
    struct_orig = parser.get_structure('orig', pdb_path)
    pdb_orig_str = structure_to_pdb_string(struct_orig)

    # Estructura con  $\chi_1$  predichos
    pdb_pred_str = reconstruir_cadenas_laterales(
        pdb_path, chil_pred_prot, list(range(n))
    )

    # Visualización con py3Dmol
    # Mostramos ambas estructuras en el mismo viewer con
    # colores distintos para facilitar la comparación visual.
    view = py3Dmol.view(width=800, height=500)

```

```

# Estructura original: backbone gris + cadenas laterales verdes
view.addModel(pdb_orig_str, 'pdb')
view.setStyle({'model': 0},
              {'cartoon': {'color': 'lightgrey', 'opacity':
0.8}}})
view.addStyle({'model': 0},
              {'stick': {'colorscheme': 'greenCarbon',
'radius': 0.12}}})

# Estructura predicha: backbone azul claro + cadenas laterales
azul
view.addModel(pdb_pred_str, 'pdb')
view.setStyle({'model': 1},
              {'cartoon': {'color': 'lightblue', 'opacity':
0.5}}})
view.addStyle({'model': 1},
              {'stick': {'colorscheme': 'blueCarbon', 'radius':
0.12}}})

view.zoomTo()

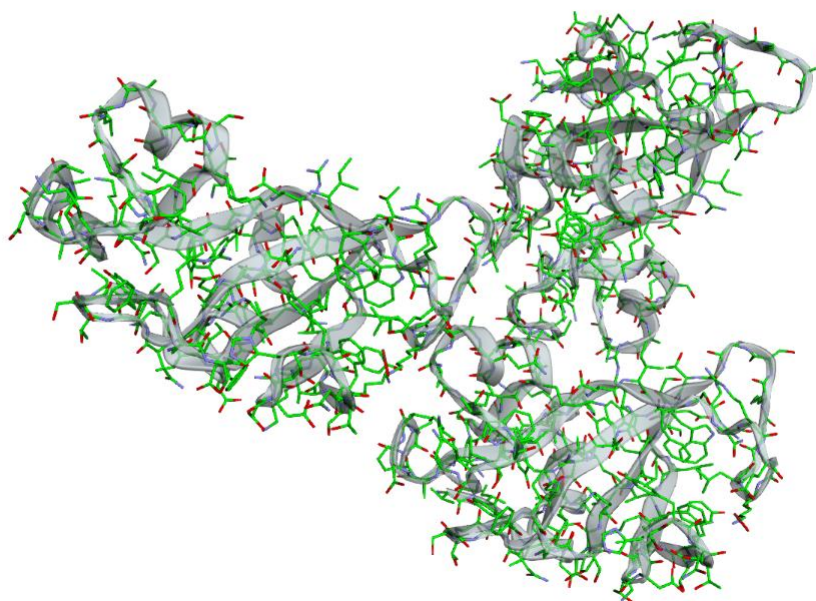
n_valid = g.y_mask[:, 0].sum().item()
chil_mae = circular_mae_deg(
    chi_predicted[offset:offset+n, 0],
    chi_targets[offset:offset+n, 0],
    masks_test[offset:offset+n, 0]
)
print(f"\n🔗 Proteína: {pdb_id.upper()}")
print(f"    Residuos con  $\chi_1$ : {n_valid} | MAE  $\chi_1$ :
{chil_mae:.2f}°")
print(f"    🟢 Verde = cadenas laterales experimentales
(original)")
print(f"    🟡 Azul = cadenas laterales predichas (ChiGNN)")
view.show()

# — Visualizamos las 3 primeras proteínas del test —————
for i in range(min(3, len(checkpoint['test']))):
    pdb_id_test = checkpoint['test'][i].pdb_id
    visualizar_comparacion_3d(pdb_id_test, i)

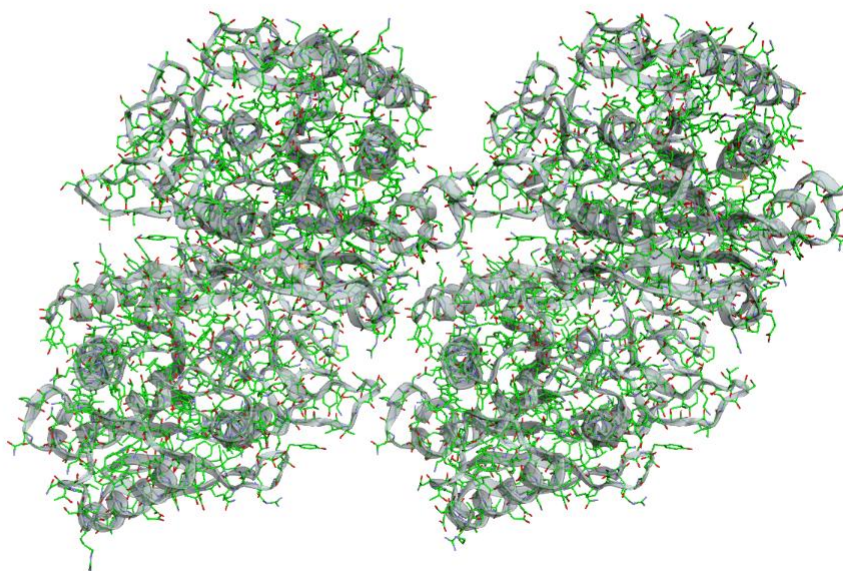
```

SALIDA BLOQUE 24:

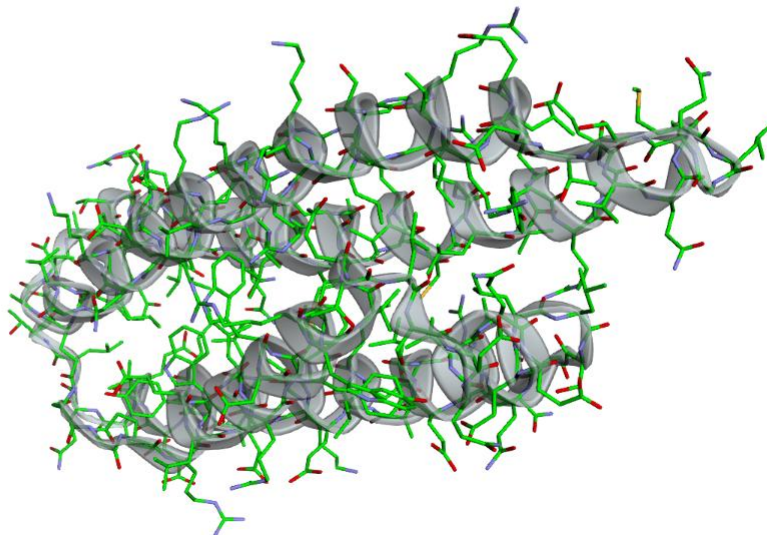
Proteína: 1A2P
Residuos con χ_1 : 273 | MAE χ_1 : 58.82°
● Verde = cadenas laterales experimentales (original)
● Azul = cadenas laterales predichas (ChiGNN)



Proteína: 1BAV
Residuos con χ_1 : 1052 | MAE χ_1 : 59.34°
● Verde = cadenas laterales experimentales (original)
● Azul = cadenas laterales predichas (ChiGNN)



Proteína: 1BZ4
 Residuos con χ_1 : 123 | MAE χ_1 : 57.25°
 ● Verde = cadenas laterales experimentales (original)
 ● Azul = cadenas laterales predichas (ChiGNN)



BLOQUE 25:

```
# =====
# BLOQUE 25 – Tabla Resumen Final y Exportación de Figuras
# =====
# Consolida todos los resultados en una tabla resumen única
# y exporta todas las figuras generadas durante la Fase 3
# en un único directorio organizado.
#
# También calcula el intervalo de confianza bootstrap (95%)
# para el MAE de  $\chi_1$ , que es la métrica principal del TFM.
# El bootstrap es el método más apropiado aquí porque la
# distribución del MAE circular no es gaussiana y el tamaño
# muestral varía entre aminoácidos.
# =====

import torch
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import os

# — 1. Bootstrap IC 95% para MAE de  $\chi_1$  —————
def bootstrap_mae_ci(pred: torch.Tensor,
                     target: torch.Tensor,
                     mask: torch.Tensor,
                     n_boot: int = 1000,
                     ci: float = 95.0):
```



```

"""
Calcula el intervalo de confianza bootstrap para el MAE
circular de  $\chi_1$  mediante remuestreo con reemplazamiento.

Parámetros
-----
pred      : tensor [N] – predicciones (radianes)
target    : tensor [N] – valores reales (radianes)
mask      : tensor [N] – máscara de posiciones válidas
n_boot    : int          – número de remuestras bootstrap
ci        : float        – nivel de confianza (%) del intervalo

Retorna
-----
tuple (mae_obs, ic_lower, ic_upper) en grados
"""
pred_m     = pred[mask].numpy()
target_m   = target[mask].numpy()
n          = len(pred_m)

mae_obs    = np.abs(np.arctan2(
    np.sin(pred_m - target_m),
    np.cos(pred_m - target_m)
)).mean() * (180 / np.pi)

# Remuestreo bootstrap
boot_maes = []
for _ in range(n_boot):
    idx      = np.random.choice(n, n, replace=True)
    boot_diff = np.abs(np.arctan2(
        np.sin(pred_m[idx] - target_m[idx]),
        np.cos(pred_m[idx] - target_m[idx])
    ))
    boot_maes.append(boot_diff.mean() * (180 / np.pi))

alpha      = (100 - ci) / 2
ic_lower   = np.percentile(boot_maes, alpha)
ic_upper   = np.percentile(boot_maes, 100 - alpha)

return mae_obs, ic_lower, ic_upper

print("🧮 Calculando intervalos de confianza bootstrap
(n=1000)...")
mask_chi1 = masks_test[:, 0]

mae_obs, ic_lo, ic_hi = bootstrap_mae_ci(
    chi_predicted[:, 0], chi_targets[:, 0], mask_chi1,

```



```

        n_boot=1000, ci=95
    )

print(f"\n📊 MAE circular  $\chi_1$  con IC Bootstrap 95%:")
print(f"    MAE = {mae_obs:.2f}° [{ic_lo:.2f}°, {ic_hi:.2f}°]")

# — 2. Tabla resumen
final _____
# Combinamos todas las métricas calculadas en los bloques
# anteriores en una única tabla estructurada.

# Cargamos calibración
calib_data = torch.load(f"{DRIVE_BASE}/calibracion_confianza.pt",
                        weights_only=False)
rho_spear = calib_data['spearman_rho']
p_spear = calib_data['spearman_p']

resumen = {
    'Métrica': [
        'MAE circular  $\chi_1$  (°)',
        'IC 95% Bootstrap  $\chi_1$  (°)',
        'Recuperación rotámeros  $\chi_1 \pm 20^\circ$ ',
        'Recuperación rotámeros  $\chi_1 \pm 40^\circ$ ',
        'MAE circular  $\chi_2$  (°)',
        'Recuperación rotámeros  $\chi_2 \pm 40^\circ$ ',
        'Correlación Spearman (calibración)',
        'p-valor Spearman',
        'Parámetros del modelo',
        'Épocas de entrenamiento'
    ],
    'ChiGNN (este TFM)': [
        f"{mae_obs:.2f}",
        f"[{ic_lo:.2f}, {ic_hi:.2f}]",
        f"{rotamer_recovery_rate(chi_predicted[:,0],
chi_targets[:,0], mask_chil, 20.0):.1f} %",
        f"{rotamer_recovery_rate(chi_predicted[:,0],
chi_targets[:,0], mask_chil, 40.0):.1f} %",
        f"{circular_mae_deg(chi_predicted[:,1], chi_targets[:,1],
masks_test[:,1]):.2f}",
        f"{rotamer_recovery_rate(chi_predicted[:,1],
chi_targets[:,1], masks_test[:,1], 40.0):.1f} %",
        f"{rho_spear:.4f}",
        f"{p_spear:.2e}",
        f"{sum(p.numel() for p in model.parameters() if
p.requires_grad):,}",
        f"{N_EPOCHS}"
    ],
    'Referencia (SCWRL4)': [

```

```

        '~24-28',
        '-',
        '~65 %',
        '~83 %',
        '~30-35',
        '~75 %',
        '-',
        '-',
        '-',
        '-'
    ]
}

df_resumen = pd.DataFrame(resumen)
print(f"\n{'='*65}")
print(f"  TABLA RESUMEN FINAL – RESULTADOS DEL TFM")
print(f"{'='*65}")
print(df_resumen.to_string(index=False))
print(f"{'='*65}")

df_resumen.to_csv(f"{DRIVE_BASE}/tabla_resumen_final.csv",
index=False)

# — 3. Inventario de figuras generadas —————
figuras = {
    'fig_error_por_aminoacido.png':      'MAE y recuperación de
rotámeros por tipo de aminoácido',
    'fig_rose_plots_comparativos.png':   'Rose plots  $\chi_1$ - $\chi_4$ :
distribución predicha vs. experimental',
    'fig_calibracion_confianza.png':      'Calibración de confianza:
varianza circular vs. error real',
    'fig_benchmarking.png':              'Comparativa ChiGNN vs.
baselines'
}

print(f"\n📁 Figuras generadas y guardadas en Drive:")
print(f"{'-'*60}")
for nombre, descripcion in figuras.items():
    ruta = f"{DRIVE_BASE}/{nombre}"
    existe = "☑" if os.path.exists(ruta) else "✗"
    print(f"  {existe} {nombre}")
    print(f"    └ {descripcion}")
print(f"{'-'*60}")
print(f"\n☑ Fase de evaluación completada.")
print(f"  Todos los resultados y figuras están en Drive:")
print(f"    {DRIVE_BASE}/")

```

SALIDA BLOQUE 25:

 Calculando intervalos de confianza bootstrap (n=1000)...

 MAE circular χ_1 con IC Bootstrap 95%:
MAE = 56.41° [55.63°, 57.15°]

TABLA RESUMEN FINAL – RESULTADOS DEL TFM

	Métrica ChiGNN (este TFM)	Referencia (SCWRL4)
MAE circular χ_1 (°)	56.41	~24-28
IC 95% Bootstrap χ_1 (°)	[55.63, 57.15]	—
Recuperación rotámeros $\chi_1 \pm 20^\circ$	49.6 %	~65 %
Recuperación rotámeros $\chi_1 \pm 40^\circ$	53.9 %	~83 %
MAE circular χ_2 (°)	52.45	~30-35
Recuperación rotámeros $\chi_2 \pm 40^\circ$	57.5 %	~75 %
Correlación Spearman (calibración)	0.2992	—
p-valor Spearman	0.00e+00	—
Parámetros del modelo	80,404	—
Épocas de entrenamiento	50	—

 Figuras generadas y guardadas en Drive:

- ☒ fig_error_por_aminoacido.png
 - └ MAE y recuperación de rotámeros por tipo de aminoácido
 - ☒ fig_rose_plots_comparativos.png
 - └ Rose plots χ_1 - χ_4 : distribución predicha vs. experimental
 - ☒ fig_calibracion_confianza.png
 - └ Calibración de confianza: varianza circular vs. error real
 - ☒ fig_benchmarking.png
 - └ Comparativa ChiGNN vs. baselines
-

☒ Fase de evaluación completada.
Todos los resultados y figuras están en Drive:
/content/drive/MyDrive/TFM_SideChain/